Paper 1 / Computer Systems

OCR H446/01 / Specification order

Choose a topic. Page numbers match the PDF viewer.

1.1 / Processors, input, output and storage

1.1.1 / Processor structure and architecture **2**

1.1.2 / Types of processor **20**

1.1.3 / Input, output and storage **28**

1.2 / Software and software development

1.2.1 / Systems software and operating systems **36**

1.2.2 / Applications generation **74**

1.2.3 / Software development **85**

1.2.4 / Types of programming language **94**

1.3 / Exchanging data

1.3.1 / Compression, encryption and hashing **112**

1.3.2 / Databases **121**

1.3.3 / Networks **151**

1.3.4 / Web technologies **164**

1.4 / Data types, data structures and algorithms

1.4.1 / Data types and representation **171**

1.4.2 / Data structures **188**

1.4.3 / Boolean algebra and logic hardware **224**

1.5 / Legal, moral, cultural and ethical issues

1.5.1–1.5.2 / Legislation and ethical argument **246**

Mixed revision

Mixed memory banks **257**

Worked examples, practice and answers stay within each topic.

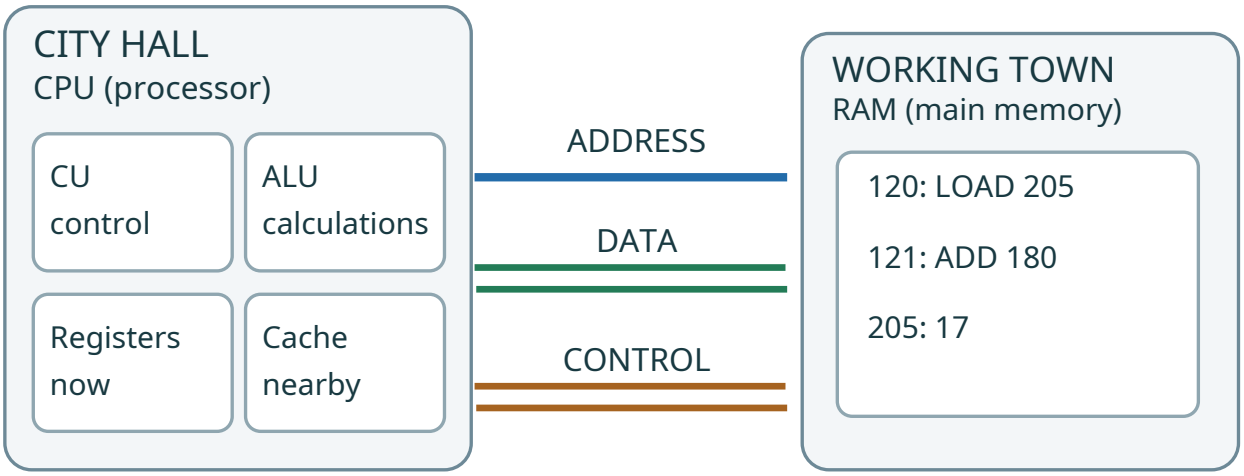
THE CPU HIGHWAY

H446 1.1.1 | Road networks, register transfers and processor architecture

REMEMBER THIS / THE COMPUTER IS A CONNECTED COUNTRY

The CPU is the city control centre; RAM is the busy working town; cache is a small fast depot inside the CPU; and registers are the immediate workspaces within the centre. Buses are the communication highways.

THE CPU HIGHWAY: CONNECTED COMPONENTS



ARCHIVES: secondary storage (SSD/HDD) is connected through I/O hardware



COLOUR KEY Blue = address Green = data Orange = control signals Grey = I/O link

Translate the picture into exam terminology

Memory image	Exam-accurate meaning
City Hall / CPU	The processor executes instructions; the CU coordinates control, and the ALU performs arithmetic/logic.
Cache depot / registers	Cache keeps frequently accessed instructions or data; registers hold current addresses, instructions and results.
Working town / RAM	RAM stores the instructions and data of currently running programs.
Three highways / buses	Address carries location, data carries content, control carries command and timing signals.
Archives / SSD or HDD	Non-volatile secondary storage is accessed using I/O hardware, not as if it were a RAM address.

NOW BE PRECISE

A bus is a set of electrical signal lines, not a literal road. Instructions and data are patterns of bits. This map is a reminder about where information goes, not a literal representation of CPU wiring.

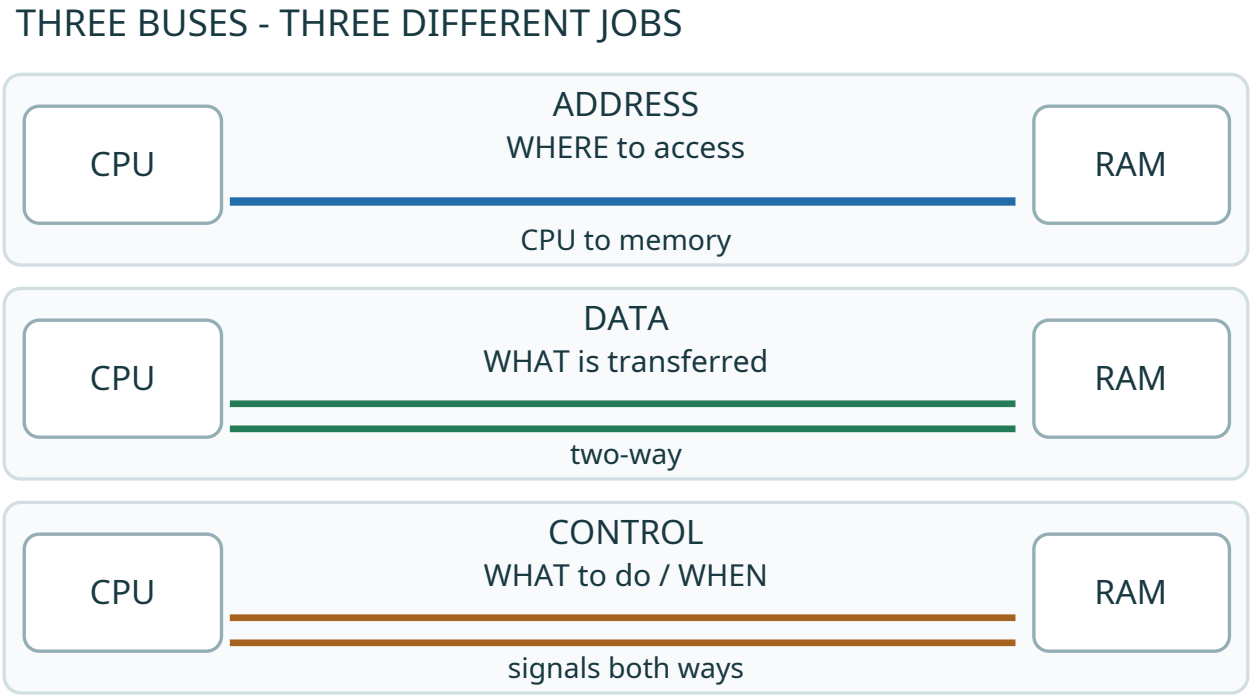
Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

ALU	Arithmetic Logic Unit; performs arithmetic and logical operations.
Control unit (CU)	Decodes instructions and coordinates processor operations.
Program counter (PC)	Holds the address of the next instruction to fetch.
MAR	Memory Address Register; holds the memory address being accessed.
MDR	Memory Data Register; holds data or instructions transferred to or from memory.
CIR	Current Instruction Register; holds the fetched instruction being decoded or executed.
Cache	Small, fast memory holding frequently used data and instructions.
Bus	Carries addresses, data or control signals between components.

Three highways, three different jobs

Address identifies the location. Data is the content. Control coordinates the operation.



Blue = address Green = data Orange = control signals

Bus	Direction (simplified model)	Exact function
Address	CPU → memory (one way)	Carries address of memory or I/O location accessed.
Data	CPU ↔ memory (two ways)	Carries data or instructions between CPU and memory.
Control	Signals in both directions	READ, WRITE, READY and other coordination/timing signals.

REMEMBER: A-D-C

Address = where; Data = what; Control = what action/when. A-dress is not the actual package; an address gives its location.

A READ from RAM location 205

Step	Which pathway?	What happens
1	Address	MAR is loaded with 205 and the address is signalled to memory.
2	Control	Control unit coordinates a memory READ operation.
3	Data	The contents stored in RAM[205] travel back on the data bus into the MDR.

COMMON MISCONCEPTION

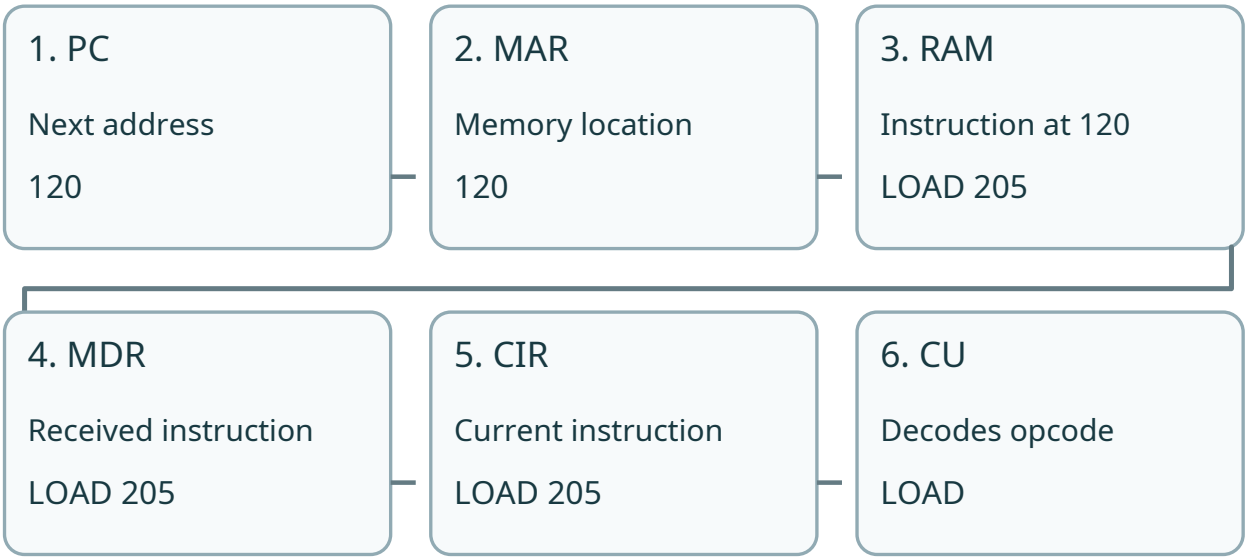
A 12-bit address bus can distinguish 2^{12} = 4,096 locations; a 16-bit data bus may transfer 16 data bits per bus transfer. Address width does not mean 12 bits are fetched, and bus widths alone do not determine CPU speed.

Quick retrieval: Which bus carries the memory address? Which carries the READ signal? Why does the data bus need both directions?

Inside City Hall: five registers and FDE

Keep the highway picture, but name the real components and show how values change.

FETCH - DECODE - EXECUTE: FOLLOW ONE INSTRUCTION



EXECUTE | PC 121 Operand RAM[205] MDR = 17 ACC 17

Teaching model: a branch may change PC instead of sequentially advancing.

Register	What it stores	Memory hook
PC	Address of the next instruction to fetch.	Next stop.
MAR	Address of the memory location being accessed.	Street number.
MDR	Data or instruction being transferred to/from memory.	What came from that address.
CIR	The current instruction being decoded/executed.	Work order.
ACC	Result/intermediate value from ALU work in this teaching model.	Result counter.

Fetch–Decode–Execute: exam wording

Fetch: PC value is copied into MAR. The address bus identifies that memory location; a READ is requested. The instruction returns on the data bus into MDR, then is copied to CIR. PC ordinarily advances. Decode: the CU interprets the opcode and any operand. Execute: the required operation takes place; this may fetch an operand, use the ALU, update a register or change PC for a branch.

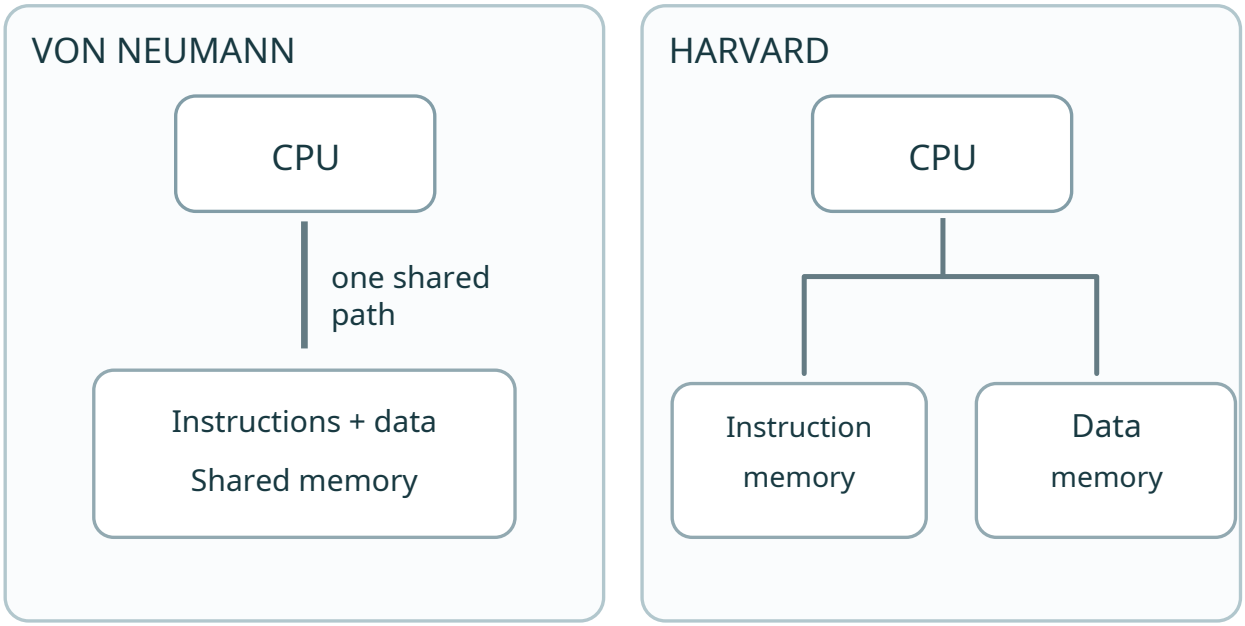
TWO COMPLEMENTARY ANALOGIES

Highway: where addresses, instructions and data travel. Kitchen/chef: what fetch, decode and execute mean. Use both only for the job they explain accurately.

Shared highways, parallel work and performance

Von Neumann vs Harvard; pipelining; clock, cache and cores.

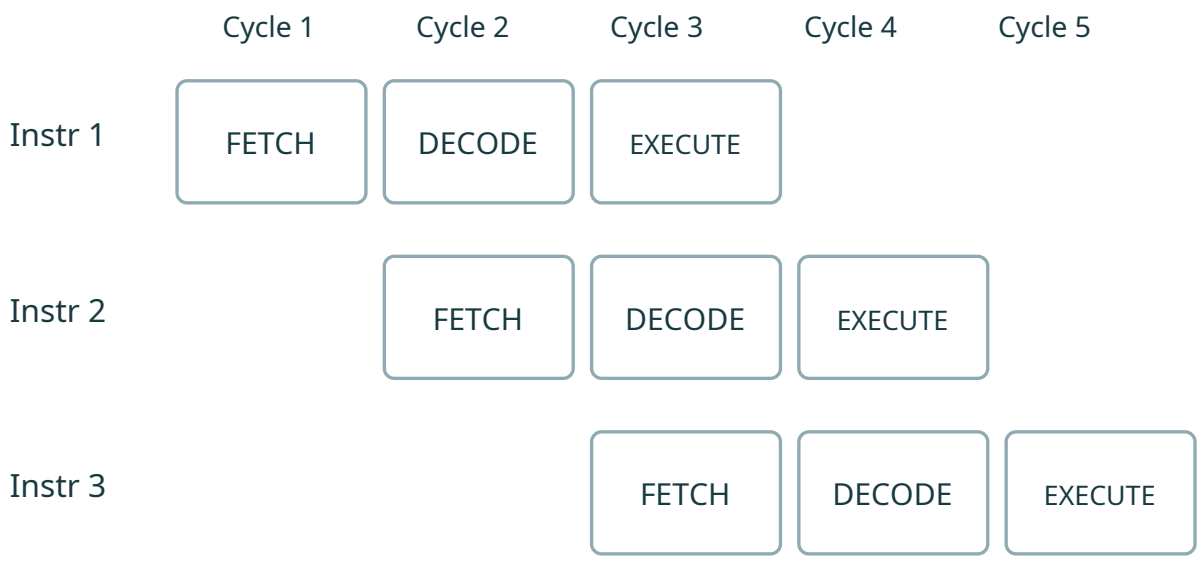
VON NEUMANN VS HARVARD



Shared transfers may contend; separate routes can overlap.

Von Neumann: shared main memory and path for program instructions and data; competing transfers can limit throughput (the Von Neumann bottleneck). Harvard: separate instruction and data memories/buses, which can permit simultaneous access. Real contemporary processors may mix design features; the diagrams are simplified models.

Pipelining and processor performance



Overlapping stages can improve throughput; stalls and hazards can interrupt flow.

Pipelining overlaps stages of different instructions. It aims to increase throughput, not necessarily shorten one instruction. Dependencies, branches and resource conflicts can introduce stalls.

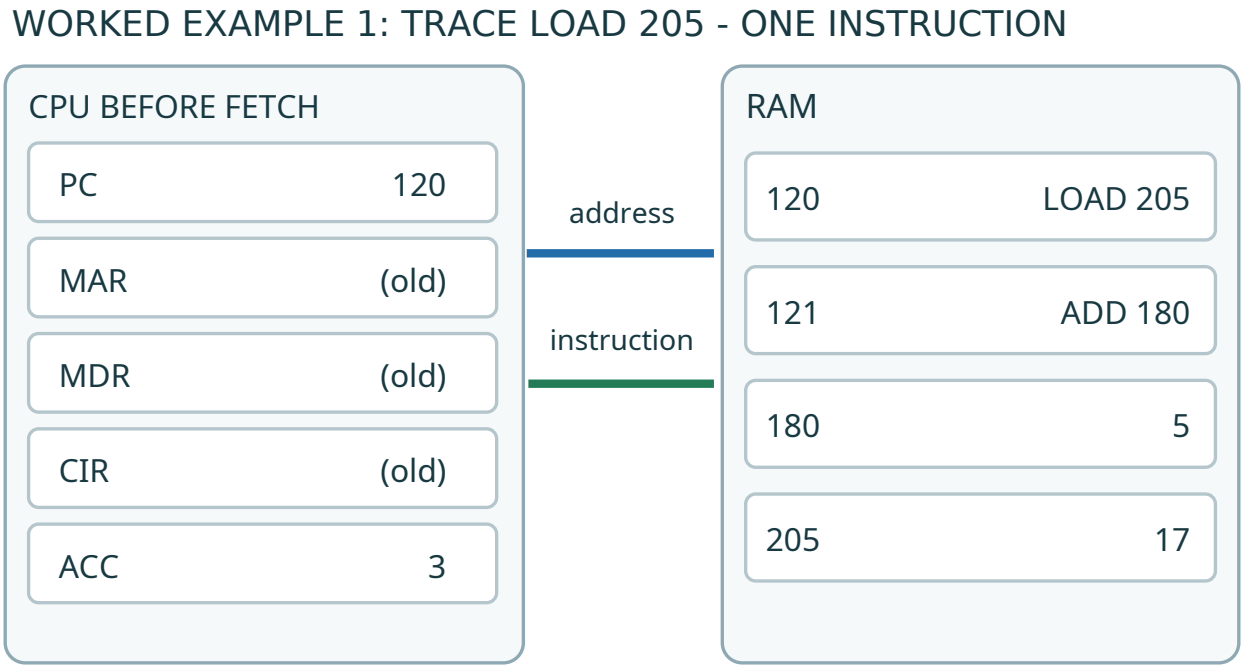
Factor	Precise explanation / limits
Clock speed	Clock cycles per second; higher clock rate alone does not guarantee more instructions per second.
Cache	Frequently used instructions/data can be accessed with lower average latency; hit rate matters.
Cores	Multiple cores can execute tasks in parallel where the software is designed for it.

REMEMBER

More traffic lanes may increase potential bandwidth; better nearby depots reduce waiting; parallel work crews help independent jobs. But these are analogies, not interchangeable definitions.

WORKED EXAMPLE 1 / trace an assembly instruction

Read the example below as a sequence of register values, not as a journey of physical cars.



After execution: CIR = LOAD 205; MAR = 205; MDR = 17; ACC = 17; PC = 121.

Blue = address Green = data/instruction Numbers are example values.

Starting state: PC = 120, ACC = 3; RAM[120] = LOAD 205; RAM[205] = 17. Assume a simple accumulator-based teaching CPU, with an ordinary sequential instruction.

Phase	What happens	Resulting state
Fetch	Copy PC to MAR; signal address 120; READ retrieves RAM[120] into MDR; copy instruction to CIR.	MAR = 120; MDR = LOAD 205; CIR = LOAD 205; PC = 121
Decode	CU recognises opcode LOAD, operand address 205.	CIR retains LOAD 205
Execute	Read RAM[205] via MAR/MDR and load its value into ACC.	MAR = 205; MDR = 17; ACC = 17; PC = 121

WORKED EXAMPLE 1 / MEMORY CHECK

120 and 205 are addresses. LOAD 205 is an instruction. 17 is data stored at location 205. PC normally points ahead after fetch; MAR and MDR may change again while executing.

WORKED EXAMPLE 2 / different question, same method

PC = 72, ACC = 7; RAM[72] = ADD 180 and RAM[180] = 5. Predict PC, CIR, MAR, MDR and ACC immediately after the instruction finishes.

SCAFFOLD BEFORE CHECKING

1. Fetch the instruction at 72. 2. Increase the PC to the next sequential address. 3. Decode ADD 180. 4. Fetch the value at address 180. 5. Add that value to the old ACC.

GUIDED CHECK / AFTER ATTEMPTING

PC = 73; CIR = ADD 180; MAR = 180; MDR = 5; ACC = 12. The instruction was at address 72, but the operand is at address 180. The ALU added 5 to the original accumulator 7.

Processor architecture · independent exam questions

Q1 [3 marks] State what the address bus, data bus and control bus each carry.

Q2 [4 marks] Explain how MAR and MDR differ, and state the separate purposes of PC and CIR.

Q3 [2 marks] A processor has a 12-bit address bus. Calculate the maximum number of distinct memory locations it can address. Show your working.

WORKING / METHOD

Processor architecture · independent exam questions

Q4 [4 marks] Explain why the data bus needs to carry signals in both directions, while the address bus in the simplified CPU–RAM model is one-way.

Q5 [4 marks] Compare Von Neumann and Harvard architectures, including a performance implication.

Processor architecture · extended responses

Q1 [6 marks] In a simple accumulator CPU, the PC contains 48. Memory location 48 stores ADD 93, memory location 93 stores 4, and ACC initially stores 9. Explain the fetch–decode–execute process, naming the registers and relevant buses, and state the resulting ACC and PC values.

Q2 [6 marks] A developer claims a CPU must be faster because it has more cores, a higher clock speed and a larger cache. Discuss why those features may improve performance, and why they do not guarantee it for every program.

Processor architecture · extended responses

Q3 [3 marks] Explain how pipelining can improve processor throughput and identify one condition that can cause a pipeline stall.

MODEL ANSWERS / Questions 1–5

Indicative mark points for the original questions in this chapter (not an OCR-issued mark scheme).

Q1 [3]

MARK POINTS

Address bus carries memory location addresses [1]; data bus carries data and/or instructions [1]; control bus carries control or timing signals [1].

Q2 [4]

MARK POINTS

MAR holds the address being accessed [1]; MDR holds the data or instruction being transferred [1]; PC holds the next instruction address [1]; CIR holds the current instruction [1].

Q3 [2]

MARK POINTS

An n-bit address bus provides 2^n distinct address patterns [1]. 2^{12} = 4,096 locations [1]. Do not infer a number of bytes unless location size is given.

Q4 [4]

MARK POINTS

A READ moves data/instructions from RAM towards the CPU [1]; a WRITE moves data from CPU towards RAM [1]. The CPU specifies which memory address is accessed [1], so address signals in this simplified architecture flow from processor to memory [1].

Q5 [4]

MARK POINTS

Von Neumann uses a shared instruction/data memory [1] and shared transfer path [1]; Harvard uses separate instruction and data memory/pathways [1], potentially allowing instruction and data access concurrently, avoiding some shared-path contention [1]. Accept valid equivalent comparisons.

EXAM WRITING

Do not write “the address bus carries data to RAM”. It carries addresses. Use words such as register, instruction, control signal, memory location and shared transfer path.

MODEL ANSWERS / Questions 6–8

For extended answers, award only distinct correct points that address the question.

Q6 [6] Fetch, decode, execute an ADD

INDICATIVE SIX MARK POINTS

[1] PC 48 is copied into MAR. [2] Address bus carries 48; a READ control signal requests the instruction. [3] RAM[48] places ADD 93 on the data bus into MDR; instruction copies into CIR. [4] PC becomes 49 for normal sequential execution. [5] CU decodes ADD and the operand address 93; CPU reads RAM[93] = 4 (via MAR/MDR). [6] ALU adds 4 to ACC 9, leaving ACC = 13. Equivalent well-sequenced accounts gain credit.

Q7 [6] Clock, cache, cores

INDICATIVE SIX MARK POINTS

[1] Higher clock speed means more cycles per second. [2] More cycles can help performance, but work per clock cycle varies. [3] Larger/effective cache reduces slower memory access for frequently used content. [4] Improvement depends on cache hits and memory-access pattern. [5] More cores can execute independent work in parallel. [6] A largely sequential program, contention or synchronisation overhead can limit the gain. Other supported trade-offs are creditworthy.

Q8 [3] Pipelining

INDICATIVE THREE MARK POINTS

Different instructions can occupy different stages at once [1], allowing a greater number of instructions to be completed per unit time after the pipeline fills [1]. A data dependency, branch or structural conflict may stall/flush the pipeline [1].

THE ANALOGY STOPS HERE

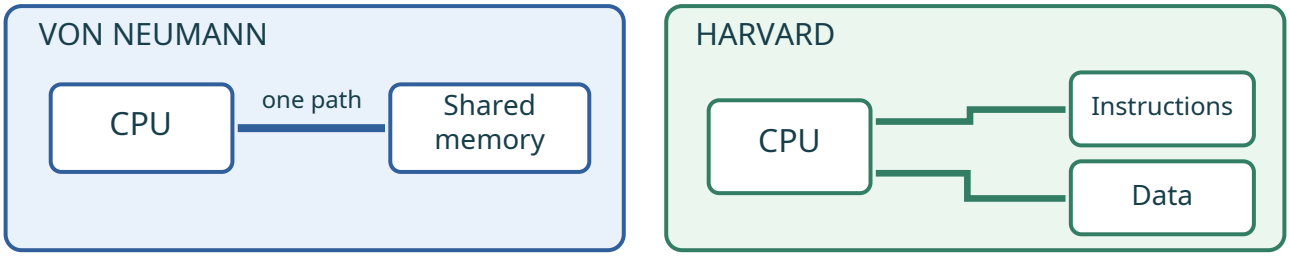
The highway metaphor explains communication and potential bottlenecks. It does not replace electronic signals, real instruction encodings, clock-cycle timing, or the actual architecture of contemporary chips. For exam answers, use the technical column—not the road names.

Processor architectures: compare the routes

Theory / Worked Examples

A processor needs instructions and data. Architecture determines how these are stored and accessed; performance depends on the actual design and workload.

Compare two memory pathways



Blue: one shared route. Green: distinct instruction and data routes.

REMEMBER THIS

Two roads into City Hall. Von Neumann has shared instruction/data memory and a shared access pathway. Harvard has separate instruction and data memories/pathways. Modern CPUs may have split instruction/data caches yet a shared main memory.

Now be precise

- **Von Neumann:** instructions and data are stored in the same addressable memory; accesses may compete for bandwidth (the Von Neumann bottleneck).
- **Harvard:** separate instruction and data storage and access routes can allow both to be fetched concurrently, subject to implementation.
- **Modified Harvard:** a processor can have separate L1 instruction and data caches and share further levels of memory. Do not claim every modern CPU is purely Harvard.

WORKED EXAMPLE 1

A CPU fetches an instruction while a previous instruction needs a data item.

Method: On a single shared memory path, requests may compete; separate instruction/data paths reduce that specific conflict, though neither guarantees faster execution.

WORKED EXAMPLE 2

A microcontroller stores fixed firmware instructions separately from changing sensor measurements.

Method: A Harvard-style arrangement is appropriate to describe the separated code/data access. Explain the distinct routes rather than relying on the words “fast” or “slow”.

COMMON MISTAKE

Separate caches do not automatically imply separate physical main memory.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 2 marks. Explain one potential bottleneck of a shared instruction/data pathway.

Q2 · 4 marks. Compare Harvard and Von Neumann arrangements using instructions and data.

Q3 · 4 marks. Explain why a contemporary CPU may not fit a simple pure-Harvard or pure-Von-Neumann label.

Mark scheme

Compare only after completing the questions.

Q1 · 2 marks

A shared access route can be needed for instruction and data fetches [1], creating contention/delays when both are requested [1].

Q2 · 4 marks

Von Neumann stores instructions and data together [1] using a shared path [1]; Harvard separates stores [1] and pathways [1].

Q3 · 4 marks

Separate instruction/data caches [1] can coexist with shared higher-level memory [1]; the organisation is hybrid/modified Harvard [1]; real speed depends on implementation and workload [1].

RECALL

Two roads into City Hall: Von Neumann has shared instruction/data memory and a shared access pathway. Harvard has separate instruction and data memories/pathways. Modern CPUs may have split instruction/data caches yet a shared main memory.

Memory bank / processor structure

City Hall	CPU executes instructions; CU coordinates and ALU calculates.
Nearby depot	Cache stores frequently accessed data/instructions.
Next address	PC holds the next instruction address; MAR holds the address accessed.
Package / work order	MDR holds transferred data/instructions; CIR holds the current instruction.
Calculation desk	ACC holds arithmetic/logic results in a simplified accumulator CPU.
Highway signs	Address bus: location. Data bus: contents. Control bus: control signals.
One common route	Von Neumann shares instruction/data memory and access paths.
Two separate routes	Harvard has separate instruction/data memories and access paths.
Assembly line	Pipelining overlaps stages of different instructions.
Speed / depot / teams	Clock speed, cache and parallelism affect performance.

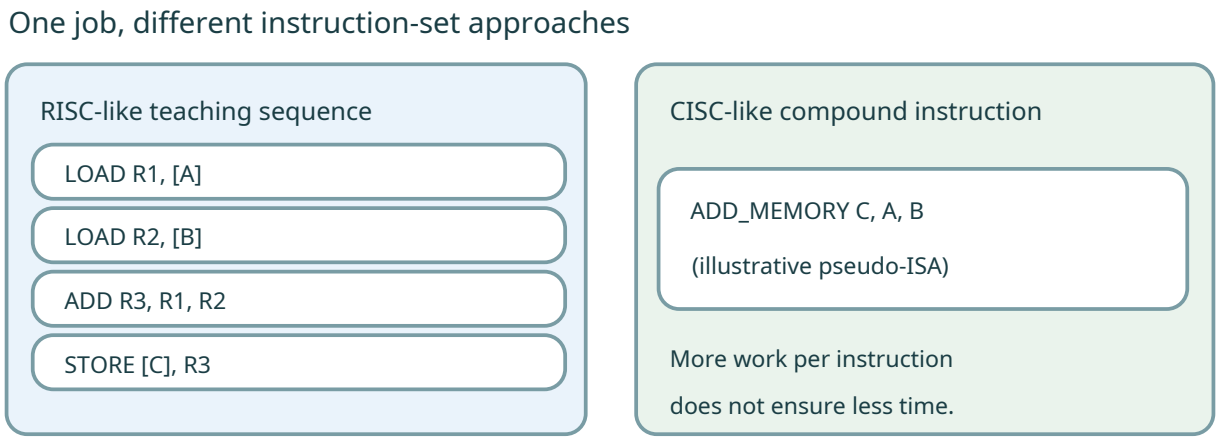
Five retrieval checks

1. Which buses are involved in reading an instruction from RAM?
2. What are the different roles of MAR and MDR?
3. Why may extra CPU cores not speed up a sequential program?
4. What does pipelining improve, and what can cause a stall?
5. Where does the highway analogy stop being technically accurate?

1.1.2 / PROCESSOR TYPES

REMEMBER THIS / DIFFERENT ROUTES, SAME DESTINATION

On the CPU highway, RISC takes several simple, regular delivery steps; CISC can encode more work in a single instruction. Neither route is automatically faster.



Now be precise: instruction sets

RISC (typical)	CISC (typical)
Simpler, regular operations; load/store designs often separate memory and arithmetic.	Richer, sometimes more complex instructions; often variable-length encodings.
May support efficient decoding and pipelining, but more instructions can increase program size.	May offer compact code, while decoding and internal execution may be complex.

MISCONCEPTION TO AVOID

RISC does not always execute more quickly, and instruction count alone does not measure performance. The teaching commands in the diagram are not real machine instructions.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

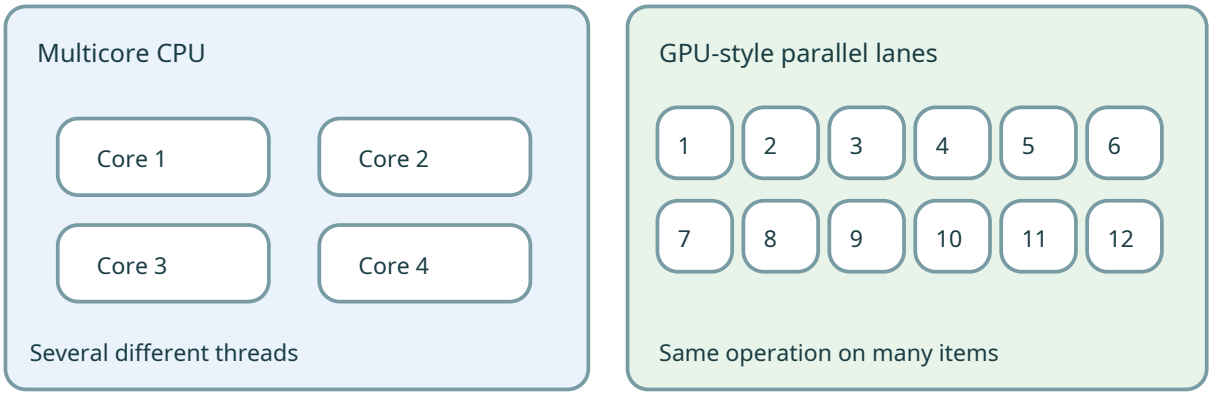
RISC	Reduced Instruction Set Computer: a simpler, generally regular instruction-set approach.
CISC	Complex Instruction Set Computer: an instruction set with richer operations.
Pipelining	Overlapping instruction execution stages to improve throughput.
Multicore	Processor design with multiple processing cores on one chip.
Parallel processing	Performing more than one operation at a time where dependencies allow.
SIMD	Single Instruction, Multiple Data: one instruction operates on several values.

1.1.2 / MULTICORE PROCESSORS AND GPUs

MEMORY HOOK / MANY CHECKOUTS OR MANY IDENTICAL PARCELS

Multicore CPUs can handle separate threads; GPUs excel at applying similar operations to large numbers of independent data elements. The analogy ends where synchronisation, branching and transfers begin.

Match the processor to the shape of the job



WORKED EXAMPLE 1 / Dividing independent work

12 equal image tiles across 4 CPU cores: 12 ÷ 4 = 3 tiles per core in an ideal workload. With equal independent work and no overhead, completion may approach a quarter of the serial time. Real performance depends on memory access, communication, scheduling and any serial work.

WORKED EXAMPLE 2 / fresh problem

A graphics task has 30 independent tiles and 5 CPU worker cores. How many tiles per core? Which two practical limits could stop the task achieving a fivefold speed-up?

Method	Expected reasoning
Divide independent work	30 ÷ 5 = 6 tiles per core.
Evaluate practical limits	Examples: serial stages, synchronisation, memory bandwidth, data transfer, uneven workloads.

NOW BE PRECISE

A multicore CPU has several processor cores. A GPU has many execution resources suited to data-parallel calculations, such as graphics operations or matrix processing; it is not automatically faster for sequential tasks.

REVISION SUMMARY / PROCESSOR TYPES

Use the hook to remember the idea, then replace the metaphor with exact H446 terminology.

Keyword	Exam-accurate explanation
RISC	Reduced Instruction Set Computer: an ISA approach using relatively simple regular instructions.
CISC	Complex Instruction Set Computer: an ISA approach permitting more complex operations.
Opcode	The operation specified by the instruction.
Code density	How much program functionality is encoded in a given amount of machine code.
Multicore CPU	Processor with multiple cores that can execute work concurrently.
GPU	Processor adapted for massive data parallelism on suitable workloads.
Synchronisation	Coordination between concurrent tasks sharing data or resources.
Bandwidth	Amount of information transferable per unit time.

QUICK SELF-TEST

1. Why can fewer instructions still take longer? 2. When does extra parallel hardware bring little benefit?
3. What does data-parallel mean?

Processor types · RISC, CISC, CPU and GPU

Q1 [4 marks] State two typical characteristics of RISC designs and two typical characteristics of CISC designs.

Q2 [6 marks] Compare the illustrative LOAD, LOAD, ADD, STORE sequence with a compound memory-add instruction. Explain why instruction count alone does not establish which executes more quickly.

Processor types · RISC, CISC, CPU and GPU

Q3 [6 marks] Evaluate the claim that a RISC processor must always be faster than a CISC processor. Refer to instruction decoding, pipelining and code density.

Q4 [4 marks] Give one graphics and one non-graphics use of a GPU, and explain what makes each suitable for data-parallel execution.

Processor types · RISC, CISC, CPU and GPU

Q5 [6 marks] A workload has 24 equal independent chunks and six CPU cores. Calculate the ideal chunks per core and explain two limits to real speed-up.

WORKING / METHOD

Q6 [6 marks] A video pipeline contains a serial input stage and an independent colour correction for every frame. Explain which parts may benefit from GPU acceleration and why speed-up is not unlimited.

Model answers / processor types

Award one mark for each distinct valid point. Credit equivalent correct phrasing; do not reward a point twice.

Q1 / 4 marks

[1] RISC typically uses a smaller or more regular instruction set.
[1] RISC commonly follows load/store organisation (arithmetic on registers).
[1] CISC may provide more complex, multi-operation instructions.
[1] CISC often supports richer addressing modes or variable-length instructions.

Q2 / 6 marks

[1] First LOAD obtains one input value from memory.
[1] Second LOAD obtains the other input value.
[1] ADD carries out the arithmetic on available operands.
[1] STORE writes the result to memory.
[1] A compound memory-add instruction can express more work in one instruction.
[1] Instruction count alone ignores cycles, decoding, memory access and pipeline behaviour.

Q3 / 6 marks

[1] Regular/simpler instruction formats can simplify decoding.
[1] This regularity can support efficient pipelining.
[1] A RISC sequence may require more instructions for the same task.
[1] That can increase code size and/or instruction-fetch work.
[1] CISC may improve code density with richer individual instructions.
[1] Actual performance depends on implementation and workload, not the ISA label alone.

Q4 / 4 marks

[1] Graphics example: image rendering or applying a filter to many pixels.
[1] Many pixels can undergo similar independent calculations concurrently.
[1] Non-graphics example: machine-learning matrix operations.
[1] Independent matrix elements/operations can be processed across parallel GPU units.

Q5 / 6 marks

[1] Calculate 24 / 6 = 4 chunks per core.
[1] With equal independent chunks, ideal distribution is four to each core.
[1] Some sequential stages of a workload cannot be parallelised.
[1] Those sequential stages limit overall acceleration even with more cores.
[1] Synchronisation, communication or task creation can introduce overhead.
[1] Memory bandwidth contention, transfers or uneven work can further reduce speed-up.

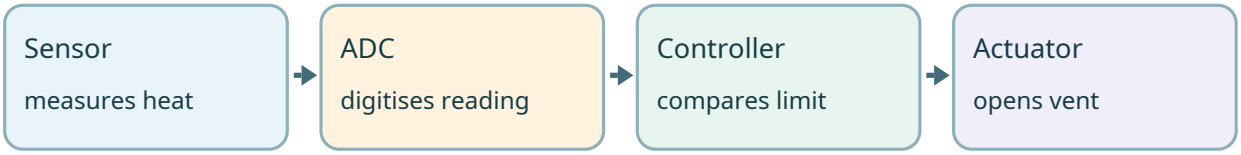
Q6 / 6 marks

[1] Independent colour correction of frames or pixels can run across GPU units.
[1] The repeated similar operations make the stage suitable for data parallelism.
[1] The stated serial input stage remains a constraint.
[1] Downstream processing may have to wait for an earlier stage.
[1] Transferring data and coordinating work introduces overhead.
[1] Memory bandwidth, GPU occupancy or reassembly can impose further limits.

1.1.3 Input, output and storage

Remember the control room: SENSE → SAMPLE → DECIDE → ACT. Sensors collect measurements; software processes them; actuators change the physical world. An ADC converts an analogue signal into digital samples when needed.

From the physical world to a controlled action



Only use an ADC when the sensor signal needs analogue-to-digital conversion.

SENSE > SAMPLE > DECIDE > ACT

NOW BE PRECISE

An input device supplies data to a computer. An output device presents information or controls a physical process. A sensor is an input transducer; an actuator creates an action. An ADC is needed only where the incoming signal is analogue and digital processing is required.

WORKED EXAMPLE 1 / Automatic door

Choose a suitable input and output for an automatic door. A proximity sensor detects an approaching person; the controller evaluates the reading, and a motor actuator opens the door. Safety sensing must also check when it is safe to close.

WORKED EXAMPLE 2 / Pedestrian crossing

A crossing must work for people with different accessibility needs. Identify one input and two output methods.

Guided explanation

Use an accessible push button as input. Signal lights give visual information; audible and/or tactile outputs assist users with different needs. The control system must obey safe timing rules.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

Input device	Supplies data or commands to a computer system.
Output device	Presents information or controls an external action.
Sensor	Input transducer that measures a physical property.
Actuator	Output device that causes a physical action.
ADC	Analogue-to-Digital Converter; represents an analogue signal digitally.
DAC	Digital-to-Analogue Converter; produces an analogue signal from digital values.
Secondary storage	Non-volatile storage for data and programs between sessions.

1.1.3 Storage technologies

Remember MAGNET, LASER, ELECTRONS: magnetic, optical and solid-state flash store bits using different physical mechanisms.

Three technologies, three physical methods

HDD	DVD	SSD
MAGNETIC	OPTICAL	FLASH
Spinning platters	Laser reads disc	Electronic cells

Non-volatile does not mean the same speed, capacity, durability or cost.

EXAM PRECISION

A magnetic hard disk uses regions on spinning platters. Optical discs are read using a laser. Flash memory stores data electronically in non-volatile cells. Compare real requirements: capacity, transfer speed, random-access latency, portability, durability and cost per GB.

WORKED EXAMPLE 1 / Photographer

A photographer transfers large image files between venues and edits them frequently. An SSD often offers faster random access and better resistance to mechanical shock than an HDD. An HDD can offer lower cost per GB for bulk capacity; neither is a replacement for backups.

WORKED EXAMPLE 2 / Offline exhibition

A school wants to distribute a fixed multimedia exhibition without relying on internet access. Would optical storage ever be sensible?

Guided explanation

Yes, when the computers have compatible optical drives and the files need little updating. Disadvantages include drive availability, capacity and updating/reuse limitations.

1.1.3 Main memory and virtual storage

Four memory and storage terms students confuse

RAM Volatile working data	ROM / firmware Non-volatile startup code
Virtual storage Remote files over a network	Virtual memory OS-managed address space

A common examination trap is mixing up virtual storage (files held on network-connected systems) with virtual memory (an operating-system memory-management technique using address translation and paging).

REMEMBER

RAM = what the computer is using now. ROM/firmware = startup instructions retained without power. Virtual storage = your files stored elsewhere. Virtual memory = how the OS manages a process's address space.

WORKED EXAMPLE 1 / Power cut

Why might an unsaved document be lost when power fails but firmware survives? The unsaved changes may exist only in volatile RAM; firmware remains in non-volatile storage.

WORKED EXAMPLE 2 / Online coursework

A student edits saved work at home using an online school drive. Give an advantage and a risk.

Guided explanation

Files may be accessible across multiple devices. Network connectivity, privacy/security and mistakenly synchronised deletions are potential risks.

EXAM PRECISION / VIRTUAL MEMORY

Virtual memory gives each process a managed virtual address space. The OS maps virtual addresses to physical memory; paging may move pages to/from secondary storage when required.

Exam practice / Input, output and storage

Q1 [4 marks] A school greenhouse opens a vent when it becomes too hot. Identify a suitable sensor and actuator, and explain the role of each.

Q2 [6 marks] Explain the pathway from an analogue temperature reading to a motor-driven action. Include the function of the ADC and the controller.

Q3 [6 marks] Compare an SSD with an HDD for a travelling photographer. Explain two benefits of the SSD and one potential disadvantage.

Q4 [4 marks] Distinguish RAM from ROM/firmware by giving one typical purpose and one volatility characteristic of each.

Q5 [6 marks] Distinguish virtual storage from virtual memory. Explain one limitation of each.

Model answers and marking points

Use these after attempting the questions. Equivalent technically accurate answers may earn credit.

Q1 [4 marks]

Indicative marking points

Temperature sensor measures ambient heat [2]. Motor/vent actuator produces movement that opens the vent [2].

Q2 [6 marks]

Indicative marking points

Analogue sensor produces a continuous measurement [1]; ADC samples/converts it to digital form [1]; controller receives the data [1], compares it to the threshold [1]; if the limit is exceeded it drives the motor [1], moving/opening the vent [1].

Q3 [6 marks]

Indicative marking points

Low seek latency / faster random access for frequent editing [2]; no mechanical read head means greater shock resistance [2]; often a higher cost per GB than HDD [2].

Q4 [4 marks]

Indicative marking points

RAM holds active programs/data [1] and is volatile [1]. ROM/firmware stores startup or low-level instructions [1] and is non-volatile [1].

Q5 [6 marks]

Indicative marking points

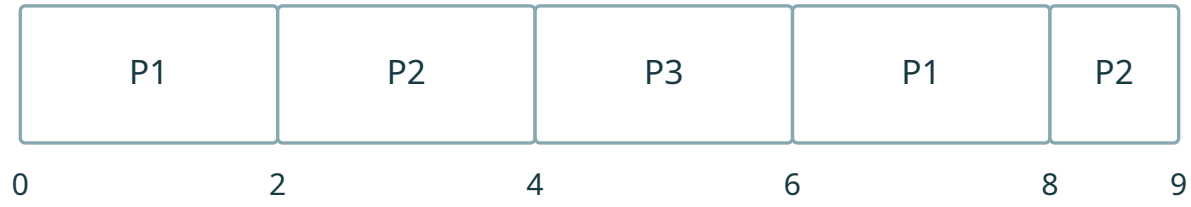
Virtual storage is remote/network-hosted files [2], potentially affected by connectivity or security [1]. Virtual memory is OS-managed virtual address space supported by paging and secondary storage [2]; heavy paging can lower performance [1].

1.2.1 / Operating systems and scheduling

Remember: The OS is an airport controller, not the aeroplane.

ROUND ROBIN: TIME QUANTUM = 2

All three processes arrive at time 0.



At t = 2, P1 is pre-empted with 2 units remaining.

ANALOGY - PICTURE IT

The controller allocates a runway to each plane; an operating system allocates processor time and other resources to processes.

NOW BE PRECISE - WHERE THE ANALOGY STOPS

Aircraft operate physically and schedules vary. In computing, process scheduling follows a defined algorithm; the CPU executes the instructions.

THEORY / KEY TERMINOLOGY

Process scheduling	Choosing which ready process receives CPU time.
Round robin (RR)	Each ready process runs for a fixed time quantum in turn.
Pre-emptive	A running process may be interrupted and placed back in the ready queue.
Paging	Fixed-size pages and frames; page tables map logical to physical memory.
Segmentation	Variable-sized logical sections of a program.
ISR	Interrupt Service Routine; code that responds to an interrupt.

Operating systems and scheduling / understand and model

The OS manages processor time, memory, files, devices and user access. First Come First Served (FCFS) follows arrival order; Shortest Job First chooses the shortest available burst; Shortest Remaining Time may pre-empt; RR rotates a fixed quantum; multi-level feedback queues move jobs between priority queues. Real-time systems prioritise meeting deadlines, not merely high average throughput. Device drivers interface with hardware; firmware/BIOS supports initialisation; virtual machines provide software-based machine environments.

WORKED EXAMPLE 1 / worked example

Scenario: all arrive at time 0; P1 needs 4 units, P2 needs 3, P3 needs 2; RR quantum = 2.

0–2: P1 (2 left). 2–4: P2 (1 left). 4–6: P3 completes.

6–8: P1 completes. 8–9: P2 completes.

Check: total occupied CPU time = 4 + 3 + 2 = 9 units.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

Use round robin with P1=3, P2=2, both arriving at 0 and a quantum of 1. Draw the Gantt chart and give both completion times.

Scaffold: Write one time unit per box. An unfinished process goes to the back of the ready queue.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

Round robin: Quantum = 1, assuming no context-switch overhead.
Ready queue: An incomplete task returns to the queue.
Completion: P1 ends at t=5; P2 ends at t=4.

Operating systems and scheduling · independent exam practice

Q1 [4 marks] Explain the difference between paging and segmentation.

Q2 [3 marks] Why can a pre-emptive scheduler improve response time for interactive users?

Q3 [2 marks] State one use each of a driver and an interrupt service routine.

Operating systems and scheduling · independent exam practice

Q4 [3 marks] Challenge: explain why a system with strict deadlines might use a real-time operating system.

Model answers / operating systems

Use the points below to self-mark the independent practice on the preceding pages.

Q1 / 4 marks

[1] Paging divides a process address space into fixed-size pages.
[1] Pages map to equal-size physical frames via a page table.
[1] Segmentation divides the program into variable-size logical regions.
[1] Segments correspond to logical units such as code, data or stack; they require their own mapping/bounds.

Q2 / 3 marks

[1] A pre-emptive scheduler may interrupt a task that is currently running.
[1] It can assign the CPU to a ready interactive task before the long task finishes.
[1] This can reduce the time a user waits for an input to be acted upon.

Q3 / 2 marks

[1] A device driver provides the OS/software with a device-specific interface or commands.
[1] An interrupt service routine handles the event represented by an interrupt request.

Q4 / 3 marks

[1] A real-time OS is designed to meet specified timing deadlines.
[1] It aims for predictable or bounded worst-case response times.
[1] A late response can be unsafe or unusable even when the result is otherwise logically correct.

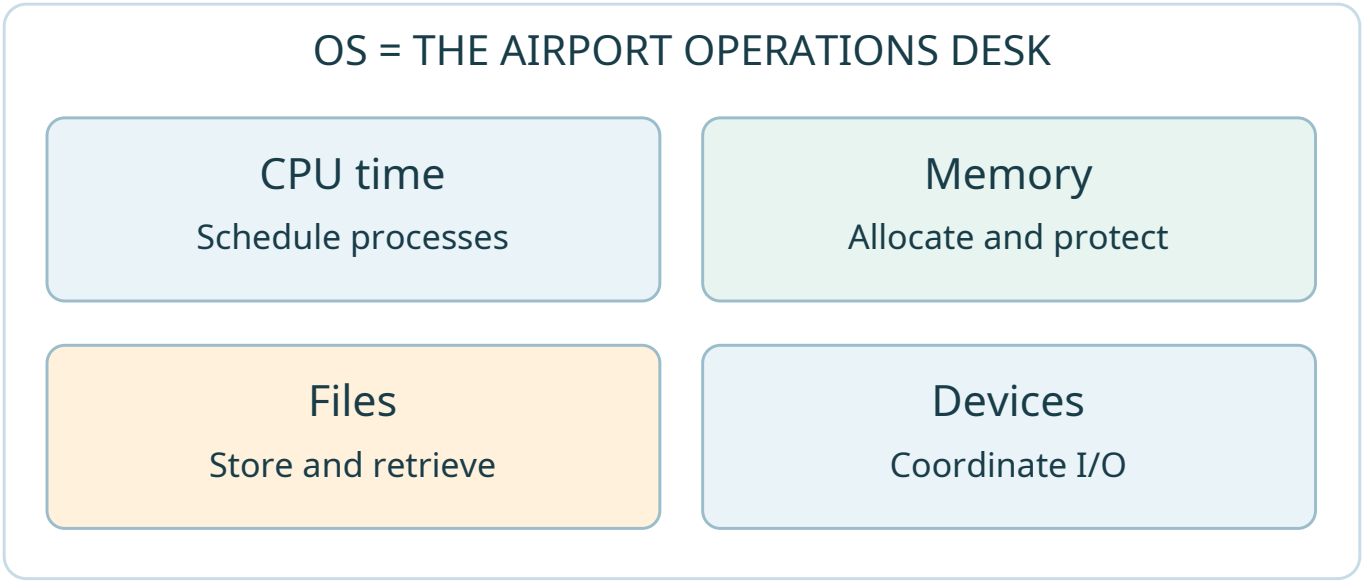
Remember

A fast average response does not guarantee that every real-time deadline is met. Context switching has an overhead; it is not itself why pre-emption helps interactive response.

Operating systems: purpose and functions

REMEMBER THIS / THE AIRPORT CONTROL DESK

An operating system is like the airport operations team: it coordinates aircraft movements, gates, staff and communications. It does not do the passengers' jobs. Similarly, an OS manages resources and services while applications carry out users' tasks.



Now be precise

OS responsibility	What the OS actually does
Processor management	Schedules ready processes and allocates CPU time.
Memory management	Allocates and protects address spaces; manages paging.
File management	Organises access to files, directories and permissions.
Device management	Coordinates I/O, drivers, buffering and requests.
Security & interface	Enforces user permissions and provides command-line or graphical services.

Important limit of the analogy: the OS manages hardware and supplies services, but hardware executes instructions and applications perform user-facing work.

OS functions: follow a real workload

WORKED EXAMPLE 1 / browser, backup and printer

A pupil writes in a browser while a backup utility copies files and a print job is sent to the shared printer. Identify the OS responsibility at each point.

Observed event	OS response
Browser and backup are both active	Scheduler switches CPU time among runnable threads/processes.
Both need working memory	OS allocates/protects memory and may use virtual memory.
Report saved to disk	File system locates files; OS coordinates storage I/O.
Print job submitted	OS print service and a printer driver communicate with the device.

EXAM PRECISION

Multitasking means more than one task makes progress. One CPU core may switch rapidly between tasks; this does not prove both instructions are executed literally at the same instant.

WORKED EXAMPLE 2 / same responsibilities, different problem

During a timed online assessment, a candidate opens a saved program, receives a keyboard event and another user signs in to the same server. Which OS functions are involved?

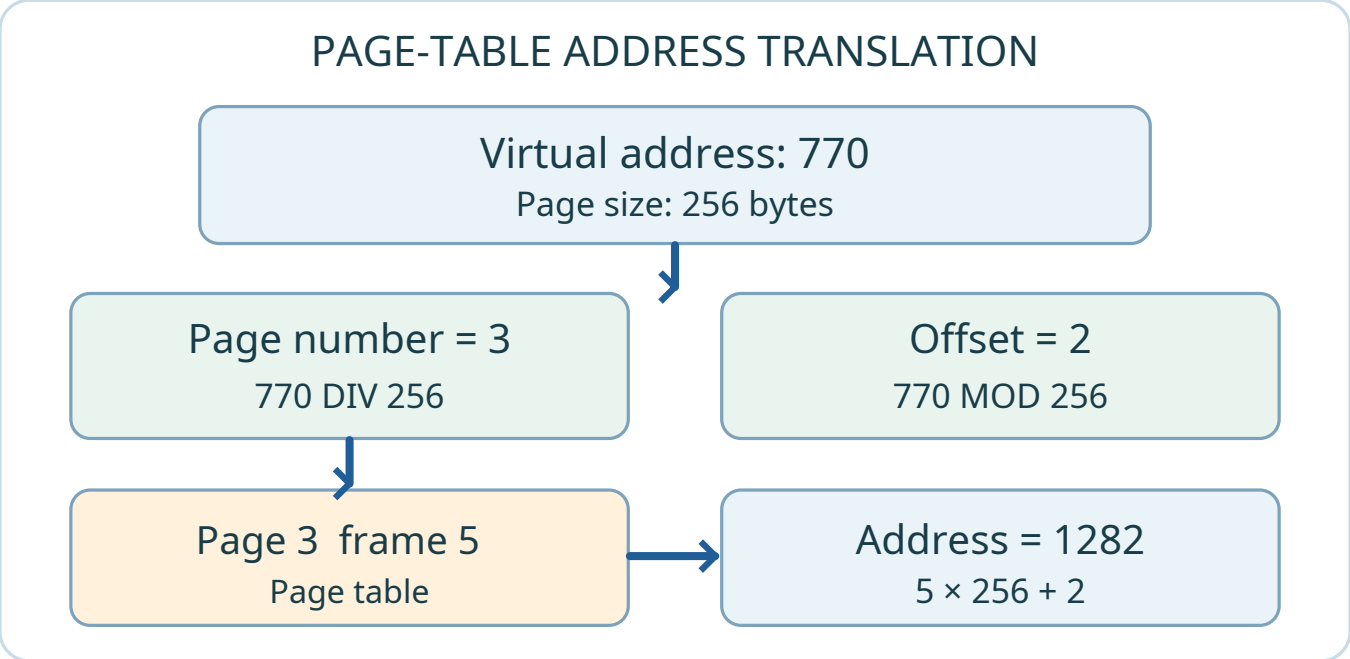
Guided check: file management retrieves the program; device/interrupt handling responds to the keyboard; account authentication and process/memory isolation support the separate user. Explain each function, rather than only naming it.

Paging, frames and virtual memory

REMEMBER THIS / A HOTEL WITH MOVABLE ROOM KEYS

A virtual page is like a numbered booking; a physical frame is the actual room. The page table says which room holds the booking. Unlike a hotel, addresses are translated electronically and the offset does not change.

WORKED EXAMPLE 1 / translate a virtual address



Page size = 256 bytes. The virtual address is 770, and virtual page 3 currently maps to RAM frame 5. Divide: 770 DIV 256 = 3; 770 MOD 256 = 2. The physical address is (5 × 256) + 2 = 1282.

WORKED EXAMPLE 2 / a different translation

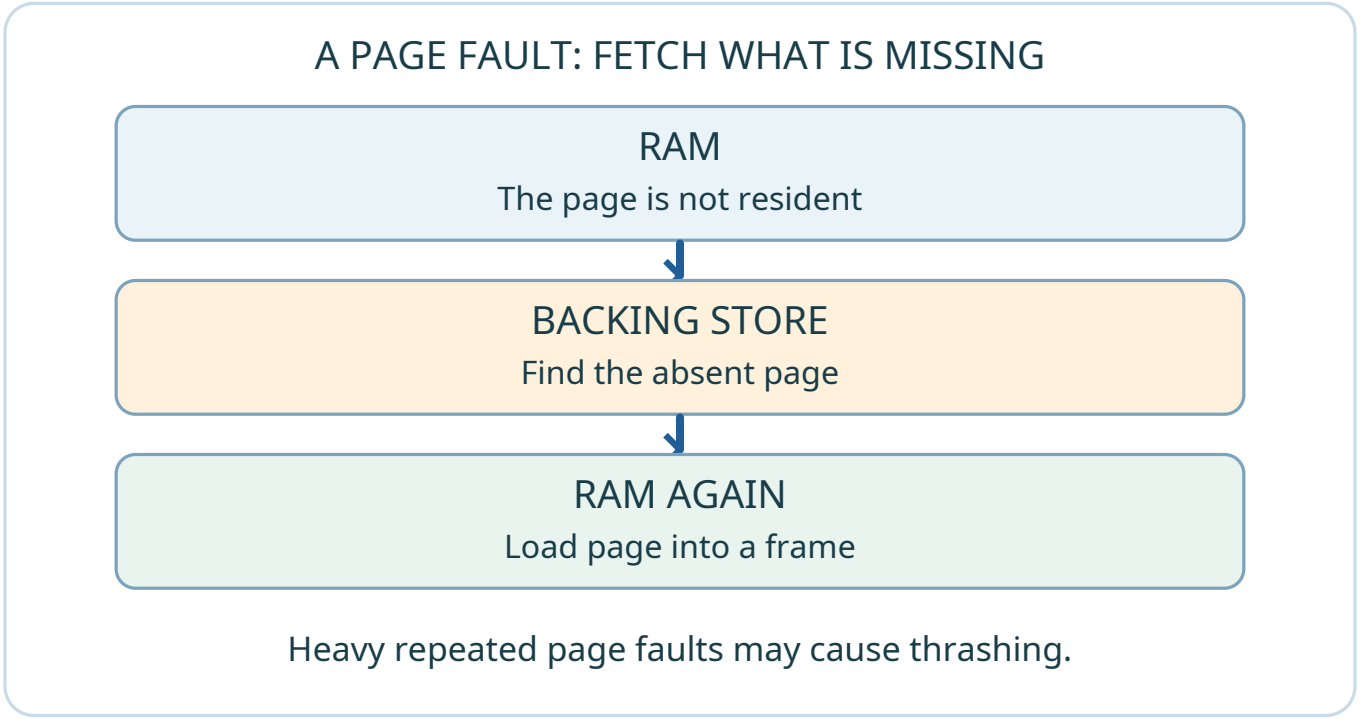
Page size = 128 bytes. Virtual address = 390. Page 3 maps to frame 7. What physical address is accessed?

STEP-BY-STEP

390 DIV 128 = 3 and 390 MOD 128 = 6. Keep the offset 6. Physical address = (7 × 128) + 6 = 902.

Vocabulary: A page is a fixed-size virtual block; a frame is a same-size physical RAM block. A page table maps them; the offset is unchanged.

Virtual memory and segmentation



What happens when a page is absent?

A page fault occurs when a process references a page not currently resident in RAM. The operating system obtains the page from backing storage, may evict another page, updates the mapping and resumes the interrupted instruction when the requested page is available. This involves slower storage I/O.

MEMORY HOOK / FIXED TILES VS FLEXIBLE ROOMS

Paging uses equal-sized blocks; segmentation uses variable-sized logical regions, such as code, stack and data. The metaphor helps recall size and purpose, but real protection and address translation are handled by hardware and the OS.

Compare: Paging uses fixed-size pages/frames and can cause internal fragmentation. Segmentation uses variable-size logical regions and can cause external fragmentation. Virtual memory keeps some pages outside RAM, but repeated page faults can cause thrashing.

Important distinction: Virtual memory is an OS memory-management mechanism; virtual storage means accessing files kept remotely, such as a cloud drive. They are not synonyms.

Exam tip

For a question on why virtual memory helps, explain what is moved, why RAM is freed, and the performance cost. Do not say it makes secondary storage as fast as RAM.

Memory management / exam practice

Complete the questions independently. Use diagrams, tables or working where the question requires them.

Q1 · 3 marks A machine uses pages of 128 bytes. State the page number and offset for virtual address 390. [Show working.]

Q2 · 3 marks Virtual page 3 is held in physical frame 7. Using 128-byte frames, calculate the physical address corresponding to virtual address 390.

Q3 · 4 marks Explain two differences between paging and segmentation, including a possible form of fragmentation.

Memory management / exam practice / continued

Q4 · 6 marks Explain how virtual memory allows another application to run when RAM is nearly full, and why performance might become poor.

Memory management / marking points

Check each question only after completing the independent practice.

Q1 / 3 marks

- 390 DIV 128 = 3 [1].
- 390 MOD 128 = 6 [1].
- Page = 3; offset = 6 [1].

Q2 / 3 marks

- Use frame 7 [1].
- 7 × 128 = 896 [1].
- Add the offset 6 to obtain physical address 902 [1].

Q3 / 4 marks

- Paging uses equal-sized pages/frames [1]; segmentation uses variable-size logical regions [1].
- Paging can have unused space inside allocated fixed blocks (internal fragmentation) [1].
- Segmentation can leave scattered free gaps (external fragmentation) [1].

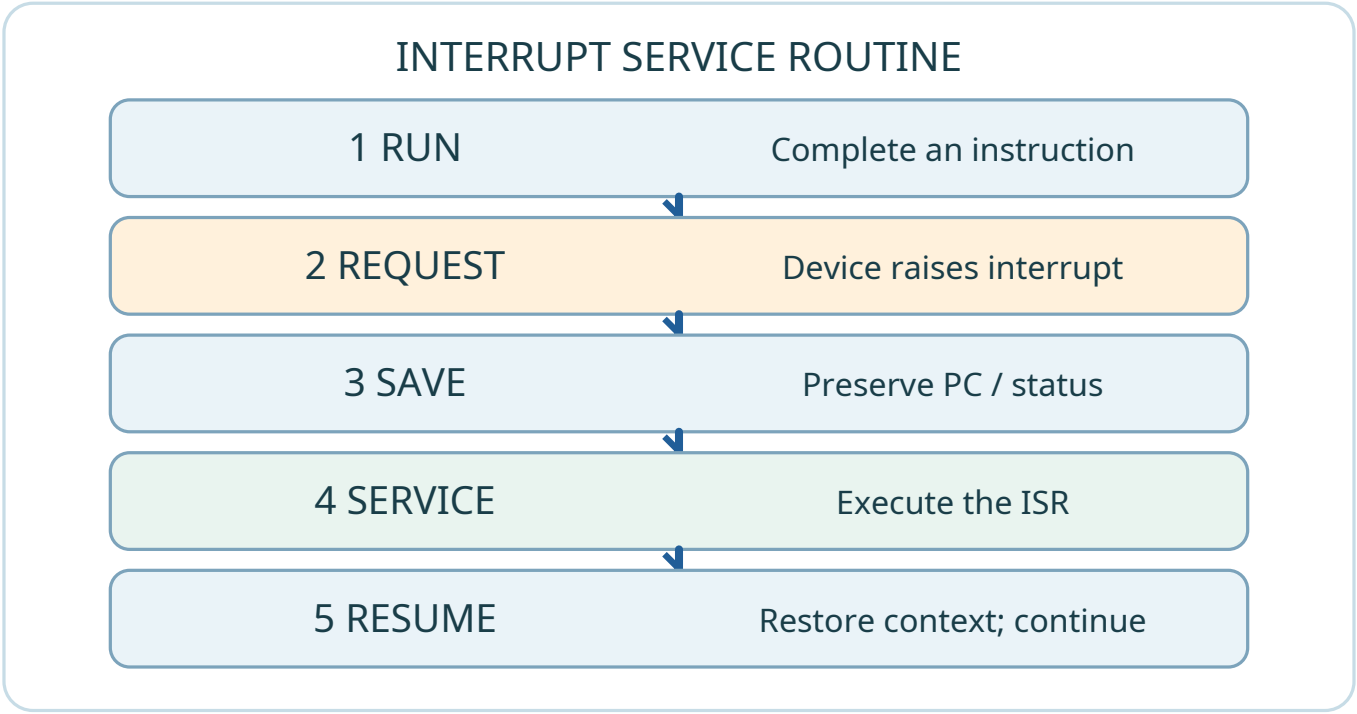
Q4 / 6 marks

- Pages not currently needed can be moved out of RAM to a backing store [1].
- This frees physical frames for the new application [1].
- When an absent page is needed, a page fault is handled by the OS [1].
- The page is loaded into RAM and page-table information updated [1].
- Storage access is much slower than RAM access [1].
- Repeated page faults and swapping can produce thrashing / poor responsiveness [1].

Interrupts, ISRs and program state

REMEMBER THIS / THE FIRE ALARM

An interrupt is like an urgent alarm during a meeting. The CPU completes an appropriate instruction boundary, preserves what it needs, deals with the event and returns. It does not normally abandon the original task.



WORKED EXAMPLE 1 / key pressed during a running program

1. A keyboard controller requests attention. 2. At a suitable instruction boundary, the CPU responds according to interrupt priority/masking. 3. The processor saves the required context (such as PC/status/registers). 4. Control transfers to the relevant ISR, which handles the event and acknowledges it. 5. The saved context is restored and the interrupted work resumes.

WORKED EXAMPLE 2 / disk read completes

A file-reading program is waiting for data. A storage controller signals completion. Identify the event, ISR action and what happens next.

GUIDED RESPONSE

Event: a device-completion interrupt. ISR: records/acknowledges completion and allows the OS to act on the result. Afterwards: restore any interrupted CPU context; the scheduler decides which ready process runs next.

Link to FDE: In the simplified model, interrupts are recognised between instructions. Preserving the PC and necessary registers lets the next fetch resume correctly.

Interrupts / exam practice

Complete the questions independently. Use diagrams, tables or working where the question requires them.

Q1 · 2 marks State what an interrupt request and an interrupt service routine (ISR) are.

Q2 · 4 marks Describe the sequence from an input device raising an interrupt to the original program resuming.

Q3 · 6 marks Explain why the CPU must preserve its program counter and other necessary context during an interrupt, and relate this to the fetch–decode–execute cycle.

Interrupts / marking points

Check each question only after completing the independent practice.

Q1 / 2 marks

• Interrupt request: a signal that an event needs CPU/OS attention [1].

• ISR: code that services the interrupting event [1].

Q2 / 4 marks

• Device raises request [1].

• CPU recognises/responds at an appropriate point [1].

• Required program context saved and ISR executed [1].

• State restored and interrupted execution can resume [1].

Q3 / 6 marks

• CPU may have been executing another program when interrupted [1].

• PC stores the next instruction address (or architecturally appropriate saved point) [1].

• Status/register values needed for correct execution are preserved [1].

• ISR processes/acknowledges the event [1].

• Restored PC allows execution to resume at the appropriate instruction [1].

• Therefore fetch/decode/execute continues correctly rather than losing the original execution position [1].

1.2.1 / Operating-system scheduling

Five approaches and why performance differs

REMEMBER THIS A CPU is one service desk with different rules for who gets served next.

ROUND ROBIN: TIME QUANTUM = 2



A = 4 units B = 3 units C = 2 units

Order: A, B, C, A, B (all arrive at t = 0)

NOW BE PRECISE A scheduling chart is a model: specify arrival times, burst times, quantum and tie-breaking rules before comparing results.

KNOW IT PRECISELY

FCFS. First come, first served; generally non-preemptive.

SJF. Shortest available burst next; in its non-preemptive form the chosen process runs to completion.

SRTF. Shortest remaining time first; a newly arrived short job can pre-empt the current job.

Round Robin. Each ready process gets up to one time quantum, then goes to the back of the queue.

Multilevel feedback. Processes move between priority queues according to a defined policy, usually to favour interactive work.

Operating-system scheduling / Worked method

Worked examples and a different guided problem.

UNDERSTAND IT

- All arrive at t=0: A burst4, B burst3, C burst2. FCFS: A 0–4, B 4–7, C 7–9.
- SJF on the same data: C 0–2, B 2–5, A 5–9. Waiting times A=5, B=2, C=0; mean 7/3 ≈ 2.33.
- Round Robin quantum2: A 0–2, B 2–4, C 4–6, A 6–8, B 8–9. Waiting times A4, B6, C4; mean 14/3 ≈ 4.67.

WORKED EXAMPLE 1 / CALCULATE WAITING TIME

With all jobs available at t=0, waiting = completion time − burst time. For RR, A completes8 and burst4, so waits4; B waits9−3=6; C waits6−2=4.

WORKED EXAMPLE 1 / SRTF PRE-EMPTION

P arrives at 0 with burst6. Q arrives at t=2 with burst2; pre-empt P, because P has 4 remaining. With no other process, Q runs 2–4 and P resumes 4–8.

WORKED EXAMPLE 2 / OS BEYOND SCHEDULING

Paging maps virtual pages to physical frames; an interrupt saves execution context and transfers control to an ISR; a virtual machine presents a guest environment above a host/hypervisor.

Scheduling: pre-emption and feedback queues

Trace the mechanism carefully; these examples are not questions to memorise.

SRTF: A SHORT JOB CAN INTERRUPT A LONGER ONE



P: arrival 0, burst 6 Q: arrival 2, burst 2

TRACE A DEFINED MULTILEVEL-FEEDBACK POLICY

Assume queue Q0 has quantum 2, queue Q1 has quantum 4. New jobs enter Q0. Unfinished jobs from Q0 are demoted to Q1. Both A (burst 6) and B (burst 1) arrive at t=0, with A ahead of B.

Time	Running	State after interval
0–2	A at Q0	A has 4 remaining; demote A to Q1.
2–3	B at Q0	B completes; Q0 is now empty.
3–7	A at Q1	A completes after its four remaining units.

COMPARE A short interactive job gets attention through the higher-priority queue, but the exact outcome depends on the policy supplied in the question.

Scheduling: five algorithms

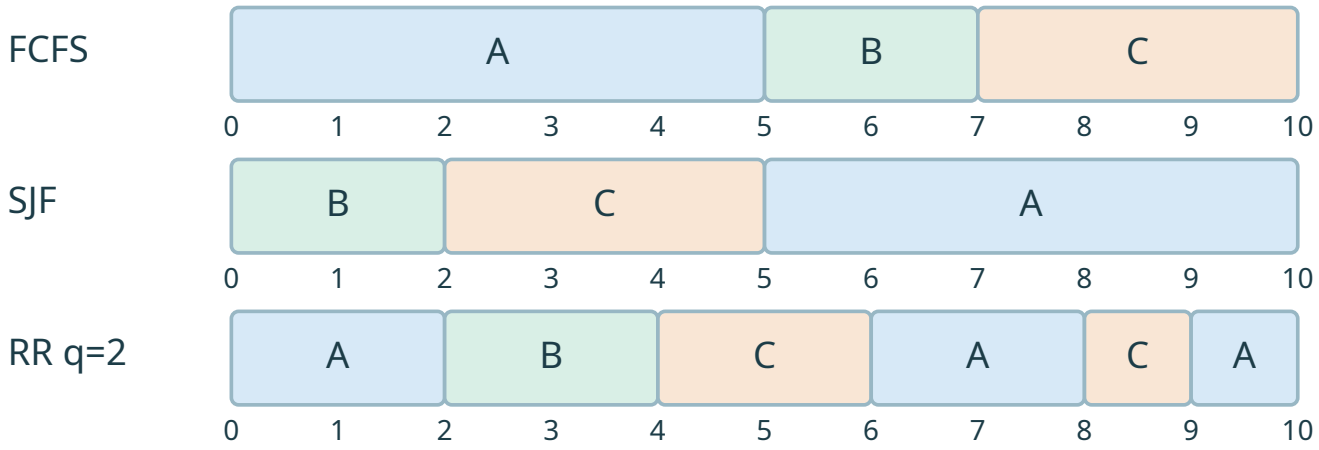
Use the same small tasks to see why different scheduling policies give different results.

REMEMBER THIS / A BUS STATION

FCFS: first in line. **SJF**: shortest complete job next. **RR**: everyone gets a turn. **SRT**: a newly arriving shorter remaining job can interrupt. **MLFQ**: multiple priority queues with promotion/demotion rules.

Exam precision: FCFS and ordinary SJF are non-pre-emptive. Round Robin uses a time quantum; SRT is pre-emptive. A multilevel feedback queue must have an explicit policy. Waiting time = completion - arrival - CPU burst, if there is no I/O waiting.

SAME JOBS: A=5, B=2, C=3; ALL ARRIVE AT t=0



For processes A (burst 5), B (burst 2), C (burst 3), all arriving at time 0, the three diagrams show different orders. Time labels give CPU boundaries; each block width represents its duration.

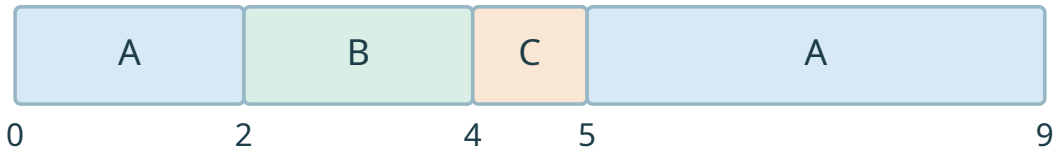
WORKED EXAMPLE 1 / EVALUATE WAITING

In SJF: B finishes at 2, C at 5 and A at 10. Waiting: B = 2-2 = 0; C = 5-3 = 2; A = 10-5 = 5. Mean = 7/3 = **2.33 time units**.

Pre-emption and priority queues

New arrivals change what the scheduler should do.

SRTF: ARRIVALS AND REMAINING TIME



MLFQ: TWO QUEUES WITH STATED POLICY

Q0: new jobs, quantum 2

Q1: demoted jobs, quantum 4

A: 0-2 in Q0; B: 2-3 in Q0; A: 3-7 in Q1.

Policy: Q0 has priority; unfinished Q0 jobs move to Q1.

SRT worked trace: A arrives at 0 (burst 6), B at 2 (burst 2), C at 3 (burst 1). A has 4 remaining at t=2, so B pre-empts it. At t=3 B and C both need one unit; the declared tie policy keeps B running. B completes at 4; C runs 4-5; A finishes 5-9.

Waiting times: A = 9 - 0 - 6 = **3**; B = 4 - 2 - 2 = **0**; C = 5 - 3 - 1 = **1**.

WORKED EXAMPLE 2 / A DIFFERENT FEEDBACK-QUEUE SCHEDULE

Q0 has priority and quantum 2; unfinished Q0 jobs move to Q1 (quantum 4). New jobs enter Q0. A (burst 6) and B (burst 1) arrive at t=0, A first. Trace: A 0-2 in Q0; B 2-3 in Q0; A 3-7 in Q1. No job arrives later. What changes if a short job arrives at t=4?

NOW BE PRECISE

Under this policy a new Q0 job arriving at t=4 pre-empts Q1 A. Other valid MLFQ policies may handle priority, ageing and queue demotion differently. Never assume an unstated rule.

Exam practice: scheduling

Draw timelines or calculate values before writing explanations.

Q1 [4 marks] Three processes arrive together: A=5, B=2, C=3. Give the SJF order and the three waiting times.

Q2 [6 marks] Use Round Robin quantum 2 for A=5, B=2, C=3, all arriving at t=0 and initially ordered A, B, C. Draw the timeline and calculate the completion times.

Exam practice: scheduling

Q3 [6 marks] A arrives at 0 (burst 6), B at 2 (burst 2), C at 3 (burst 1). Trace SRT, assuming the running process continues if remaining times are tied. State each completion time.

Q4 [6 marks] Compare SRT, RR and MLFQ for interactive applications. Explain one possible strength and one limitation of each approach.

Check your answers: scheduling

Marking points are indicative; give credit for equivalent correct reasoning.

Q1 [4] SJF B, C, A [1]; B waits 0 [1]; C waits 2 [1]; A waits 5 [1].

Q2 [6] A 0-2 [1], B 2-4 [1], C 4-6 [1], A 6-8 [1], C 8-9 and A 9-10 [1]; finishing times B4, C9, A10 [1].

Q3 [6] A 0-2 [1], B pre-empts at 2 [1], B remains at t=3 tie and finishes at 4 [1], C runs 4-5 [1], A resumes 5-9 [1], completion times A9 B4 C5 [1].

Q4 [6] SRT can favour short remaining jobs [1] but may starve long jobs [1]; RR shares CPU fairly with bounded turn lengths [1] but frequent context switches cost time [1]; MLFQ can adapt priorities for interactive processes [1] but the policy is complex and can permit starvation without safeguards [1].

Operating-system scheduling / Exam practice

Complete the questions independently. Use the space provided for your working.

Q1 4 MARKS

For A=4, B=3, C=2, all arriving at t=0, write the non-preemptive SJF order and calculate waiting times for A, B and C.

Q2 5 MARKS

Draw the Round Robin timeline for the same processes using a time quantum of 2. State when each process finishes.

Operating-system scheduling / Exam practice

Complete the questions independently. Use the space provided for your working.

Q3 6 MARKS

Explain how SRTF, Round Robin and multilevel-feedback scheduling can respond differently to short, interactive jobs.

Operating-system scheduling / Check your answers

Use these mark points after completing the independent questions.

Q1 / 4 MARKS

[1] Order C → B → A.

[2] C waits 0.

[3] B waits 2.

[4] A waits 5.

Q2 / 5 MARKS

[1] 0–2 A.

[2] 2–4 B.

[3] 4–6 C (finishes).

[4] 6–8 A (finishes).

[5] 8–9 B (finishes).

Q3 / 6 MARKS

[1] SRTF can pre-empt when a shorter remaining job arrives.

[2] It requires an estimate of remaining CPU burst length.

[3] Round Robin cycles through ready tasks by fixed quantum.

[4] Shorter quantum often improves responsiveness but increases context-switch overhead.

[5] Multilevel feedback uses multiple queues and a defined promotion/demotion policy.

[6] It can favour interactive jobs but may cause starvation without suitable ageing/fairness.

Operating-system types: recognise the need

REMEMBER / ASK FIVE QUESTIONS

Is the defining feature several cooperating machines, a dedicated device, several processes, several users or a time deadline? These describe features, not brand names; one system can fit several categories.

FIVE TYPES / ONE COMPUTER CAN FIT SEVERAL

DISTRIBUTED	Several machines cooperate
EMBEDDED	Dedicated-purpose device
MULTITASKING	Many processes make progress
MULTI-USER	Multiple accounts and permissions
REAL TIME	Meet specified deadlines

Type	Distinctive feature
Distributed	Coordinates work/resources across multiple networked computers.
Embedded	Dedicated to a limited function within a device.
Multitasking	Allows multiple tasks/processes to progress.
Multi-user	Manages multiple user accounts, access rights and sessions.
Real time	Response must meet bounded timing requirements or deadlines.

OS types: classify, justify, compare

WORKED EXAMPLE 1 / emergency-stop controller

A machine must act on an emergency-stop input within a strict deadline. It runs on a dedicated controller board.

MODEL RESPONSE

It is real time because the response must meet a predictable deadline; it is also likely embedded because the computer is dedicated to a small number of device functions. “Real time” does not merely mean “very fast”.

WORKED EXAMPLE 2 / school network server

A school server authenticates separate staff and pupil accounts, stores folders with permissions and runs printing and database services.

GUIDED RESPONSE

Multi-user: account separation and permissions. Multitasking: print and database processes are scheduled. If the service is spread over several cooperating servers, a distributed approach may also apply.

Common mistakes

A desktop PC can be multitasking and multi-user even if one person is currently signed in. A cluster is not necessarily a single distributed OS simply because computers are connected by a network. Multiple labels can be correct only when the corresponding feature is actually present.

Explain, do not just label

A strong answer gives classification + requirement + technical reason. Example: real time → braking deadline → predictable maximum response latency.

Operating systems / exam practice

Answer independently. Include precise OS functions and justify classifications.

Q1 · 4 marks Explain two different resources managed by the OS when a pupil uses a browser and a backup program.

Q2 · 5 marks For each scenario, identify the most appropriate OS type: (i) a dedicated parking-sensor controller; (ii) a system with several user accounts; (iii) a safety controller with a fixed response deadline; (iv) a laptop running multiple programs; (v) cooperating computers presenting one managed service.

Operating systems / exam practice

Q3 · 6 marks An emergency-braking controller runs on dedicated hardware. Explain why embedded and real-time are both appropriate classifications, and distinguish meeting a deadline from simply having a fast processor.

Q4 · 6 marks A school operates a central server with separate teacher and pupil logins, a print queue and several background services. Explain how the OS supports users and running processes, and explain whether the description necessarily makes the system distributed.

Operating systems / marking points

Q1 / 4 marks

• CPU scheduling/time allocation [1] with a relevant effect on the two active processes [1].

• Memory protection/allocation [1] with a relevant effect such as separate address spaces or paging [1].

• Accept another two supported OS responsibilities, each with a clear function, to a maximum of four marks.

Q2 / 5 marks

• (i) Embedded: dedicated controller [1].

• (ii) Multi-user: separate user sessions/accounts [1].

• (iii) Real time: bounded deadline [1].

• (iv) Multitasking: processes progress concurrently [1].

• (v) Distributed: cooperating machines present a coordinated service [1].

Q3 / 6 marks

• Embedded means purpose-built/specialised software and hardware [1].

• It is designed for the braking function rather than arbitrary general-purpose user tasks [1].

• Real time means an action must happen by a specified deadline [1].

• Deterministic/predictable worst-case response matters [1].

• Missing the deadline can make an otherwise correct output too late [1].

• A high average speed alone does not guarantee meeting a worst-case deadline [1].

Q4 / 6 marks

• Multi-user OS manages separate accounts/sessions [1] and access permissions/isolation [1].

• Multitasking schedules print/database/background processes [1] and allows each to make progress [1].

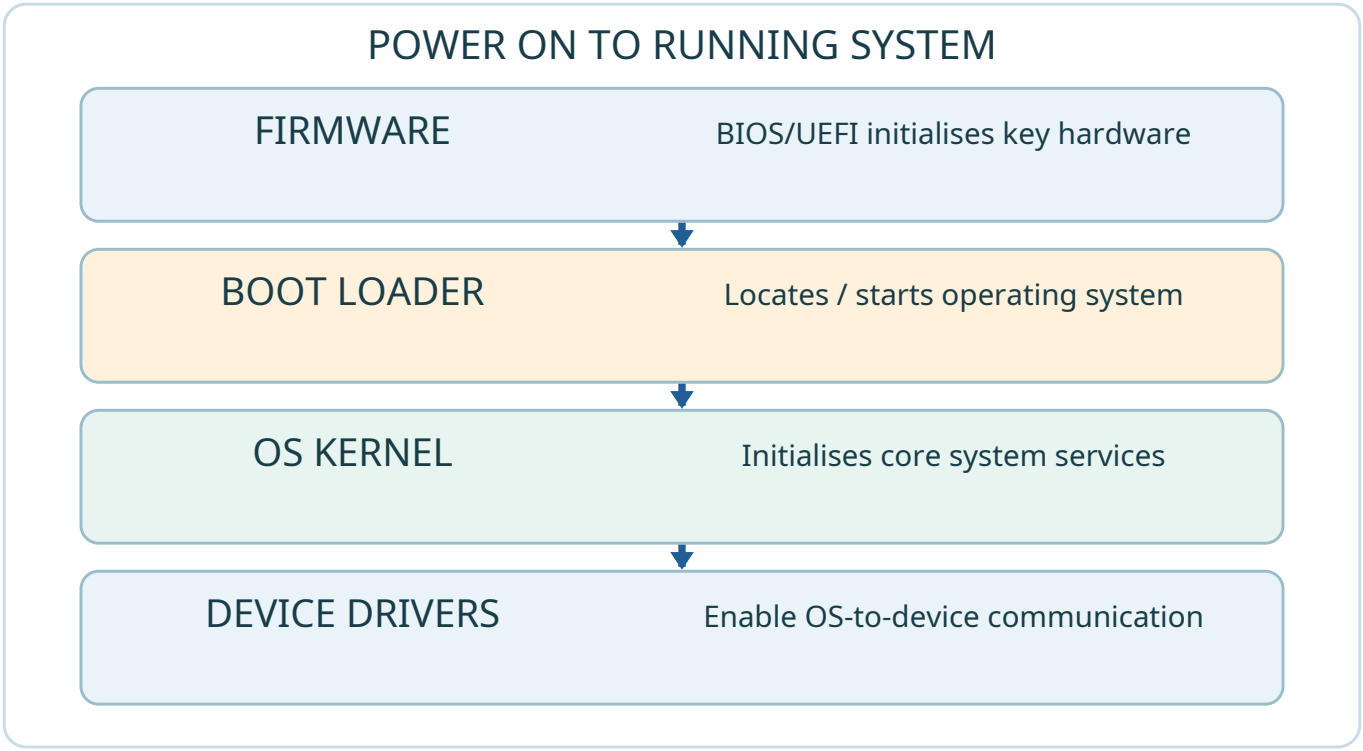
• File or device/print services coordinate shared resources [1].

• A single central server is not necessarily distributed; distribution involves multiple cooperating computers as a coordinated system [1].

Power-on: BIOS/UEFI and booting

REMEMBER / OPEN THE BUILDING BEFORE WORK BEGINS

Firmware is the opening staff: it checks and initialises key hardware, then hands control to a boot loader. The operating system then takes over. BIOS and modern UEFI are firmware interfaces; they are not the operating system itself.



WORKED EXAMPLE 1 / laptop is switched on

Power → BIOS/UEFI firmware initialises necessary components → firmware locates a boot device/boot manager → boot loader loads/starts the OS kernel → OS starts services and appropriate drivers. Actual implementations can have extra stages; this is a simplified teaching sequence.

WORKED EXAMPLE 2 / boot fails before login

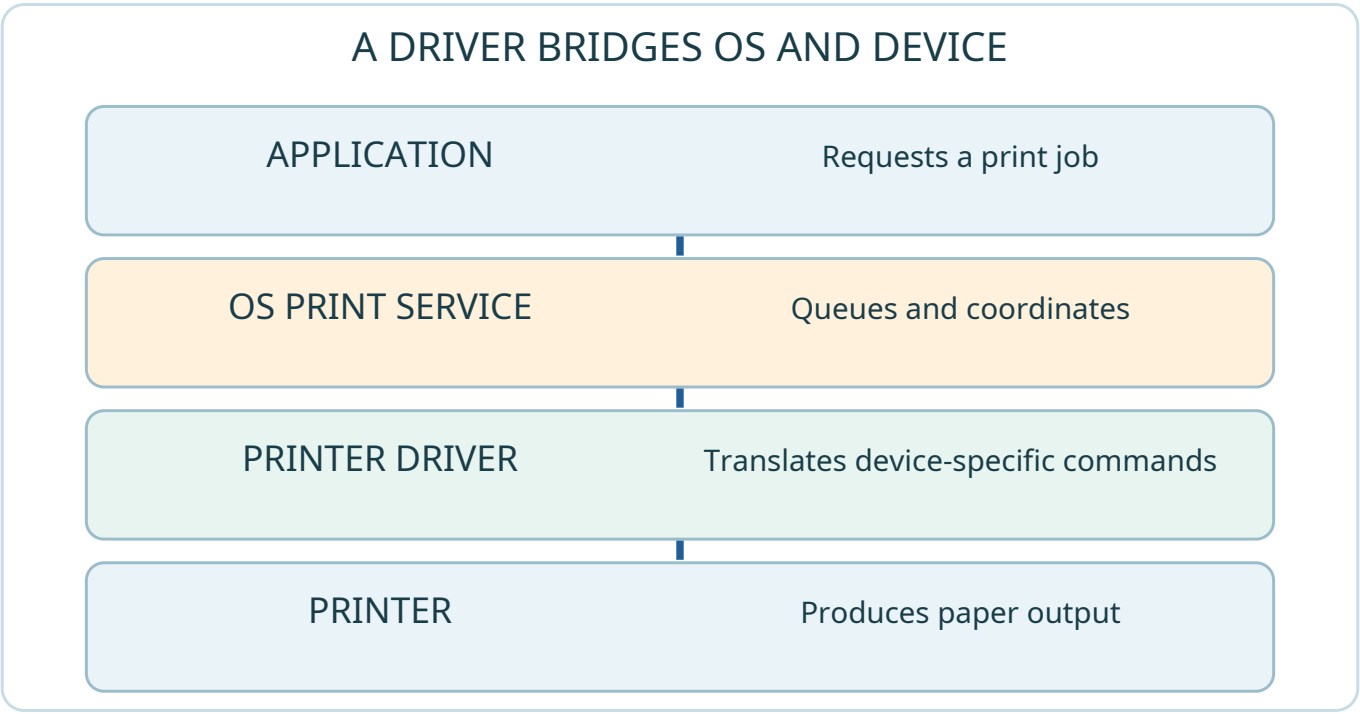
A machine detects its storage device but reports that no bootable operating system is found. Identify which early stages may have succeeded and what step has failed. Firmware can detect hardware, yet the expected boot manager or OS may be absent, corrupt or incorrectly configured.

Exam distinction: Firmware initialises and begins the boot process; the boot loader starts the OS. Do not write that the BIOS “is Windows”.

Device drivers: hardware-specific access

REMEMBER / A DRIVER IS A TRANSLATOR

A driver converts generic OS requests into operations suitable for particular hardware. Unlike a language translator, it maps OS/device operations and manages data/control specific to the device.



WORKED EXAMPLE 1 / a printer appears but cannot print

A word processor requests printing. The OS print service queues the job; a compatible printer driver prepares commands/data appropriate to the printer. A missing or incompatible driver can stop printing despite the device being physically connected.

WORKED EXAMPLE 2 / a new scanner

The OS lists a scanner, but scan requests fail. Explain why connecting the cable is insufficient. The device may require a compatible driver to map scanning commands, read device status and transfer image data to the OS.

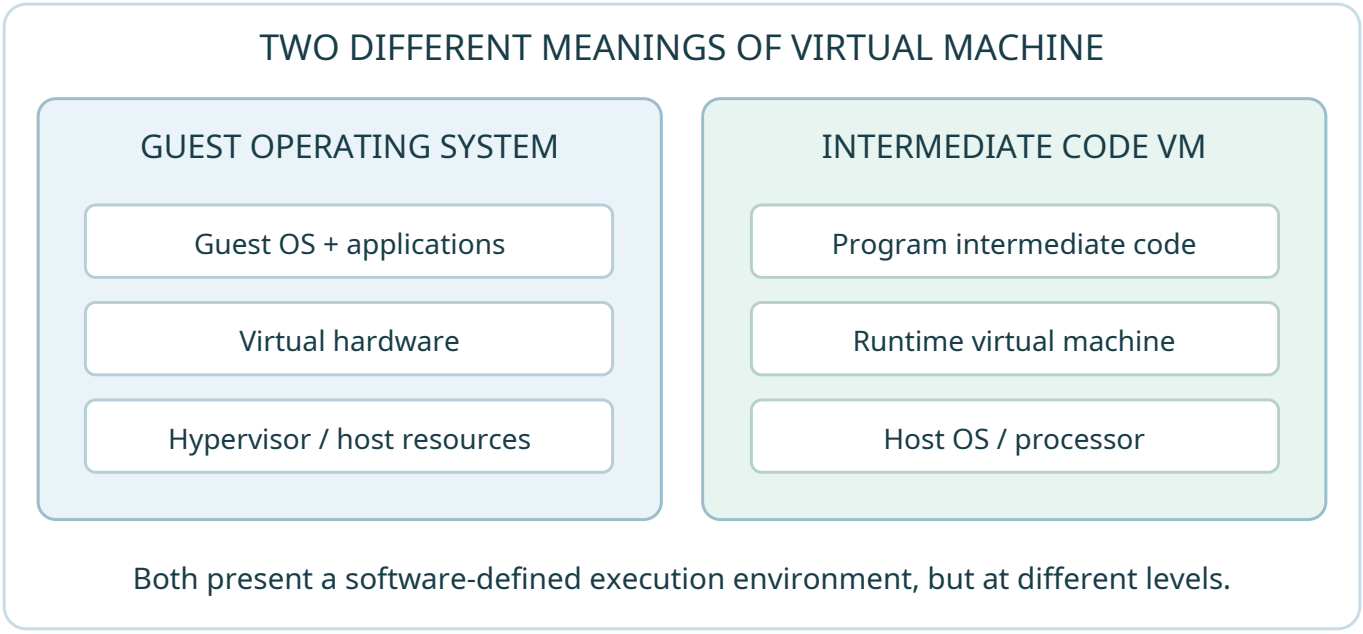
PRECISE WORDING

Drivers are system software that allow the operating system to communicate with a device or device controller. The OS need not contain each device’s low-level instructions directly.

Virtual machines: two examinable models

REMEMBER / A MACHINE MADE OF SOFTWARE

A virtual machine is software that provides an execution environment behaving like a machine. OCR expects both running an OS inside another and executing intermediate code. They are related, but not the same design.



Model A / an operating system inside another

A hypervisor provides virtual hardware (CPU, memory and devices) so a guest OS runs as though it had its own computer. It may share real host hardware with other guests. Useful for testing software, isolation and server consolidation; host resource usage and performance overhead remain.

Model B / executing intermediate code

A language runtime VM, such as the Java Virtual Machine, executes compiled bytecode using a suitable runtime implementation. This provides a common intermediate target across different host platforms; the bytecode is not the host CPU’s native instruction set.

Virtual machines: apply and evaluate

WORKED EXAMPLE 1 / test another OS on one computer

A technician needs to test a different operating system without overwriting the school laptop’s installed OS.

MODEL ANSWER

Create a guest virtual machine with virtual hardware and install the testing OS there. A hypervisor schedules guest resource access. This supports reset/snapshots but consumes host RAM/storage and is not a guarantee against every security risk.

WORKED EXAMPLE 2 / program for several platforms

A programmer distributes compiled bytecode which can run where the corresponding runtime VM is installed. Explain why this approach differs from installing a second operating system. The runtime interprets/compiles intermediate code; it does not normally provide a complete guest OS and virtual hardware.

Exam check

Term	Distinguish
Hypervisor	Runs/manages guest OS virtual machines.
Guest OS	Full operating system within a virtual environment.
Bytecode	Intermediate program form for a runtime VM.
Runtime VM	Executes bytecode on a host environment.

BIOS, drivers and VMs / exam practice

Q1 · 4 marks Describe the separate roles of BIOS/UEFI firmware, the boot loader, the operating-system kernel and a device driver during system startup.

Q2 · 6 marks A network printer is detected but will not print correctly. Explain the roles of the application, operating system and device driver, and why a driver can be essential.

BIOS, drivers and VMs / exam practice

Q3 · 6 marks A school wants to test unfamiliar operating systems on existing laptops. Explain how a guest OS virtual machine helps and evaluate two limitations or risks.

Q4 · 6 marks Compare a hypervisor-based virtual machine with a virtual machine that executes bytecode, using a clear example of the purpose of each.

BIOS, drivers and VMs / marking points

Q1 / 4 marks

- BIOS/UEFI initialises necessary hardware/starts boot sequence [1].
- Boot loader locates/starts OS kernel [1].
- OS kernel provides core resource-management services [1].
- Driver enables OS-specific communication with hardware [1].

Q2 / 6 marks

- Application creates the print request [1].
- OS print service queues/manages it [1].
- Driver translates OS requests into device-specific commands [1].
- Driver controls appropriate device data/status exchange [1].
- Correct compatibility matters for a particular hardware model/interface [1].
- Absent/incompatible driver can prevent correct operation even if attached [1].

Q3 / 6 marks

- Hypervisor supplies virtual hardware [1] on existing host resources [1].
- Guest OS can be installed/tested without replacing the normal host OS [1].
- Isolation or snapshot/reset supports safer experimentation [1].
- VM consumes host RAM/CPU/storage and may be slower [1].
- Isolation is not absolute and misconfiguration/security vulnerabilities remain possible [1].

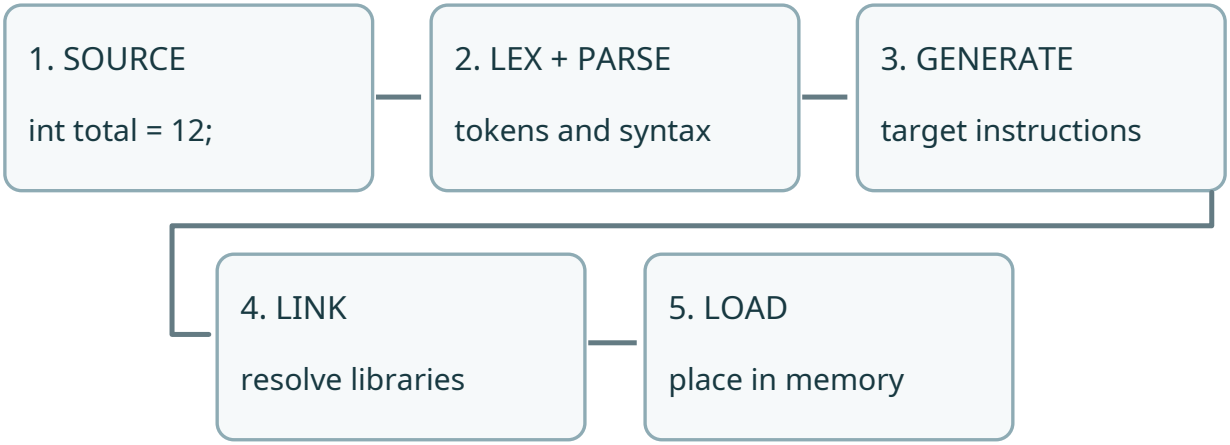
Q4 / 6 marks

- Guest VM simulates hardware needed for a full OS [1].
- Hypervisor manages guest access to physical resources [1].
- Example: test Linux inside a host Windows/macOS environment [1].
- Runtime VM processes a defined intermediate instruction form, e.g. bytecode [1].
- It runs a program rather than necessarily providing a full second OS [1].
- Example: JVM runs Java bytecode with an appropriate runtime on a host system [1].

1.2.2 / Applications and translation

Remember: The compiler is a translator; the linker is an organiser.

FROM SOURCE CODE TO A RUNNING PROGRAM



Compilation, linking and loading are distinct stages.

ANALOGY - PICTURE IT

A translator changes the language of instructions; a linker combines translated modules with required library code.

NOW BE PRECISE - WHERE THE ANALOGY STOPS

Compilers may optimise and target machine code or intermediate representations. Interpretation is an execution approach, not merely translation word-by-word.

THEORY / KEY TERMINOLOGY

Compiler	Translates source into target code; reports errors before or during translation.
Interpreter	Executes a program through an interpretation process, normally processing instructions during execution.
Assembler	Translates assembly-language mnemonics into machine instructions.
Lexical analysis	Groups source characters into tokens.
Syntax analysis	Checks token structure against the language grammar.
Linker and loader	Linker resolves external code references; loader places a program in memory for execution.

Applications and translation / understand and model

Application software meets user needs; utility software maintains or protects a system. Open-source licences allow inspection and modification under specified terms, whereas closed-source applications generally restrict access to source code. Compilation typically includes lexical analysis, parsing, code generation and optimisation. Linking resolves references across modules and libraries; loading prepares the executable in memory. A language implementation may combine compilation and interpretation, so avoid over-generalised either/or statements.

WORKED EXAMPLE 1 / worked example

Source statement: int total = 7 + 5;

Lexical stage: identify tokens such as int, identifier total, assignment operator, literals 7 and 5 and punctuation.

Syntax stage: check that the tokens form a valid declaration/assignment expression.

Code generation: produce corresponding lower-level instructions; optimiser may replace 7 + 5 with constant 12.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

For int marks = 4 + 6;, identify five categories of tokens and explain which stage checks whether the tokens form a syntactically valid statement.

Scaffold: List keyword, identifier, operator, literals and punctuation. Do not confuse lexical with syntax analysis.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

Tokens: int; marks; =; 4; +; 6; semicolon.
Syntax: A parser checks the grammar/structure.
Optimisation: Constant folding may evaluate 4+6 as 10.

Applications and translation · independent exam practice

Q1 [4 marks] Distinguish compilation from interpretation.

Q2 [3 marks] Explain the purpose of a linker after compilation.

Q3 [4 marks] Describe lexical analysis and syntax analysis.

Applications and translation · independent exam practice

Q4 [4 marks] A school wants to adapt an accessibility feature. Evaluate open-source against closed-source software.

Model answers and marking points

Applications, interpreters, compilers and licensing

Compare with your own response. Credit equivalent technically accurate statements.

Q1 / 4 marks

1. Compilation translates source into a target representation such as object code, native code or bytecode [1].

2. The translated output can subsequently be used without re-reading the original source in the same way [1].

3. Interpretation executes/evaluates instructions during program execution, whether from source or an intermediate form [1].

4. An interpreter/runtime is involved during execution, while the build/run stages differ; valid hybrid or JIT examples are accepted [1].

Q2 / 3 marks

1. Separately compiled source files create object modules or code that may contain unresolved references [1].

2. The linker resolves references to symbols defined in other modules or libraries [1].

3. It combines/arranges the necessary compiled code and references to produce a linked program [1].

Q3 / 4 marks

1. Lexical analysis recognises sequences of characters as tokens [1].

2. Examples include names, literals, keywords and operators [1].

3. Syntax analysis checks token sequences against the language grammar [1].

4. It builds/checks grammatical structure or reports syntactic errors [1].

Q4 / 4 marks

1. An open-source licence may permit inspection and modification of the source [1].

2. This can allow staff/developers to adapt an accessibility feature [1].

3. With closed-source software, changes to inaccessible proprietary source may depend on the supplier [1].

4. A justified trade-off such as supplier support, compatibility, expertise, licensing or maintenance [1].

Exam tip: Do not assume all compiled programs are faster or all open-source licences have identical terms.

Applications: choose for the purpose

REMEMBER / THE TOOL DOES THE USER'S JOB

An application enables a particular user task. Do not select software from its familiar name alone: identify the user's task, functional needs, accessibility, compatibility, security and cost.

MATCH SOFTWARE TO THE ACTUAL NEED

APPLICATION	Write, analyse, create
UTILITY	Maintain, protect, recover

WORKED EXAMPLE 1 / a school writes a report and analyses marks

Reporting: choose a word processor for a structured document with accessible headings. Marks: choose a spreadsheet for formula-driven calculation and charts. If several staff need shared records, a managed database application may be better than emailing separate spreadsheet versions.

WORKED EXAMPLE 2 / a photo department

A teacher needs to edit high-resolution photographs and export web-sized images. Choose image-editing software, checking image formats, colour management, hardware performance and whether non-destructive adjustments are needed.

EXAM PRECISION

A good justification links a feature to the user requirement and, where requested, evaluates a realistic limitation. "It is good software" is not evidence.

Utilities: keep the system working

REMEMBER / BACKSTAGE CREW

Application = the performance the user wanted; utility = backstage maintenance, protection and recovery. Some software overlaps, so classify by its main purpose in the scenario.

Utility	Typical purpose
Backup	Creates recoverable copies; restoration must be tested.
Antimalware	Scans and helps detect/quarantine malicious code.
Compression	Reduces storage or transfer size; may be lossless.
Disk cleanup	Removes unneeded temporary/cache files safely.
Encryption	Protects confidentiality of stored/transmitted data.
Disk defragmentation	Reorders HDD blocks; normally not useful or recommended for SSDs.

WORKED EXAMPLE 1 / coursework protection

A pupil worries about losing a coursework folder. A backup utility with versioned recovery addresses that risk; compressing a folder alone is not a backup unless a safe separate copy is retained.

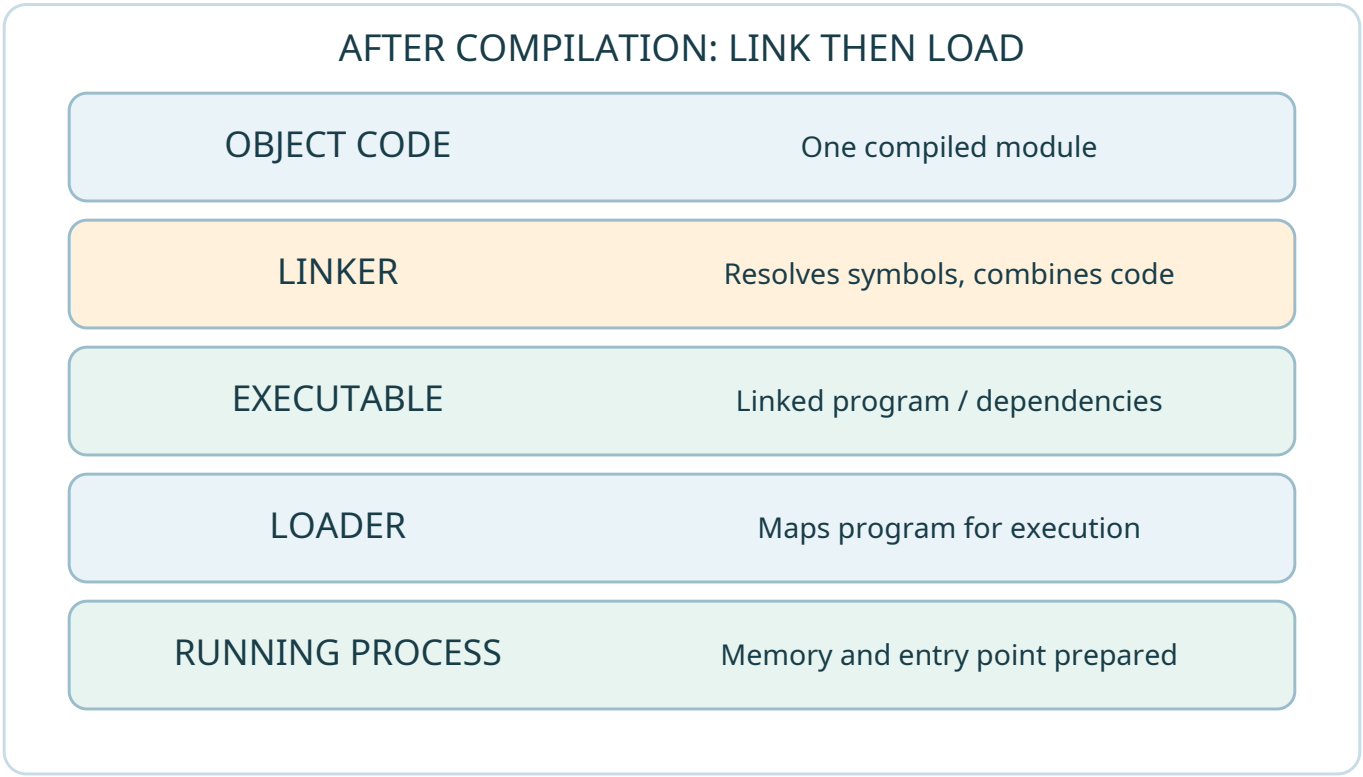
WORKED EXAMPLE 2 / a full school laptop

Choose a disk-cleanup utility to remove safe temporary files, then diagnose what actually consumes storage. Do not recommend routine HDD-style defragmentation for an SSD. Check the requirement before selecting a tool.

Linkers, loaders and libraries

REMEMBER / JOIN THE PIECES, THEN OPEN THE SHOW

The linker assembles references between compiled modules and selected libraries. The loader prepares the executable for running. A library supplies reusable prewritten code.



WORKED EXAMPLE 1 / missing PrintReport() implementation

A compiled module refers to a function defined in a different compiled object file. The linker resolves its external symbol and connects the code. If the required implementation/library is missing, a link-time unresolved-symbol error may occur even if parsing and compilation of the individual source file succeeded.

WORKED EXAMPLE 2 / built program fails to start

An executable exists, but an essential shared library cannot be found at runtime. Explain why compilation and linking may have succeeded, yet the loader/runtime dependency resolution prevents startup. Distinguish the stage of failure.

DO NOT CONFUSE

Compiler: translates source code to lower-level output. Linker: resolves/composes compiled modules. Loader: prepares/maps an executable for running. Some environments distribute these jobs across different tooling or runtime stages.

Applications, utilities and linking / practice

Q1 · 4 marks A school wants software for statistical analysis of assessment data and separate software to restore deleted files. Identify a suitable application and utility, giving a reason for each.

Q2 · 6 marks A department must select an image-editing application for high-resolution photographs. Explain three requirements it should consider and why each matters to the user.

Applications, utilities and linking / practice

Q3 · 6 marks Compare backup, compression and disk-cleanup utilities. Explain why creating a compressed archive on the same drive is not, by itself, a reliable backup strategy.

Q4 · 6 marks Two source files compile separately. One calls PrintReport() defined in the other. Explain the roles of object files, linker, library and loader, and identify a possible reason the program might fail to run.

Applications, utilities and linking / marking points

Q1 / 4 marks

• Spreadsheet/statistical analysis software is an appropriate application [1], because formulas/statistical functions process marks [1].

• Backup/recovery utility is an appropriate utility [1], because it preserves recoverable copies or earlier versions [1].

• Accept equivalent justified specialist applications.

Q2 / 6 marks

• Correct import/export format support [1] avoids losing compatibility/quality [1].

• Ability to edit at high resolution or preserve original data [1] supports demanding image work [1].

• Performance/compatibility/accessibility/user licensing or budget [1] linked to a genuine user need [1].

• Accept three well-justified distinct requirements, two marks each.

Q3 / 6 marks

• Backup preserves copies for restoration [1].

• Compression reduces file size/transfer volume [1].

• Cleanup removes selected unnecessary files [1].

• Same-drive archive can be lost if the drive fails [1].

• It may also be overwritten/deleted/affected by damage or malware [1].

• Separate protected backup and recovery test improves resilience [1].

Q4 / 6 marks

• Compiler produces object code/compiled modules [1].

• Linker resolves references to code in other modules [1].

• Libraries supply reusable routines [1].

• Linker includes/references appropriate library code [1].

• Loader prepares the executable in memory for execution [1].

• Missing library/symbol at link or loading time may prevent successful execution [1].

1.2.3 Software development

Remember: waterfall is a staircase, Agile is a series of laps, XP is a pit crew, Spiral is a risk radar, and RAD is the quick prototype. Each memory hook is a starting point, not an exact description of every project.

Five ways to manage software development

WATERFALL	AGILE	XP	SPIRAL	RAD
phases	iterations	practices	risk	prototype

Select the method to suit the project, not the other way around.

Fixed requirements | frequent feedback | technical risk | fast prototypes

NOW BE PRECISE

Waterfall works through relatively distinct stages; Agile uses iterative, incremental delivery and feedback; extreme programming (XP) adds disciplined engineering practices such as pair programming and test-driven development; Spiral explicitly evaluates risks each cycle; RAD uses rapid prototypes and close user input.

WORKED EXAMPLE 1 / High-risk project

A satellite-control project has major technical and safety uncertainties. Spiral is a defensible choice because early cycles can prototype risky components and evaluate the results before committing to the next stage.

WORKED EXAMPLE 2 / Revision app

Students request changes after trying each version of a revision app. Which approach suits the feedback cycle?

Guided response

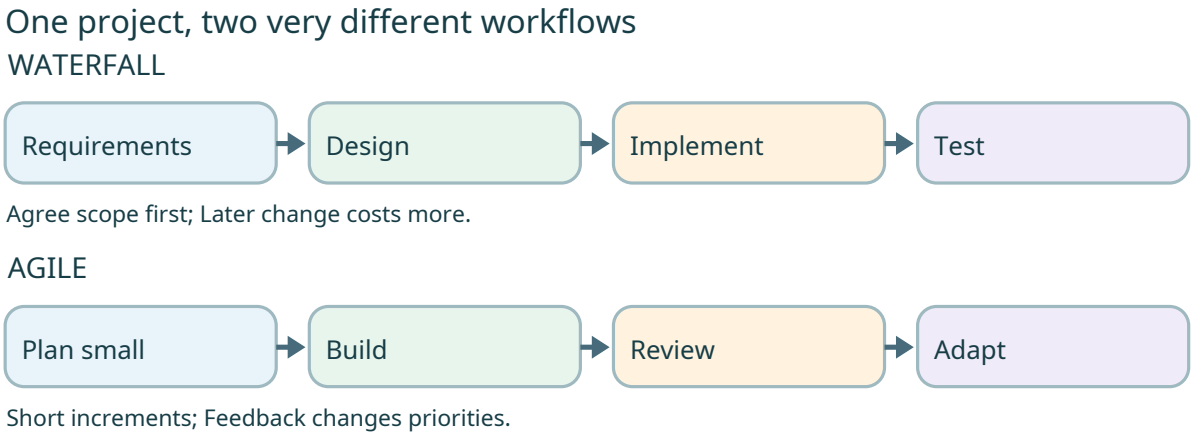
An Agile approach can release an initial quiz feature, gather student feedback, then prioritise improvements. Explain the benefit of working increments and the cost of regular consultation.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

Waterfall	Sequential development model with relatively distinct stages.
Agile	Iterative, incremental development with regular feedback.
Spiral	Risk-driven development that explicitly evaluates risks each cycle.
RAD	Rapid Application Development: prototyping and close user involvement.
Extreme Programming	Agile engineering practices, including pair programming and test-driven development.
Prototyping	Producing an early model to test requirements and obtain feedback.
Iteration	One cycle of development, testing and improvement.

Choosing a development approach



A methodology is a structured approach, not a guarantee of success. Waterfall can suit stable approved requirements, but late changes are expensive. Agile can absorb changed priorities, but needs communication and disciplined planning.

XP / practical precision

Pair programming, test-driven development, continuous integration and small releases can detect issues earlier, but require team commitment. XP is a particular Agile approach, not an entirely unrelated lifecycle.

RAD / practical precision

A café unsure of its booking-screen layout can give quick feedback on a prototype. RAD helps clarify the interface, but rapidly produced prototypes can require careful redesign before release.

Writing and following algorithms

The specification also expects precise algorithm writing and tracing. For a non-empty array, a largest-value algorithm sets the current maximum to the first element, then updates it only when a larger value is found. Its precondition matters: an empty array has no first element.

COMMON MISTAKE

Fewer stages or shorter code does not automatically mean a method is faster or better. Justify a recommendation against requirements, risk, budget, stakeholder feedback and testing.

Exam Practice

Q1 · 3 marks State one defining feature of Waterfall, Agile and Spiral development.

Q2 · 4 marks A café frequently changes its proposed mobile booking interface. Recommend a development methodology and explain two reasons for your choice.

Q3 · 4 marks Explain two XP practices that can improve the quality of software as a team makes frequent changes.

Q4 · 6 marks Compare Waterfall and Spiral for a high-risk medical control system. Include advantages, limitations and a justified conclusion.

Q5 · 6 marks An algorithm finds the largest number in an integer array. Describe its precondition and trace the maximum values when processing [4, 2, 9, 6]. Explain how you would test it.

Model answers and marking points

Attempt the questions before checking your responses. Other technically accurate explanations may also earn credit.

Q1

Waterfall: work progresses through planned phases [1]. Agile: repeated increments with feedback [1]. Spiral: risk evaluation forms part of repeated cycles [1].

Q2

A prototyping-focused RAD or iterative Agile approach is defensible [1]. Early versions allow users to examine the interface [1], feedback can change the design [1], and this reduces the risk of building a layout users do not want [1].

Q3

For example, pair programming supports immediate peer review [2]; test-driven development writes tests before implementation and detects regressions [2]. Continuous integration or frequent tests are also creditworthy if explained.

Q4

Waterfall gives defined documentation/approval stages [1], useful for stable requirements [1], but late change may incur costly rework [1]. Spiral explicitly identifies and reduces major technical risks [1] through prototyping/testing [1], although its management cost is higher [1]. A well-justified choice uses the project risk and formal approval requirements.

Q5

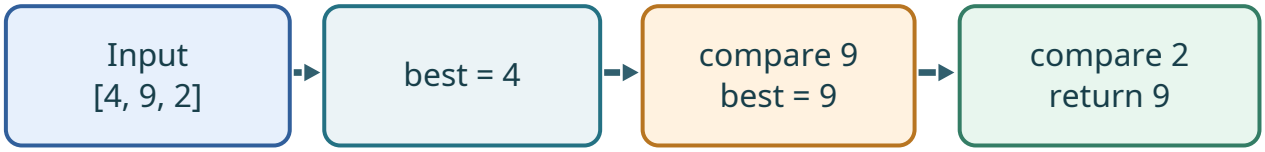
Precondition: nonempty array [1]. Initialise best=4 [1]; compare 2 so best stays 4 [1]; process 9 so best=9 [1]; process 6 so best remains 9 [1]. Testing: check a single-element array, all-negative values, duplicate values, and define behaviour for empty input [1].

Write, follow and verify algorithms

Theory / Worked Examples

Following an algorithm means recording state after each operation. Writing one means making its preconditions, decisions and termination explicit.

State, test, update, stop



Precondition: non-empty list; one pass; O(n) time, O(1) auxiliary space.

REMEMBER THIS

A recipe with a test kitchen. A recipe must specify ingredients (inputs), steps, a stopping point, and a result. An algorithm also needs test cases that challenge assumptions.

Now be precise

- **Precondition:** what must be true before starting, such as an array being nonempty.
- **Invariant:** a useful statement that remains true as a loop progresses; for a maximum scan, best is the largest value seen so far.
- **Trace:** record index, current value and best after each comparison. Then test an empty input and negative-only inputs where relevant.

WORKED EXAMPLE 1

Trace Largest([4, 9, 2]) with best initially equal to the first element.

Method: Initial best=4. Read 9 → best=9; read 2 → best remains 9. Return 9. Preconditions: nonempty array. Time O(n), extra space O(1).

WORKED EXAMPLE 2

Trace Largest([-8, -3, -12]) without assuming a starting best of 0.

Method: Initial best=-8, then -3 replaces it; -12 does not. Return -3. Starting best=0 would be wrong for all-negative input.

COMMON MISTAKE

An algorithm that works on the example may still fail at a boundary.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 3 marks. State a precondition and a loop invariant for the maximum algorithm.

Q2 · 4 marks. Complete a trace of Largest([6, 1, 8, 5]). Give best after each comparison.

Q3 · 6 marks. Write pseudocode that counts values greater than 10 in a list and explain two tests.

Mark scheme

Compare only after completing the questions.

Q1 · 3 marks

Input has at least one element [1]; best is the maximum of elements visited [2].

Q2 · 4 marks

Initial best6 [1]; after1 still6 [1]; after8 becomes8 [1]; after5 remains8 [1].

Q3 · 6 marks

count=0 [1]; loop through every element [1]; if value>10 then increment [1]; return count [1]; test empty list gives0 [1]; test boundary 10 is not counted [1].

RECALL

A recipe with a test kitchen: A recipe must specify ingredients (inputs), steps, a stopping point, and a result. An algorithm also needs test cases that challenge assumptions.

1.2.4(a)(b)(e) / COMPUTER SYSTEMS

Programming paradigms and objects

The same problem can be expressed procedurally or using objects. The important distinction is how instructions, data and state are organised.

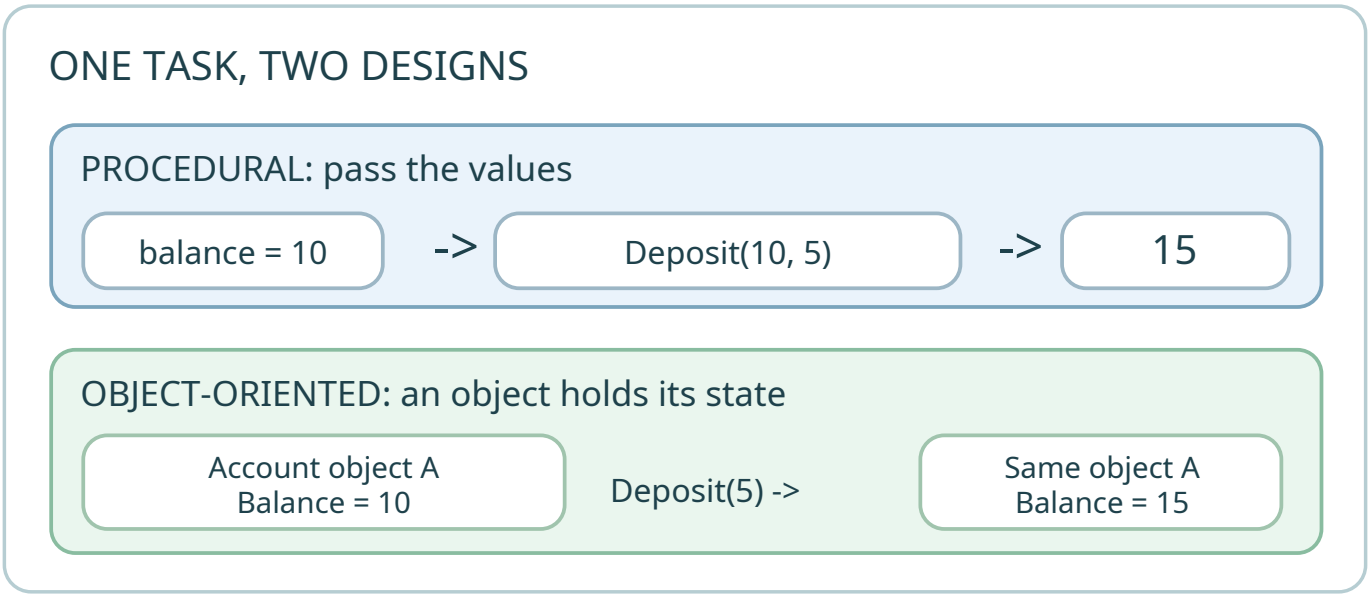
REMEMBER THIS

Procedural programming is like a recipe: call instructions with ingredients. An object is like a vending machine: state and allowed operations are grouped together.

NOW BE PRECISE

Procedural code groups operations into subroutines; object-oriented code uses classes to define objects with attributes and methods. Encapsulation controls access to state. Inheritance and polymorphism support reuse and different behaviour behind a shared interface.

Where the analogy stops: A real object is not a physical box. C# supports multiple styles; a short standalone function is not automatically a purely functional program.



One problem, two designs: the data and method are arranged differently.

EXAM PRECISION / PARADIGMS AND MODULARITY

Procedural: express steps using statements and subroutines. Declarative: specify a desired result (e.g. SQL). Functional: compose functions while limiting side effects. Modularity means separating a system into components; a program can be both procedural and modular.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

Procedural	Programming organised as statements and subroutines operating on data.
Object-oriented	Programming organised around objects with state and methods.
Encapsulation	Grouping state and methods and controlling access to state.
Inheritance	Creating a class with features inherited from another class.
Polymorphism	Different implementations working through a shared interface.
Functional	Programming centred on functions while limiting side effects.
Declarative	Specifying the desired outcome without detailing each execution step.

WORKED EXAMPLE 1 / Two independent accounts

```
class Account {
  public int Balance { get; private set; }
  public Account(int start) { Balance = start; }
  public void Deposit(int amount) { Balance += amount; }
}
Account a = new Account(10);
Account b = new Account(20);
a.Deposit(5);
```

The state of a changes from 10 to 15. The state of b remains 20 because it is a different object. A method can operate on the state belonging to one instance.

WORKED EXAMPLE 2 / A different Score object

```
Score red = new Score(7);
Score blue = new Score(11);
blue.Add(4);
```

Predict the result first. Only blue changes: red stays 7 and blue becomes 15. Explain which attribute is changed and why there is no effect on red.

EXAM PRECISION

Remember the vocabulary: class = blueprint; object = instance; attribute = stored state; method = behaviour; inheritance = one class derives from another; polymorphism = a shared operation can have type-dependent behaviour.

Exam practice

Q1 · 3 marks State what is meant by a class, an object and a method.

Q2 · 4 marks Explain how encapsulation helps protect the balance stored by an Account object.

Q3 · 4 marks Two Account objects begin with balances 12 and 27. The first receives Deposit(8); the second receives Withdraw(2). State both final balances and explain why they change independently.

Q4 · 6 marks A base class Notification defines Send(). EmailNotification and SmsNotification each provide a different implementation. Explain inheritance and polymorphism in this design, and one maintenance benefit.

Q5 · 6 marks Compare a procedural function Deposit(balance, amount) that returns a new balance with an object method Account.Deposit(amount). Include one benefit and one limitation of each design.

EXAM CHECK / Q5

Compare organisation and state, not rigid rules: procedural and object methods can both return values and/or mutate state. Explain how arguments, object fields, encapsulation and the caller relate.

Model answers and marking points

Attempt the questions before checking your work. Equivalent technically accurate answers may also receive credit.

Q1 · 3 marks

Class: a definition for a type, including data and behaviour [1]. Object: an instance of a class with its own state [1]. Method: a named operation associated with a class/object [1].

Q2 · 4 marks

Hide or restrict direct access to Balance [1]; require changes through controlled methods or properties [1]; methods can validate amounts [1]; this reduces invalid state or unintended modification [1].

Q3 · 4 marks

First balance 20 [1], second 25 [1]. They are separate object instances [1] and each method updates the state belonging to its own instance [1].

Q4 · 6 marks

The derived classes inherit from a common base [1]; shared members can be reused [1]. Each class can override/implement Send differently [1]. A caller can use a base type/interface reference [1] while the runtime selects behaviour for the actual object [1]. Common interfaces reduce duplication or simplify extension [1].

Q5 · 6 marks

Procedural form receives explicit arguments [1] and can be easily tested/reused [1], but callers must use its returned value correctly [1]. OOP stores balance with methods on the object [1], aiding encapsulation [1], but a class hierarchy can add complexity for simple tasks [1].

MARKING NOTE / Q5

Credit either style for returning or changing values when correctly described. Award comparison marks for explicit parameters, organisation of code/data, encapsulation, reuse and justified trade-offs.

1.2.4(c)(d) / COMPUTER SYSTEMS

Assembly, LMC and addressing

Assembly programs work close to the instruction level. OCR also expects you to follow and write simple Little Man Computer (LMC) programs and distinguish four addressing modes.

REMEMBER THIS

Think of numbered post-office boxes. The accumulator is the counter where calculations happen; a mailbox holds a stored instruction or value. The address tells you which box to use.

NOW BE PRECISE

Immediate addressing uses a literal operand; direct reads the named memory location; indirect reads an address stored at another location; indexed adds a register/offset to a base to find the effective address.

Where the analogy stops: LMC is an educational model, not a complete real processor. The bracket notation shown for addressing modes is illustrative generic assembly, not valid LMC syntax.

FOUR WAYS TO FIND AN OPERAND

Memory: M[20]=7 M[21]=20 M[22]=3 X=2

IMMEDIATE	#5 Literal value	5
DIRECT	[20] Read M[20]	7
INDIRECT	[[21]] M[21]->20, then M[20]	7
INDEXED	[20+X] 20+2=22, then M[22]	3

The same memory map produces different results under different addressing modes.

LMC: THE FIRST INPUT MUST BE STORED

INP 4
ACC = 4
STA FIRST

->

FIRST
mailbox = 4
value retained

->

INP 7
ADD FIRST
OUT 11

Memory keeps the first input while a second INP overwrites ACC.

LMC: STA preserves the first input before a second INP replaces ACC.

WORKED EXAMPLE 1 / LMC addition

```
INP
STA FIRST
INP
ADD FIRST
OUT
HLT
FIRST DAT 0
```

Inputs 3 then 5: ACC becomes 3; STA stores 3 in FIRST; the next INP makes ACC 5; ADD FIRST makes ACC 8; OUT displays 8.

WORKED EXAMPLE 2 / Trace different inputs

```
INP
STA FIRST
INP
ADD FIRST
OUT
HLT
FIRST DAT 0
```

Use inputs 6 and 9. Write the ACC and FIRST values after each instruction. Check that OUT displays 15, and explain why STA is necessary.

EXAM PRECISION

Addressing memory example: with M[30]=4, M[31]=30, M[32]=9 and X=2: #6 gives 6; [30] gives 4; [[31]] follows the pointer to obtain 4; [30+X] gives 9.

Exam practice

Q1 · 4 marks Define immediate, direct, indirect and indexed addressing.

Q2 · 6 marks Memory contains M[50]=8, M[51]=50 and M[52]=6; X=2. Calculate the results of #3, [50], [[51]] and [50+X]. Show both effective-address steps.

Q3 · 5 marks Trace the LMC program above using inputs 7 and 4. State ACC after each input, the stored value, the value after ADD and the output.

Q4 · 6 marks Write an LMC program that inputs two non-negative numbers, subtracts the second from the first, and outputs the result. Give the output for inputs 9 and 2.

Q5 · 5 marks Explain how BRA, BRZ and BRP affect the program counter, and why a backwards branch needs a terminating condition.

Model answers and marking points

Attempt the questions before checking your work. Equivalent technically accurate answers may also receive credit.

Q1 · 4 marks

Immediate: operand value is in the instruction [1]. Direct: operand identifies its memory location [1]. Indirect: specified location contains an effective address [1]. Indexed: base/address plus offset or index register gives effective address [1].

Q2 · 6 marks

Immediate #3=3 [1]; direct [50]=M[50]=8 [1]; indirect first reads M[51]=50 [1] then M[50]=8 [1]; indexed effective address 50+2=52 [1] then value M[52]=6 [1].

Q3 · 5 marks

First INP→ACC=7 [1]; STA FIRST→FIRST=7 [1]; second INP→ACC=4 [1]; ADD FIRST→ACC=11 [1]; OUT prints 11 [1].

Q4 · 6 marks

INP; STA A [1]; INP; STA B [1]; LDA A [1]; SUB B [1]; OUT; HLT; A DAT 0; B DAT 0 [1]; output 7 [1]. This assumes the result is representable in the chosen LMC model.

Q5 · 5 marks

BRA always branches [1]; BRZ branches when ACC=0 [1]; BRP branches when ACC is non-negative [1]; a taken branch changes the next instruction address in PC [1]; an unconditional backwards branch without exit can loop indefinitely [1].

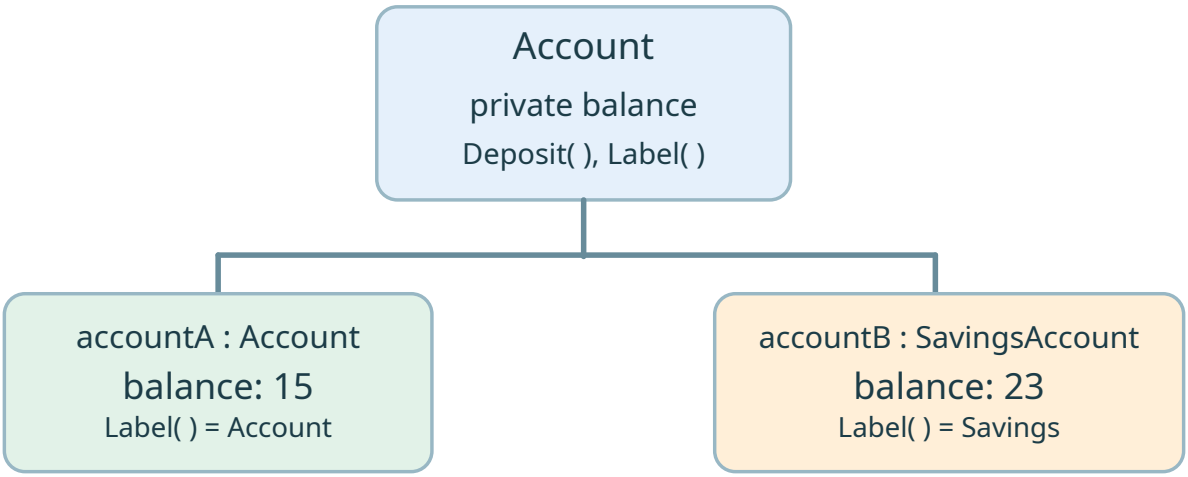
Classes, objects and encapsulation

Paper 1: explain the concepts precisely, then evaluate how the design behaves.

REMEMBER THIS / BLUEPRINT AND BANK ACCOUNTS

A **class** is a blueprint; each **object** is an actual bank account with its own balance. **Private** fields are not directly writable from outside; public methods can enforce rules. The blueprint analogy does not mean that all objects share their stored state.

CLASS IS A PLAN; EACH OBJECT HAS ITS OWN STATE



Both variables can be typed Account. Overridden Label( ) uses the real object type.

Inheritance means a subclass reuses or extends attributes and methods defined by a superclass. **Polymorphism** permits a call to the same operation to execute different overridden behaviour for different object types. A reference typed as Account may hold a SavingsAccount instance.

WORKED EXAMPLE 1 / TRACE TWO DIFFERENT INSTANCES

```
Account a = new Account(10);
Account b = new SavingsAccount(20);
a.Deposit(5); b.Deposit(3);
// a.Balance = 15, b.Balance = 23
```

Both instances are independent. If the base class defines virtual Label() and SavingsAccount overrides it, then **b.Label()** returns Savings even though b is stored in an Account variable.

Inheritance and dynamic dispatch

The same method name can lead to different behaviour at runtime.

```
public class Account {
  private int balance;
  public Account(int start) { balance=start; }
  public int Balance => balance;
  public void Deposit(int value) {
    if(value>0) balance += value;
  }
  public virtual string Label() => "Account";
}
public class SavingsAccount : Account {
  public SavingsAccount(int start) : base(start) {}
  public override string Label() => "Savings";
}
```

WORKED EXAMPLE 2 / A DIFFERENT PROBLEM

Two variables refer to **new SavingsAccount(8)** and **new Account(12)**. A call Deposit(4) is made on the first object. Predict both balances and Label() results, then explain the benefit of the virtual/override mechanism.

COMMON MISTAKE

An overridden method is not the same as overloading. Overriding changes inherited virtual behaviour in the subclass; overloading uses the same method name with a different parameter signature.

Exam practice: object-oriented languages

Q1 [4 marks] Define class, object, attribute and method. Use an Account example for one of the terms.

Q2 [6 marks] Explain how the private balance and the public Deposit method in the code protect object state. Describe what happens for Deposit(-3).

Exam practice: object-oriented languages

Q3 [6 marks] An Account variable stores a SavingsAccount object. Explain inheritance and polymorphism, and trace the result of Label() before and after Deposit(7). Why is Label() unaffected by the deposit?

Q4 [6 marks] Compare the benefits and limitations of inheritance versus writing unrelated classes that repeat the same Deposit code.

Check your answers: OOP

Q1 [4] Class is a template/definition [1]; object an instance [1]; attribute stored data/state [1]; method behaviour/operation [1].

Q2 [6] Balance is private [1] and cannot be assigned directly by outside code [1]; public Deposit controls modification [1], checks value>0 [1], valid positives add to balance [1], negative -3 makes no change [1].

Q3 [6] SavingsAccount derives from Account [1], reuses inherited members [1]; base typed reference can refer to a subclass [1]; overriding Label enables dynamic dispatch [1]; output is Savings both times [1]; Deposit changes state not object type or Label implementation [1].

Q4 [6] Inheritance shares behaviour [1], reduces repetition [1], and supports substitution/overrides [1]; tight coupling to a base design [1], deep hierarchies can complicate understanding [1], and some relationships favour composition instead [1].

1.3.1 / Compression, encryption and hashing

Remember: ZIP, lock and fingerprint: three different jobs.

COMPRESSION, ENCRYPTION AND HASHING

LOSSLESS	ENCRYPT	HASH
AAAAABBBCC	plaintext	message
5A3B2C	ciphertext	digest
Exact recovery	Key enables decryption	Not reversible

These operations solve different problems; a hash is not encryption.

ANALOGY - PICTURE IT

Compression packs a suitcase; encryption locks it; hashing produces a fixed-size digital fingerprint.

NOW BE PRECISE - WHERE THE ANALOGY STOPS

A hash is not guaranteed unique: collisions are possible. Encryption is reversible with the correct key; hashing is not decryption.

THEORY / KEY TERMINOLOGY

Lossless	The exact original can be reconstructed.
Lossy	Some information is discarded irreversibly to reduce size.
Run-length encoding	Stores a run of repeated symbols as a count and value.
Symmetric encryption	Sender and receiver use the same secret key.
Asymmetric encryption	A mathematically related public/private key pair supports specified operations.
Cryptographic hash	One-way mapping to a fixed-length digest; collisions are possible.

Compression, encryption and hashing / understand and model

Lossless methods suit program files and records that must be exact; lossy methods can suit media where acceptable perceptual detail may be removed. RLE is useful for long runs, but short runs may expand data after overhead. Dictionary techniques exploit repeated sequences. In symmetric encryption both parties need a shared secret; asymmetric schemes use distinct key roles. A password system commonly stores a salted password hash and verifies by recomputation. Hashing checks integrity but does not keep data secret.

WORKED EXAMPLE 1 / worked example

Run-length example: AAAAA BBB CC → 5A3B2C (displayed without spaces).

Verify: expand 5A, 3B, 2C to recover ten original symbols.

Trade-off: six characters here instead of ten, but an actual bit-size saving depends on the encoding and run-count storage.

Security choice: use encryption for confidentiality; use a cryptographic hash to compare fingerprints for integrity.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

Encode MMMMNNP using a basic (count, symbol) RLE representation. Explain whether your string necessarily uses fewer bits in a real file.

Scaffold: Show all three runs even where there is only one symbol.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

RLE: 4M2N1P.
Decoded result: MMMMNNP.
Size caveat: The count bytes, delimiters and encoding determine the actual size.

Compression, encryption and hashing / practice

Q1 [4 marks] Compare lossy and lossless compression in the context of student data.

Q2 [3 marks] Why might run-length encoding make some data larger?

Q3 [4 marks] Compare symmetric and asymmetric encryption.

Compression, encryption and hashing / practice

Q4 [3 marks] Challenge: explain why hashing is not a substitute for encryption when protecting a confidential message.

DIAGRAM / EXPLANATION

Model answers and marking points

Compression, encryption and hashing

Compare with your own response. Credit equivalent technically accurate statements.

Q1 / 4 marks

1. Lossless compression can reconstruct the exact original information [1].
2. It is appropriate for marks, IDs and records where any changed bit could be harmful [1].
3. Lossy compression discards some information permanently [1].
4. Lossy delivery copies may suit media, but not records requiring exact recovery; explain the context [1].

Q2 / 3 marks

1. Data with short runs or rapidly changing symbols has little repetition to exploit [1].
2. Run-length encoding must store symbols and run lengths/counts [1].
3. That overhead can use more bits than the original representation [1].

Q3 / 4 marks

1. Symmetric encryption uses a shared secret key for encryption/decryption [1].
2. It is usually suitable for efficient bulk-data encryption [1].
3. Asymmetric cryptography uses a related public/private key pair [1].
4. Explain a relevant difference, such as key distribution, performance or signature/key-exchange roles [1].

Q4 / 3 marks

1. A cryptographic hash produces a digest/fingerprint of the message [1].
2. The digest cannot be decrypted to recover the original plaintext [1].
3. Encryption is needed to protect message confidentiality while allowing an authorised holder of the key to recover it [1].

Exam tip: Compression, encryption and hashing serve different purposes. A hash does not conceal the message.

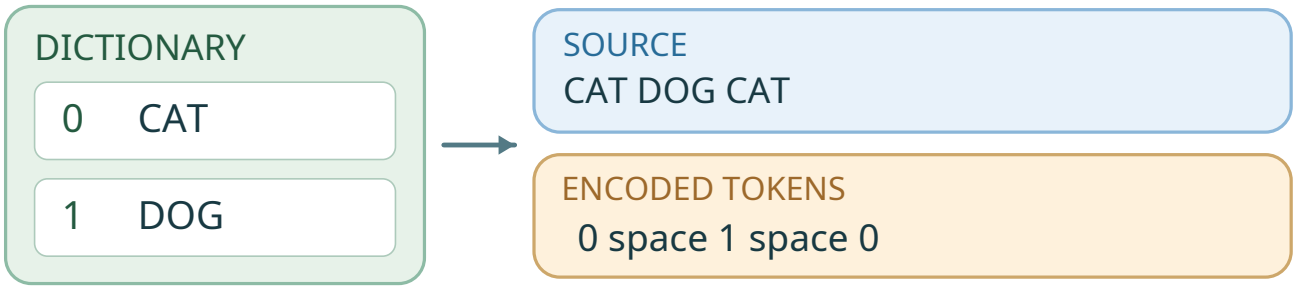
1.3.1 / DICTIONARY COMPRESSION AND BIT COSTS

Compress without losing a single character: run-length encoding and dictionary coding.

REMEMBER: A SHARED PHRASEBOOK

Dictionary coding replaces repeated words or sequences with short references. The sender and receiver need the same codebook (or an agreed way to rebuild it). The reference is not useful unless the decoder can reconstruct the original.

Dictionary coding: shared codebook



Decode by replacing 0 and 1 using the SAME dictionary; spaces remain literal.

How to measure the saving

Count all encoded data: tokens, literal separators, dictionary entries and any control information. A short-looking encoded string is not necessarily fewer bits. The calculation depends on the stated representation.

RLE: count the bits, including overhead

ORIGINAL	AAAABBBCC	9 x 8 = 72 bits
RLE	(A,4) (B,3) (C,2)	6 x 8 = 48 bits

NOW BE PRECISE

Run-length encoding (RLE) records a symbol and the length of each consecutive run. Dictionary coding reuses references for repeated sequences, even when repetitions are separated. Both are lossless if the encoding and decoder have enough information.

WORKED EXAMPLE 1: fully specified dictionary example

A fixed codebook maps 0 to CAT and 1 to DOG. Encoding CAT DOG CAT produces token sequence 0, SPACE, 1, SPACE, 0. Decoding those five tokens reproduces exactly CAT DOG CAT. The dictionary is assumed pre-shared; it must be counted if it needs transmitting.

WORKED EXAMPLE 2: different example

Codebook: 2 = BLUE, 3 = RED. Encode RED BLUE RED. The answer is 3, SPACE, 2, SPACE, 3; the two literal spaces must be preserved. What extra information would be required if the receiver did not have this dictionary?

Independent practice

Q1 [4 marks]

Encode AAAABCCCCC using maximal (symbol, run length) pairs. Decode your representation to demonstrate that it is lossless.

Q2 [5 marks]

A fixed dictionary maps 0 = SUN and 1 = MOON. Encode MOON SUN MOON using the symbols 0, 1 and the literal SPACE token. Explain how the decoder reconstructs the exact message.

Q3 [6 marks]

Under these assumptions, compare the number of bits for the source AAAABBBCC and its RLE representation: each original character is 8 bits; each RLE pair uses one 8-bit symbol and one 8-bit count. Calculate both sizes and the saving. Explain one reason RLE could increase the size of some data.

Check and improve: compression marking points

Q1 [4 marks] (A,4), (B,1), (C,5) [3: correct run lengths]; expanding the three runs gives the exact original AAAABCCCCC [1].

Q2 [5 marks] 1, SPACE, 0, SPACE, 1 [3: correct order and separators]. The decoder maps both 1 tokens to MOON and 0 to SUN [1] and preserves spaces [1].

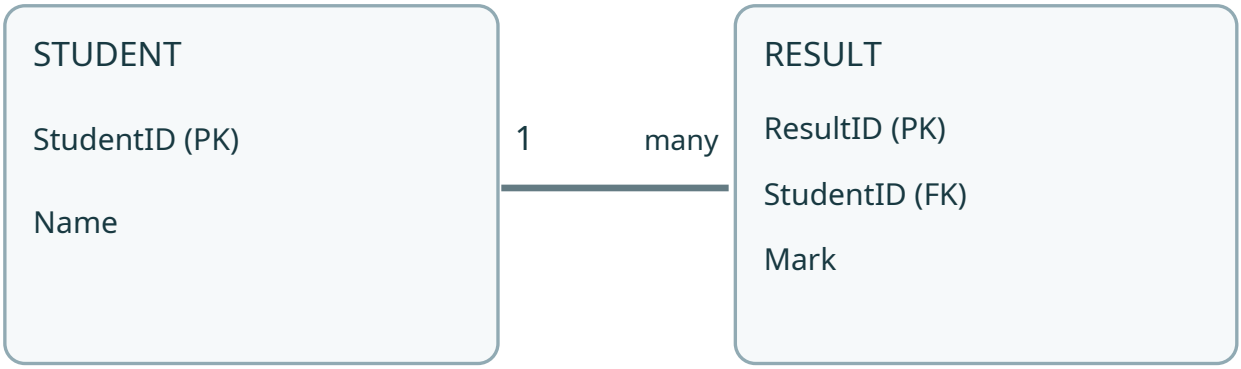
Q3 [6 marks] Source: 9 characters × 8 = 72 bits [2]. RLE: three pairs × (8 + 8) = 48 bits [2]. Saving = 24 bits [1]. If runs are very short, symbol/count overhead can exceed the source size [1].

Exam tip: state the assumptions about symbol size, count size and dictionary transmission. Different assumptions legitimately give different bit costs.

1.3.2 / Relational databases and SQL

Remember: A primary key is a unique library card number.

ONE STUDENT CAN HAVE MANY RESULTS



StudentID is unique in STUDENT but may repeat in RESULT.

ANALOGY - PICTURE IT

A student has a unique ID; a result record stores that ID so the database can link the result to the correct student.

NOW BE PRECISE - WHERE THE ANALOGY STOPS

Names are not guaranteed unique. Foreign keys may repeat and must meet referential-integrity constraints, not act as primary keys.

THEORY / KEY TERMINOLOGY

Primary key (PK)	Field or combination uniquely identifying a row.
Foreign key (FK)	Field referencing a key in a related table.
1NF	Atomic column values; no repeating groups.
2NF	In 1NF and every non-key attribute fully dependent on the whole candidate key.
3NF	In 2NF and no non-key attribute transitively dependent on a candidate key.
INNER JOIN	Combines rows that satisfy a specified matching condition.

Relational databases and SQL / understand and model

Relational databases organise records into tables linked with keys. ER diagrams record entities, relationships and cardinalities. Normalisation reduces update anomalies: ensure atomic values (1NF), remove partial dependencies from composite keys (2NF), and remove transitive non-key dependencies (3NF). Transactions aim for ACID: atomicity, consistency, isolation, durability. An index can accelerate searches but adds storage and maintenance overhead. SQL SELECT retrieves rows; WHERE filters; JOIN relates tables.

WORKED EXAMPLE 1 / worked example

Schema: Student(StudentID, Name); Result(ResultID, StudentID, Mark).

Rows: Student(12, Jo), Student(21, Eli); Result(41, 12, 78), Result(42, 21, 61).

Query: SELECT Student.Name, Result.Mark FROM Student INNER JOIN Result ON Student.StudentID = Result.StudentID WHERE Result.Mark >= 70;

Output: Jo | 78. Matching StudentID connects the rows; WHERE filters the mark.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

Design Member(MemberID, FullName) and Loan(LoanID, MemberID, DaysOverdue). Write a JOIN query listing names whose DaysOverdue is greater than zero.

Scaffold: Join using the shared MemberID and put the DaysOverdue condition in WHERE.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

Tables: Member and Loan.
Matching fields: Member.MemberID = Loan.MemberID.
Filtering: WHERE Loan.DaysOverdue > 0.

Relational databases and SQL · independent exam practice

Q1 [2 marks] Explain why Name alone should not normally be the primary key for Student.

Q2 [4 marks] Distinguish primary keys and foreign keys.

Q3 [4 marks] Explain how 3NF reduces update anomalies.

Relational databases and SQL · independent exam practice

Q4 [5 marks] State the four ACID properties and explain atomicity.

Model answers and marking points

Check the creditworthy points after completing the independent questions.

Q1 / 2 marks

1. Names may be shared by different pupils and therefore may not be unique.
2. A primary key must uniquely identify each student record.

Q2 / 4 marks

1. A primary key uniquely identifies a record in its own table.
2. A primary-key value cannot be null.
3. A foreign key refers to a valid key in another or the same table.
4. Foreign-key values may repeat in rows and link related records.

Q3 / 4 marks

1. Identify the transitive dependency: one non-key attribute depends on another non-key attribute.
2. Separate the dependent attributes into a table where their determinant is the key.
3. Store the related key in the original table rather than duplicating the dependent value.
4. A fact such as a customer name is updated in one place, avoiding inconsistent copies.

Q4 / 5 marks

1. Atomicity.
2. Consistency.
3. Isolation.
4. Durability.
5. Atomicity requires all parts of a transaction to commit together, or all to be rolled back on failure.

REMEMBER

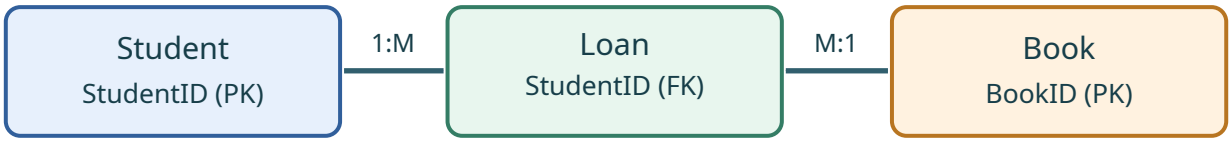
A pupil ID is a safer primary key than a name. In SQL JOIN, use the matching key fields rather than relying on names.

Data models: flat files, keys and relationships

Theory / Worked Examples

A school library can store loans as repeated rows in a flat file, or separate students, books and loans into related tables.

One student can have many loans



The Loan table also contains BookID (FK); each loan references one book.

REMEMBER THIS

A library card number, not a pupil name. Names are not reliably unique. A key is a stable identifier; a foreign key points to a matching record in another table.

Now be precise

• **Flat file:** a single table or file containing records; repeated student details can cause update anomalies.

• **Primary key:** uniquely identifies a row. **Foreign key:** links to a referenced row, supporting referential integrity. **Secondary key:** an additional indexed/search field that need not be unique.

• **ER model:** shows Student 1-to-many Loan and Book 1-to-many Loan. A composite key may also be needed in an association table. Indexing speeds some queries but adds update/storage cost.

WORKED EXAMPLE 1

Student 14 borrows Book 5 and Book 9. Model the records.

Method: Student(14, "Ada") once; Book(5,...), Book(9,...); Loan(101,14,5), Loan(102,14,9). StudentID in Loan is a foreign key.

WORKED EXAMPLE 2

Student 22 borrows two books. Explain why storing the student address in every Loan row is risky.

Method: The address would be repeated and could become inconsistent after one row is updated. Put it once in Student and use the student identifier in Loan.

COMMON MISTAKE

A foreign key does not have to be unique; otherwise the many side could not contain repeated references.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 3 marks. Distinguish primary, foreign and secondary keys.

Q2 · 4 marks. Explain two problems with repeating pupil address in every library loan record.

Q3 · 6 marks. Design Student, Book and Loan table keys for multiple loans; give two relationships.

Marking points

Check the creditworthy points after completing the independent questions.

Q1 / 3 marks

1. Primary key: a value that uniquely identifies a record.
2. Foreign key: a field referencing a valid key in a related table.
3. Secondary key: a field used for retrieval and indexing; it need not be unique.

Q2 / 4 marks

1. The address is duplicated in many loan rows.
2. The duplicate data consumes unnecessary storage.
3. Changing an address requires updating multiple rows.
4. Missing an update leaves conflicting addresses: an update anomaly.

Q3 / 6 marks

1. Student table with StudentID as its primary key.
2. Book table with BookID as its primary key.
3. Loan table with LoanID as its primary key.
4. Loan contains StudentID and BookID as foreign keys to Student and Book.
5. A student may have many loan rows; each loan refers to one student.
6. A book may have many loan rows over time; each loan refers to one book.

REMEMBER

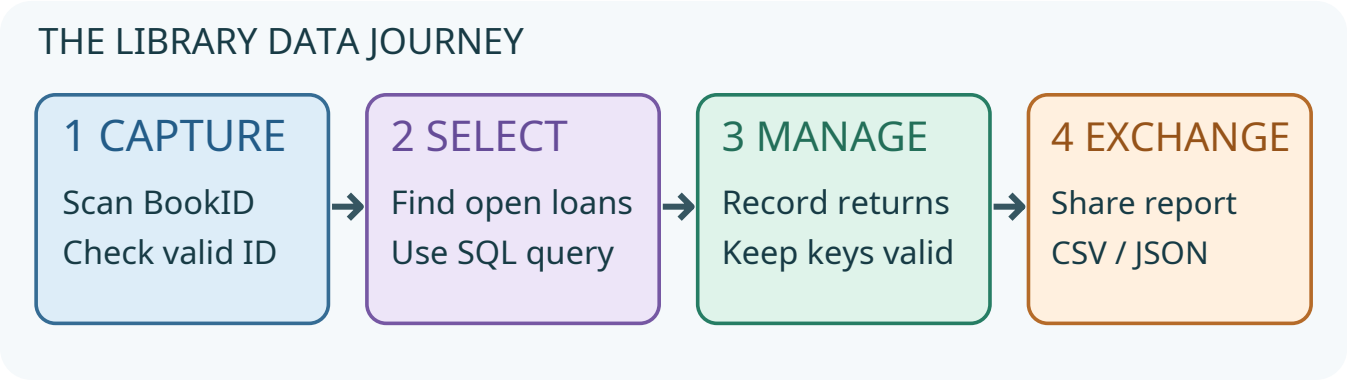
Keep the fact in one place. A foreign key links records; it does not automatically copy all the other attributes.

1.3.2(B) / DATABASES

The data journey: capture, select, manage, exchange

REMEMBER C-S-M-E

Capture → Select → Manage → Exchange. Think of a school library: receive an accurate loan record; find the loans you need; keep records correct; transfer the right information to another system.



The four stages — do not mix them up

Stage	What happens	A real method
Capture	Obtain data	Barcode scan, form, OCR, OMR
Select	Find required data	SQL SELECT with WHERE
Manage	Maintain stored data	INSERT, UPDATE, DELETE
Exchange	Send data elsewhere	CSV, JSON, secure API

NOW BE PRECISE

A **capture method** collects an input. **Selection** retrieves records matching criteria; it is not the same thing as changing those records. **Management** includes updates, constraints and backups. **Exchange** moves a suitable representation between systems.

1.3.2(B) / UNDERSTAND

Choosing the correct method

Capture: match the source and accuracy needs

Manual form. Flexible when unusual information is needed; typing and transcription errors are possible.

Barcode or QR code. Fast identification of labelled items, provided a suitable scanner and valid identifier are available.

OCR / OMR. OCR recognises printed characters; OMR detects marked positions on a structured form. Neither guarantees that the input is correct.

Automatic sensor/import. Useful for repeated measurements or large batches; calibration, formats and duplicates must be checked.

> **REMEMBER V BEFORE V**
>
> **Validation** tests a rule such as an allowed range or required field. **Verification** checks that captured data agrees with its source. Neither guarantees real-world truth.

Select: ask the database a specific question

OPEN LIBRARY LOANS

```
SELECT LoanID, BookID
FROM Loan
WHERE Returned = FALSE;
```

This query selects two columns and only those rows where Returned is false. It does not delete or modify any stored loans.

Manage: protect and update the record

A BOOK HAS BEEN RETURNED

```
UPDATE Loan
SET Returned = TRUE
WHERE LoanID = 304;
```

The WHERE clause limits the update to loan 304. A primary key identifies a record; a foreign key links it to an existing row in a related table. Permissions and backups help protect the stored information.

1.3.2(B) / UNDERSTAND

Exchanging data between systems

CHOOSING A DATA EXCHANGE FORMAT

CSV: flat rows

ID,BookID
304,B17
Simple tabular exchange

JSON: nested records

{ "loan": 304,
"books": ["B17"] }

Keeps arrays and structure

Both CSV and JSON can represent a simple flat record, but they suit different structures. CSV uses rows and columns; JSON can represent nested objects and arrays. The receiving system must understand the field meanings and data types.

A safe data transfer has several checks

Format and schema. Agree field names, expected types and required fields; reject or quarantine invalid records.

Data minimisation. Share only fields needed for the purpose. A library statistics report may not need pupils' names.

Secure transport and permissions. Use appropriate authenticated access and protected transmission, such as HTTPS, rather than sending raw personal data openly.

Recovery. Keep transaction logs and safe import/rollback procedures so a failed import does not corrupt the main database.

> **COMMON MISTAKE**
>
> **Exporting** records does not delete them. **Encryption** protects confidentiality, but it does not prove the transferred data are accurate or that access is authorised.

1.3.2(B) / WORKED APPLICATIONS

WORKED EXAMPLE 1 → WORKED EXAMPLE 2

WORKED EXAMPLE 1 — a library loan

Question. A library scans BookID B17 for borrower U05, records LoanID 304 and later produces a report for its school dashboard. Explain the data journey.

Stage	Model response
Capture	Scan B17; validate the identifier format.
Select	Look up B17 and check it exists and is not already loaned.
Manage	Insert loan 304; enforce valid BookID and BorrowerID references.
Exchange	Export only the required report fields through an authorised route.

Why the order matters: check that the referenced book and borrower exist before committing a loan; a broken foreign key would undermine referential integrity.

WORKED EXAMPLE 2 — a different problem

A sports club lends numbered equipment to pupils. The teacher captures equipment ID E12 and learner ID M04. The club then records the return and supplies a weekly count of equipment still on loan.

Capture. Which is more reliable: scanning an existing label or retyping its ID, and what is the fallback?

Select. What criteria would identify equipment that has not been returned?

Manage. How could an UPDATE mark this loan as returned without changing other records?

Exchange. What information does the weekly count need, and what information should stay private?

> **GUIDED SOLUTION**
>
> Scan the labelled ID where available; use a controlled manual fallback. Select records with **Returned = FALSE**. Update the identified loan using its primary key. Share a count of outstanding loans, not an unnecessary list of pupil names.

1.3.2(B) / INDEPENDENT WORK

Exam practice

Answer every question before turning to the marking points.

Q1 (4 marks) A school tracks music instruments. Name the stage used when it scans an instrument ID, searches outstanding loans, records a return and sends a weekly inventory file.

Q2 (6 marks) A club enters 400 equipment identifiers each term. Compare scanning a barcode with typing IDs manually. Explain two different checks that could reduce incorrect stored identifiers.

1.3.2(B) / INDEPENDENT WORK

Exam practice

Use the following schema: Loan(LoanID, BookID, BorrowerID, Returned). Returned is a Boolean field.

Q3 (6 marks) Write SQL to return LoanID and BookID for all unreturned loans. Then write SQL that changes only LoanID 304 to Returned = TRUE.

Q4 (6 marks) A service transfers library records to another application. Some records contain a list of borrowed books. Compare CSV and JSON, choose a suitable format, and explain two safeguards during exchange.

1.3.2(B) / SEPARATE MARKING POINTS

Check your answers

Q1 / 4 marks

Scan = capture [1]; search = select [1]; record a return = manage [1]; send an inventory file = exchange [1].

Q2 / 6 marks

Barcode capture can reduce repeated typing [1] and speed up high-volume identification [1], but labels/readers can fail and a manual fallback may be needed [1]. Manual entry is flexible but creates more transcription risk [1]. Validation checks allowed form/length or existence [1]; verification compares the captured ID to its source, e.g. a second scan or check [1]. Equivalent well-explained responses earn credit.

Q3 / 6 marks

SELECT LoanID, BookID FROM Loan WHERE Returned = FALSE; Award for SELECT required fields [1], correct table [1], WHERE criterion [1]. **UPDATE Loan SET Returned = TRUE WHERE LoanID = 304;** Award for UPDATE the correct table [1], SET assignment [1], WHERE on the identified loan [1]. Accept equivalent valid SQL syntax.

Q4 / 6 marks

CSV stores flat rows/columns [1]; JSON can represent nested arrays and records [1]. Choose JSON for variable-length borrowed-book lists with an explicit reason [1]. Appropriate safeguards include validated schema/types [1], authenticated/secure transfer [1], and data minimisation or permissions [1]. Equivalent specific protections accepted.

SELF-CHECK

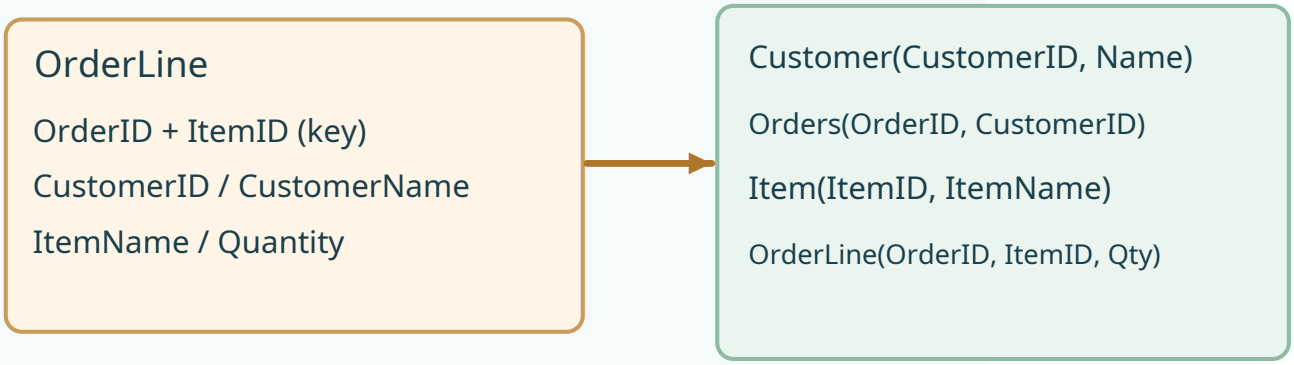
Can you give a real example of **each** stage without confusing retrieval, modification and transmission? If not, repeat the worked library journey in your own words.

1.3.2 / Databases in practice

Normalisation, SQL and reliable transactions

REMEMBER THIS Think of a library catalogue: each fact belongs on one shelf and is linked by a key.

FROM ONE CROWDED TABLE TO 3NF



NOW BE PRECISE Analogy limit: a database is not a literal filing cabinet; keys and constraints enforce relationships programmatically.

KNOW IT PRECISELY

1NF. Atomic values; no repeating groups.

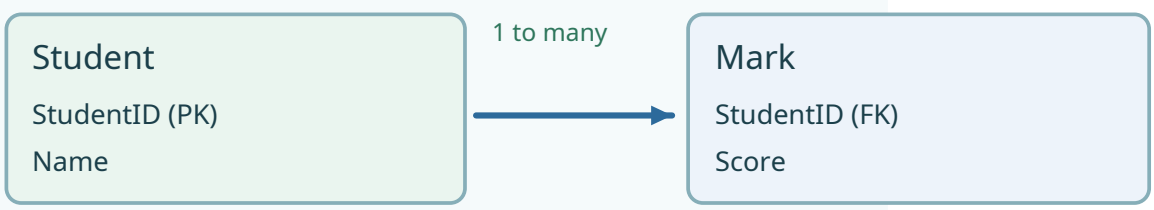
2NF. 1NF, with non-key attributes dependent on the whole composite key.

3NF. 2NF, with no transitive dependency of non-key attributes on a key.

Foreign key. A field referencing a candidate/primary key in another table; constraints support referential integrity.

ACID. Atomicity, Consistency, Isolation and Durability.

PRIMARY KEY FOREIGN KEY



Databases in practice / Worked method

Worked examples and a different guided problem.

UNDERSTAND IT

- Assume each order has one customer and each item has one name. In OrderLine(OrderID, ItemID, CustomerID, CustomerName, ItemName, Qty), the key is (OrderID, ItemID).
- At 2NF separate data dependent only on OrderID or ItemID. At 3NF move CustomerName to Customer, because CustomerID determines CustomerName.
- Capture data with a validated form/import; select with SQL; manage using updates, transactions and backups; exchange using controlled exports or APIs.

WORKED EXAMPLE 1 / SELECT WITH JOIN

Student(StudentID,Name) and Mark(MarkID,StudentID,Score) have a one-to-many relationship. Use the foreign key to join names to marks above 80.

SQL TO TRACE

SELECT Student.Name, Mark.Score FROM Student INNER JOIN Mark ON Student.StudentID = Mark.StudentID WHERE Mark.Score > 80;

WORKED EXAMPLE 2 / TRANSACTION

A school booking first checks capacity, then inserts the booking and reduces availability. Use one transaction so either both changes commit, or both roll back if an error occurs.

Normalisation: 1NF to 3NF

Trace the mechanism carefully; these examples are not questions to memorise.

IDENTIFY THE FUNCTIONAL DEPENDENCIES

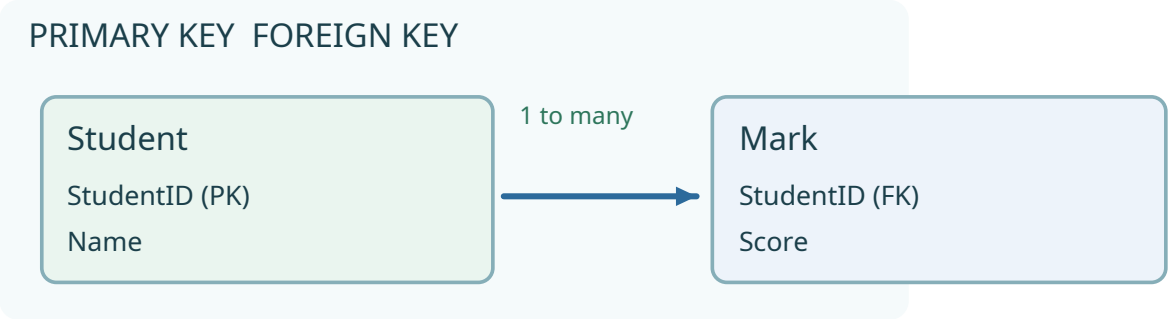
Assume each order has one customer and each item has a fixed name. (OrderID, ItemID) identifies an order line. OrderID determines CustomerID; CustomerID determines CustomerName; ItemID determines ItemName; the full key determines Qty.

Stage	Result
1NF	OrderLine(OrderID, ItemID, CustomerID, CustomerName, ItemName, Qty): atomic values.
2NF	Orders(OrderID, CustomerID, CustomerName); Item(ItemID, ItemName); OrderLine(OrderID, ItemID, Qty).
3NF	Customer(CustomerID, CustomerName); Orders(OrderID, CustomerID); Item(ItemID, ItemName); OrderLine(OrderID, ItemID, Qty).

WHY IT MATTERS Removing partial and transitive dependencies prevents repeated names and makes updates less error-prone.

SQL: retrieve and modify data

Trace the mechanism carefully; these examples are not questions to memorise.



SQL / carefully scoped conditions

```
SELECT Student.Name, Mark.Score
FROM Student INNER JOIN Mark
 ON Student.StudentID = Mark.StudentID
WHERE Mark.Score >= 70;
INSERT INTO Mark VALUES (9,3,82);
UPDATE Mark SET Score = 85 WHERE MarkID = 9;
DELETE FROM Mark WHERE MarkID = 9;
SELECT Name FROM Student WHERE Name LIKE 'A%';
```

REMEMBER DELETE removes selected rows; DROP TABLE removes the whole table definition. WHERE conditions prevent unwanted mass changes.

SAFE TRANSACTION PROCESSING

An order can insert a row and update stock in one transaction. Commit both only after checking constraints; otherwise roll back. Isolation and row locking protect concurrent updates. Durability protects committed records.

Databases in practice / Exam practice

Complete the questions independently. Use the space provided for your working.

Q1 4 MARKS

Explain why OrderLine(OrderID, ItemID, CustomerID, CustomerName, ItemName, Qty) is not in 2NF when its composite key is (OrderID,ItemID). Name two appropriate decompositions.

Q2 5 MARKS

Write SQL to display Student.Name and Mark.Score for marks of at least 70, using Student(StudentID,Name) and Mark(MarkID,StudentID,Score).

Databases in practice / Exam practice

Complete the questions independently. Use the space provided for your working.

Q3 6 MARKS

Explain referential integrity and why an online order should use a transaction with ACID properties and record locking.

Databases in practice / Check your answers

Use these mark points after completing the independent questions.

Q1 / 4 MARKS

[1] CustomerID/CustomerName depend on OrderID, not the whole composite key.

[2] ItemName depends on ItemID, not the whole composite key.

[3] Create a separate Orders table linked by OrderID.

[4] Create an Item table linked by ItemID (and OrderLine retaining both keys and Qty).

Q2 / 5 MARKS

[1] SELECT Student.Name, Mark.Score

[2] FROM Student

[3] INNER JOIN Mark

[4] WHERE Mark.Score >= 70

[5] Join condition compares Student.StudentID with Mark.StudentID (accept equivalent aliases).

Q3 / 6 MARKS

[1] A foreign key must refer to an existing linked key (unless null is permitted).

[2] Atomicity: all related writes commit or none do.

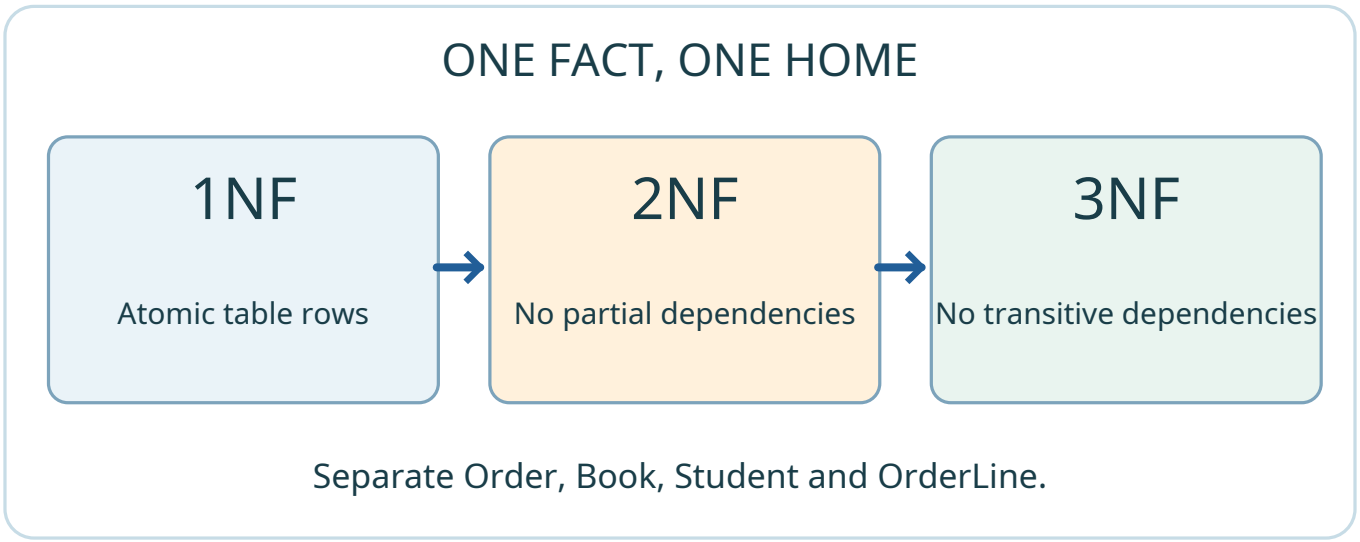
[3] Consistency: constraints remain satisfied.

[4] Isolation: concurrent operations do not expose inconsistent intermediate changes.

[5] Durability: committed records persist after a failure.

[6] Record locking can prevent concurrent conflicting updates or lost updates.

Normalisation, step by step



A library ordering spreadsheet starts with atomic rows and columns OrderID, BookID, StudentID, StudentName, BookTitle, Qty. A line is uniquely identified by the composite key (OrderID, BookID).

WORKED EXAMPLE 1 / follow each dependency

Dependencies: (OrderID, BookID) → Qty; BookID → BookTitle; OrderID → StudentID; StudentID → StudentName. The first uses the composite key. BookTitle depends on only part of that key; StudentName depends transitively through StudentID.

1NF: one book per row; fields are atomic. 2NF: remove partial dependencies (e.g., BookID → BookTitle) into their own tables. 3NF: remove the transitive dependency StudentID → StudentName by separating Student.

FINAL 3NF TABLES

Student(StudentID PK, StudentName)
Book(BookID PK, BookTitle)
Order(OrderID PK, StudentID FK)
OrderLine(OrderID FK, BookID FK, Qty) with composite PK (OrderID, BookID).

WORKED EXAMPLE 2 / spot the dependency

A new BookTitle is corrected in one Book row. Why is that preferable to updating every order line? Because a single authoritative value prevents repeated inconsistent book titles and reduces update anomalies.

SQL: join, filter and change safely

Use the following small school-library database. The Returned field uses 0 = not returned and 1 = returned.

Student	Book	Loan
1 Ada; 2 Ben; 3 Cleo	10 Algorithms; 11 Networks; 12 Databases	90: S1/B10/0
		91: S2/B11/0
		92: S1/B12/1

WORKED EXAMPLE 1 / select open loans with student and book names

```
SELECT Student.Name, Book.Title
FROM Student
INNER JOIN Loan ON Student.StudentID = Loan.StudentID
INNER JOIN Book ON Loan.BookID = Book.BookID
WHERE Loan.Returned = 0;
```

The result is Ada / Algorithms and Ben / Networks. Loan 92 is not returned because its Returned value is 1. The JOIN conditions match keys; WHERE filters rows.

WORKED EXAMPLE 2 / change one loan without changing all rows

```
UPDATE Loan SET Returned = 1 WHERE LoanID = 91;
```

This modifies just the row with LoanID 91. Omitting WHERE could set Returned = 1 for every loan. Use a key in the condition when a particular record is intended.

Other commands students must distinguish

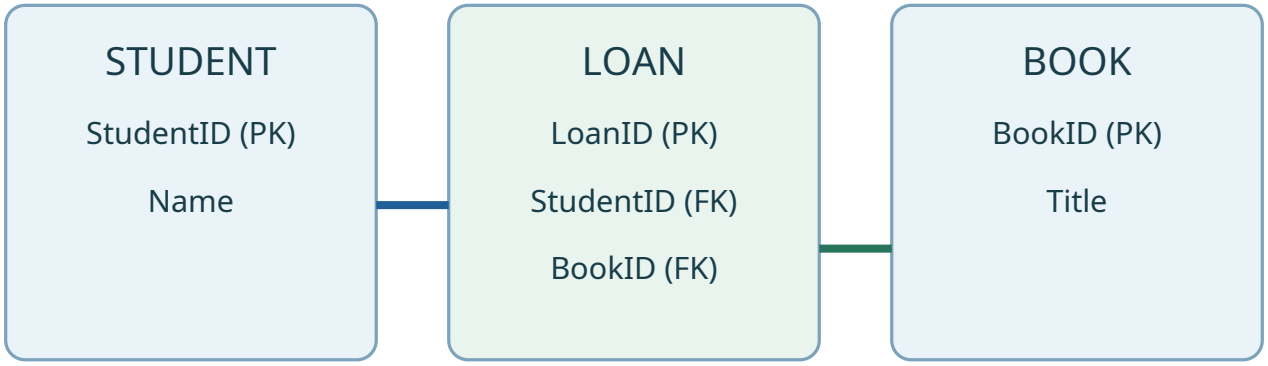
```
INSERT INTO Book (BookID, Title) VALUES (13, 'Python');
DELETE FROM Loan WHERE LoanID = 90;
SELECT Name FROM Student WHERE Name LIKE 'A%';
-- DROP TABLE Loan removes the table structure itself.
```

Referential integrity: keys that agree

REMEMBER THIS / A LIBRARY LOAN NEEDS REAL RECORDS

A loan entry cannot point to a student or book that the database has never heard of. The foreign key is the reference; the primary key identifies the existing parent record. Referential integrity prevents orphan references.

FOREIGN KEYS MUST POINT TO REAL ROWS



A missing parent ID causes a foreign-key violation.

WORKED EXAMPLE 1 / diagnose the failed insert

Student contains IDs 1 and 2. Book contains IDs 10 and 11. A proposed Loan row is (LoanID 93, StudentID 8, BookID 10). Student 8 does not exist, so a foreign-key constraint rejects the insert (assuming foreign-key enforcement). The book reference is valid, but both must be valid.

WORKED EXAMPLE 2 / delete a referenced student

Loan 90 references StudentID 1. A request deletes Student 1. Should the database allow it?

CHECK THE POLICY

With a RESTRICT/NO ACTION relationship, deletion is refused until referencing loans are handled. With deliberately configured CASCADE, the referencing loans may be deleted too. State the policy; do not assume all DBMSs silently delete children.

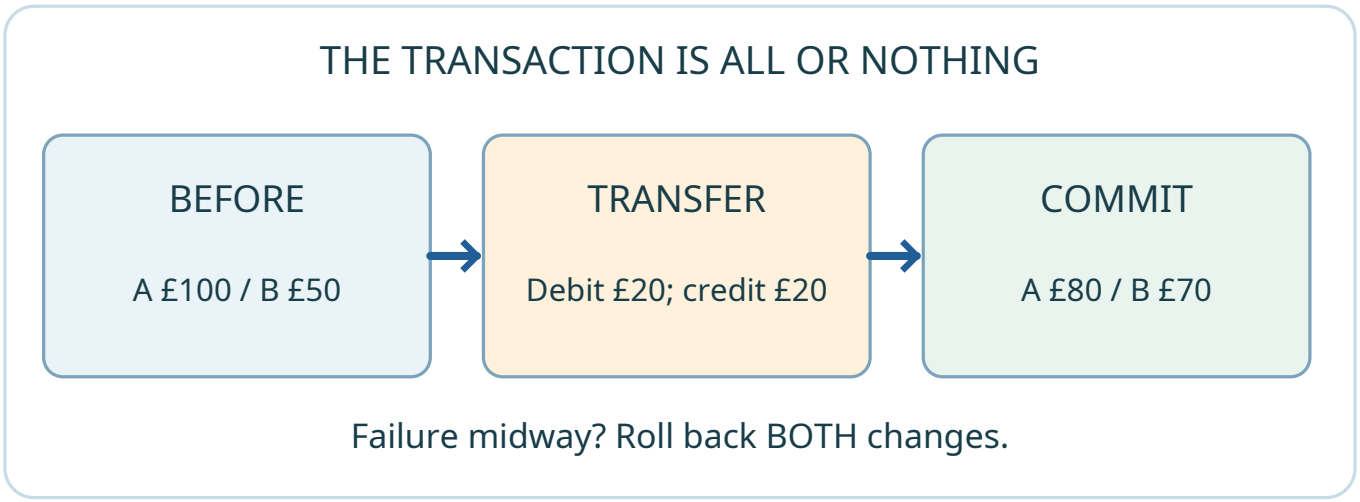
How this differs from validation

Validation checks whether an entered value follows rules, such as a positive integer ID. Referential integrity goes further: does that referenced row actually exist? An ID can be correctly formatted yet invalid as a foreign key.

Transactions, locking and ACID

REMEMBER THIS / AN ALL-OR-NOTHING BOOKING

A two-stage transaction is like a booking that must reserve a ticket and record the payment. Completing only one half leaves the data inconsistent. The database should commit both related changes or neither.



ACID	Precise meaning
Atomicity	All operations in the transaction succeed, or all are rolled back.
Consistency	Constraints and required invariants remain satisfied.
Isolation	Concurrent transactions do not corrupt one another’s intermediate work.
Durability	After commit, changes persist even if a failure follows.

WORKED EXAMPLE 1 / last seat double-booking

Two customers see one remaining seat. Without isolation, both may read 1 and each try to reserve it. A suitable locking/isolation strategy ensures the read-and-update is protected, so only one reservation can succeed.

WORKED EXAMPLE 2 / a transfer fails halfway

A bank transfer deducts £20 from A but the credit to B fails. Atomicity requires rollback: A returns to £100, B remains £50. The total remains £150. A committed transfer would leave £80 and £70.

Vocabulary: A transaction is one logical unit of work. Record locking can prevent conflicting updates; redundancy is duplicated data.

Databases / exam practice

Complete the questions independently. Use diagrams, tables or working where the question requires them.

Q1 · 4 marks A Loan table has StudentID as a foreign key referencing Student.StudentID. Explain what happens when a loan references a missing student and why.

Q2 · 6 marks Given the library schema above, write SQL that returns the names of pupils with an unreturned loan. Use a JOIN and appropriate filter; state the returned names.

Use clear indentation and label any variable changes.

Databases / exam practice / continued

Q3 · 6 marks Show the 3NF tables needed for OrderID, BookID, StudentID, StudentName, BookTitle and Qty, using the composite line key (OrderID, BookID). Explain one dependency eliminated.

Q4 · 6 marks Explain how atomicity and isolation would prevent a failed two-account transfer or two customers booking the last available place from corrupting data.

Databases / exam practice / continued

Q5 · 4 marks Explain how record locking helps with concurrent updates and how redundant copies of StudentName across many Loan rows can cause an update anomaly.

Databases / marking points

Check each question only after completing the independent practice.

Q1 / 4 marks

• StudentID in Loan is a foreign key [1].

• It must match an existing value in the Student primary key [1].

• Missing StudentID is an invalid reference / orphan row [1].

• The DBMS rejects the change under enforced referential integrity, unless a valid parent is created first [1].

Q2 / 6 marks

• SELECT a student-name field [1] FROM Student [1].

• JOIN Loan [1] using matching StudentID keys [1].

• WHERE Returned = 0 [1].

• Result comprises Ada and Ben, once each for the two open loans in the supplied data [1].

Q3 / 6 marks

• Student(StudentID PK, StudentName) [1].

• Book(BookID PK, BookTitle) [1].

• Order(OrderID PK, StudentID FK) [1].

• OrderLine(OrderID FK, BookID FK, Qty) [1].

• Composite key (OrderID, BookID) of OrderLine [1].

• Explain a removed partial dependency BookID→BookTitle or transitive StudentID→StudentName [1].

Q4 / 6 marks

• Atomicity means related changes commit together or all are undone [1].

• Debit without credit must therefore be rolled back on failure [1].

• Isolation prevents concurrent transactions interfering with intermediate state [1].

• Both customers might otherwise read the same final available seat [1].

• Suitable locking/serialization or atomic conditional update allows only one success [1].

• Stored seat counts and reservations therefore remain consistent [1].

Q5 / 4 marks

• A suitable record lock controls concurrent write access [1].

• This can prevent conflicting updates or lost changes [1].

• Repeated StudentName values create redundant copies [1].

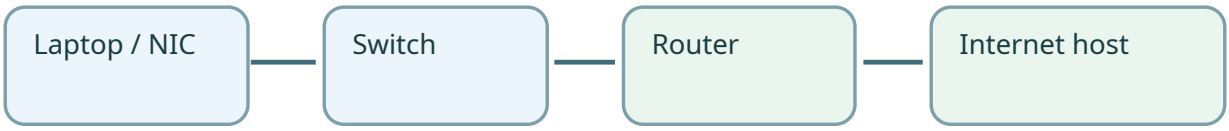
• Changing only some copies creates inconsistent names / an update anomaly [1].

1.3.3 NETWORKS / Networks, protocols and network security

REMEMBER THIS / THE SCHOOL ROAD NETWORK

NIC = vehicle access; switch = local junction; router = junction between networks. The model helps explain traffic movement but cannot replace protocol and addressing detail.

A pupil opens a website from the school LAN



Switch: local Ethernet frames

Router: between networks

Return traffic may follow a different route.

Blue: school LAN / Green: between networks

Now be precise

Component	Role
NIC	Interface that connects a device to a network.
Switch	Forwards LAN frames, generally using MAC addresses.
Router	Forwards IP packets between networks.
Wireless access point	Connects Wi-Fi clients to the local network.

MISCONCEPTION

A switch and router do not perform the same forwarding task. A WAN is not defined by using Wi-Fi; it spans a wide geographical area.

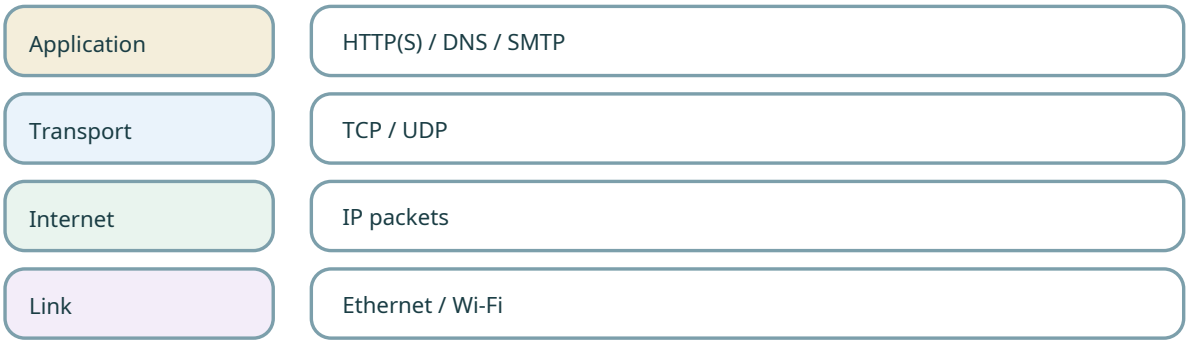
Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

Protocol	Agreed rules for communication between network devices or applications.
NIC	Network Interface Controller: connects a device to a network.
Switch	Forwards frames within a LAN, generally by MAC address.
Router	Forwards IP packets between networks.
IP address	Logical address used for routing packets across IP networks.
MAC address	Link-layer identifier used to deliver frames on a local network.
DNS	Domain Name System: resolves names to records such as IP addresses.
Packet switching	Dividing data into packets that may travel independently.

The TCP/IP stack / four different jobs

Four TCP/IP layers: each has a different job



WORKED EXAMPLE 1 / Tracing protocols

For https://library.example, DNS resolves the name to an IP address (unless a suitable result is cached). TCP provides reliable transport to the service; TLS secures the connection; HTTPS transfers the requested web content.

WORKED EXAMPLE 2 / new example

A teacher sends an email message via SMTP. Identify the application, transport and internet protocols involved.

GUIDED ANSWER

Application: SMTP. Transport: normally TCP for SMTP transfer. Internet: IP for addressing and forwarding. Link: Ethernet or Wi-Fi carries frames on each local hop.

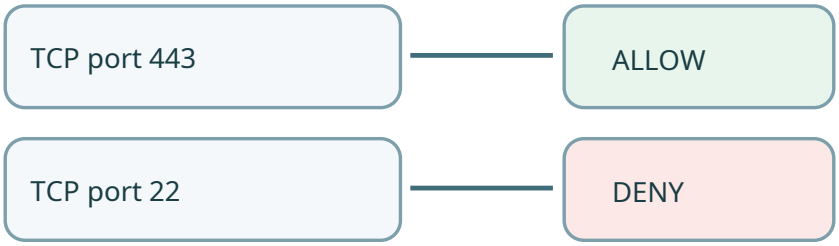
REMEMBER THE LIMIT

A layer is not one particular piece of hardware; the four-layer model describes functions and protocols. DNS resolves names, not the encryption of web pages.

Packet switching and defence in depth

Packet switching shares network links among messages. Separate IP packets can take different paths. TCP adds ordered reliable delivery where the application needs it; IP itself does not guarantee delivery or order.

Firewall: a defined traffic rule, not a safety guarantee



Rule applies to destination port in this simplified example.

WORKED EXAMPLE 1 / network policy

An inbound TCP request addressed to port 443 matches the illustrated ALLOW rule. A request to port 22 matches DENY. Rules use more than ports in real deployments.

WORKED EXAMPLE 2 / independent transfer

A forward proxy is used for pupil browsing. Explain the route when a site is permitted and when it is blocked.

MODEL THE PATH

Allowed: client → proxy → website (or suitable cached response). Blocked: proxy applies policy and does not forward the request. A proxy is not proof that permitted content is safe.

LAN/WAN; TCP/IP layers; DNS; packet vs circuit switching; switch/router; firewalls, proxies and open standards.

Networks, protocols and network security / exam practice

Q1 [4 marks] Distinguish a LAN from a WAN. Explain what a switch and a router each do.

Q2 [6 marks] A pupil loads a webpage from an external server via a wireless access point, switch and router. Explain the path and name one relevant protocol at each of the application, transport and internet layers.

Networks, protocols and network security / exam practice

Q3 [4 marks] Two TCP segments arrive in the order 2, then 1. Explain how TCP can present data in order and give one benefit of packet switching.

Q4 [6 marks] A school firewall permits inbound TCP port 443 and blocks inbound TCP port 22. Explain the decisions and why the allow rule cannot guarantee the server is secure.

Model answers and marking points

Networks and protocols

Compare with your own response. Credit equivalent technically accurate statements.

Q1 / 4 marks

1. A LAN serves a relatively local site or area [1].
2. A WAN spans larger areas and links more distant networks [1].
3. A switch forwards frames within a LAN using link-layer information [1].
4. A router forwards packets between IP networks [1].

Q2 / 6 marks

1. The laptop/NIC communicates over Wi-Fi with an access point [1].
2. Frames pass through the local network/switch towards the router [1].
3. The router forwards packets towards the external server through other networks [1].
4. An appropriate application-layer example is HTTP or HTTPS [1].
5. An appropriate transport-layer example is TCP [1].
6. An appropriate internet-layer example is IP [1].

Q3 / 4 marks

1. TCP sequence numbers identify the correct byte-stream order [1].
2. TCP may buffer segments arriving out of order [1].
3. It presents/reassembles data in the correct order to the application [1].
4. Packet switching shares links efficiently or lets packets use alternative routes [1].

Q4 / 6 marks

1. The inbound rule permits traffic matching destination TCP port 443 [1].
2. The inbound rule blocks traffic matching destination TCP port 22 [1].
3. The rules filter by defined direction, protocol and destination port [1].
4. A permitted connection can still target a vulnerable web service [1].
5. Patching and secure configuration reduce the attack surface [1].
6. Authentication, authorisation, logging or other suitable controls remain necessary [1].

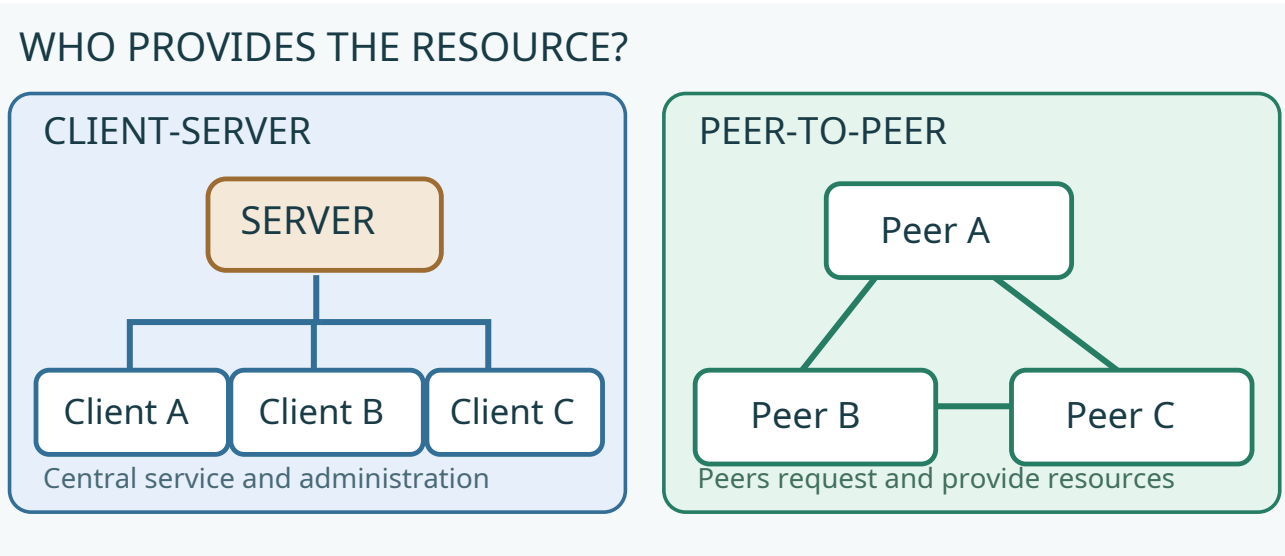
Exam tip: Port 443 is commonly HTTPS and port 22 is commonly SSH; a port number does not make traffic intrinsically safe.

1.3.3(E) / NETWORK MODELS

Client-server and peer-to-peer

REMEMBER: WHO DOES THE JOB?

Client-server: a managed service provides files, logins or other resources to clients. **Peer-to-peer:** devices can both request and provide resources. These describe **roles**, not how network cables are arranged.



Feature	Client-server	Peer-to-peer
Responsibility	Central service administration	Distributed among peers
Shared resource	Usually maintained by server(s)	Provided by participating peers
Accounts/backup	Can be centrally controlled	Must be coordinated across peers
Costs	Server and administration costs	Can be simple for small groups

WATCH THE MISCONCEPTION

Client-server does **not** necessarily mean one physical server. A service can use several servers for resilience. Peer-to-peer does **not** mean that every machine must have its own direct cable to every other machine.

1.3.3(E) / UNDERSTAND

What the choice changes

Client-server: strengths and limitations

Central control. Access rights, shared files, software updates and backup policies can be administered consistently.

Potential costs. Dedicated service capacity, system administration and security measures cost money and require expertise.

Resilience. A service outage can affect many clients, but replication and failover can reduce this risk. It is not inherently only one server.

Peer-to-peer: strengths and limitations

Simplicity. A small temporary group can share selected resources without deploying a dedicated service server.

Distributed availability. A file shared by only one peer becomes unavailable when that peer leaves; versions and backup may diverge.

Security management. Each peer needs appropriate controls. Consistency is more difficult as the group grows.

A MODEL IS NOT A TOPOLOGY

Service model	Physical topology
WHO stores files? WHO manages logins?	HOW are devices linked? Star, mesh, bus or ring

EXAM TIP

Do not say “client-server is better because there are more computers”. Explain **why** the scenario needs central permissions, consistent backups or reliability. Recommend peer-to-peer where its lower setup overhead genuinely matters.

1.3.3(E) / APPLY IN CONTEXT

WORKED EXAMPLE 1 → WORKED EXAMPLE 2

WORKED EXAMPLE 1 — shared school files

Question. A school uses 80 staff laptops and needs common accounts, protected files and a reliable daily backup. Which network model is more suitable, and what is the main trade-off?

Step 1 — requirement. Accounts and confidential shared files must be managed consistently.

Step 2 — recommendation. Client-server gives managed access controls and a shared storage/backup service.

Step 3 — limitation. There are server/administration costs, and service outages must be managed, for example through resilient design.

WORKED EXAMPLE 2 — different context

Four students at a temporary media workshop share draft video clips for one afternoon. No central account directory or long-term shared storage is required.

Choose. Which model could be quick to organise, assuming suitable software and connectivity?

Justify. Where are the files stored, and what happens if the supplying device disconnects?

Evaluate. What changes if the workshop becomes a permanent department with sensitive shared records?

> **GUIDED RESPONSE**
>
> Peer-to-peer can suit an ad-hoc group where each participant offers resources. A clip hosted on one peer may disappear if that peer leaves. A permanent, sensitive service would favour stronger central management and backup.

1.3.3(E) / INDEPENDENT WORK

Exam practice

The network model concerns which device provides a service. A star topology, for example, can support either model.

Q1 (4 marks) Describe client-server and peer-to-peer networking. State one practical difference in where shared resources are provided.

Q2 (6 marks) A college wants central sign-in, shared coursework and nightly backups. Explain two advantages and one disadvantage of a client-server approach for this college.

1.3.3(E) / EXTENDED RESPONSE

Exam practice

Q3 (9 marks) A small design studio has five staff and expects to grow to 40. It handles confidential client files, but the owner wants low costs. Discuss the suitability of client-server and peer-to-peer networks. Give a justified recommendation.

Aim to compare both models, use the studio’s requirements and reach a reasoned conclusion.

Model answers and marking points

Client-server and peer-to-peer networks

For the 9-mark evaluation, use holistic judgement; points are illustrative and not an official OCR level-of-response grid.

Q1 / 4 marks

1. Clients request services/resources offered by a server process [1].
2. The server can provide centralised administration or shared resources [1].
3. In peer-to-peer networking, participating devices may act as clients and providers [1].
4. Services/resources may be shared across peers rather than a single central provider [1].

Q2 / 6 marks

1. Central accounts allow consistent permissions/managed sign-in [1] and explain their usefulness to college users [1].
2. A shared file service enables managed coursework or organised backups [1] and explain its usefulness [1].
3. Identify a relevant disadvantage such as cost, maintenance or service dependency [1] and explain its consequence [1].

Q3 / extended response / 9 marks

1. A relevant advantage of client-server for confidential records and controlled permissions.
2. A supported explanation of central backups/version management.
3. A client-server cost/availability disadvantage linked to the studio.
4. A relevant peer-to-peer advantage for a five-person team.
5. A peer-to-peer confidentiality/management limitation.
6. A peer availability/version control concern.
7. Apply at least two points to the shift from five to forty staff.
8. Compare trade-offs rather than presenting one-sided lists.
9. Reach a supported recommendation and recognise a practical safeguard/qualification.

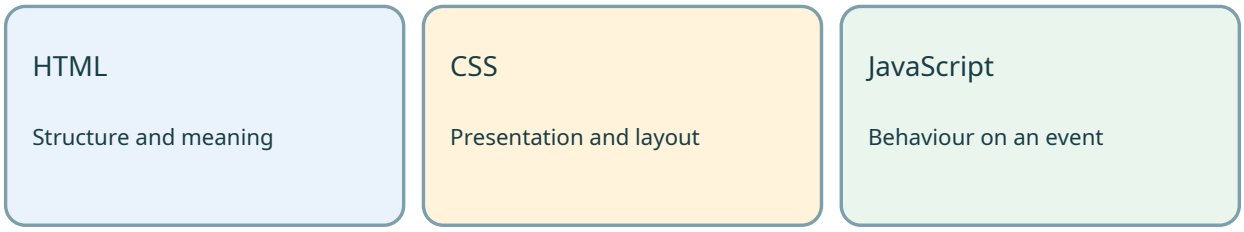
Exam tip: For Q3, use the nine indicators as guidance, not a rigid checklist: 7–9 for sustained applied evaluation and justified judgement; 4–6 for balanced but uneven analysis; 1–3 for limited descriptive points.

1.3.4 WEB TECHNOLOGIES / Web technologies and client-server processing

REMEMBER / THE WEB THEATRE

HTML builds the stage, CSS chooses the costumes, and JavaScript responds to cues. Use the analogy to distinguish responsibilities; the DOM is the actual structured document model.

One webpage: three different jobs



Remember: DOM is the document model JavaScript can update.

HTML form and GET request

A form collects fields. A label connects to its input through matching for and id; the input name becomes the submitted key.

WORKED EXAMPLE 1 / Working with HTML forms

If a GET form submits a field name="q" with value binary to /find, its URL is /find?q=binary. GET data appears in the URL and is unsuitable for secret credentials.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

HTML	Markup language defining a webpage's structure and meaning.
CSS	Rules controlling the presentation of web content.
JavaScript	Programming language often used for interactive web behaviour.
DOM	Document Object Model: structured representation of a webpage for scripts.
Client-side	Processing performed in a browser or client application.
Server-side	Processing performed on a server that responds to client requests.
HTTP GET	Method typically requesting resources; parameters can appear in URLs.
HTTP POST	Method commonly submitting data in the request body.

CSS, JavaScript and the DOM

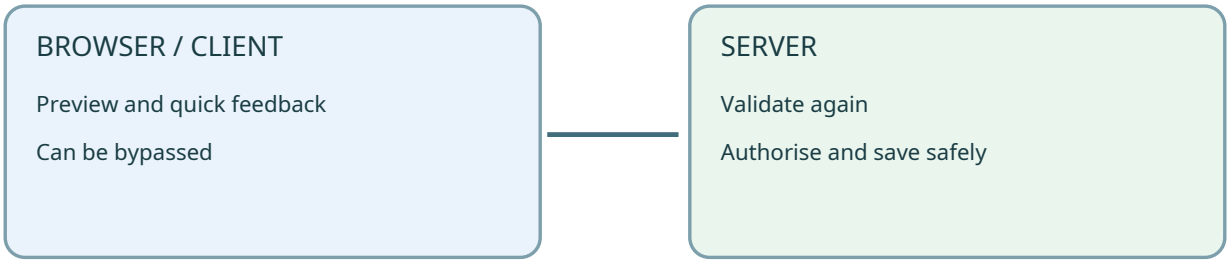
The browser loads document structure, applies CSS styling, and executes JavaScript when appropriate events occur. A click handler runs on a click event, not automatically when the handler is registered.

WORKED EXAMPLE 1 / TRACE THE EVENT

```
let count = 0;
const button = document.getElementById("go");
button.addEventListener("click", () => { count++; });
```

After two clicks, count = 2. No click means count remains zero.

Client helps; server enforces authoritative rules



Never trust browser-only validation for a security decision.

WORKED EXAMPLE 2 / build a secure booking

A pupil enters 1000 seats even though the HTML input says max="5". Explain why the server must refuse it, even if the browser normally blocks it.

GUIDED EXPLANATION

Client controls can be bypassed. The server must authenticate/authorise, validate values against business rules, and protect authoritative records and transactions.

Search, crawling, indexing and PageRank

A crawler follows links and discovers documents. Indexing organises the content for retrieval. PageRank estimates importance through a directed web-link network; a linking page divides its contribution over outgoing links.

WORKED EXAMPLE 1 / Understanding PageRank

In a simplified undamped model, if page A has rank 0.40 and two outgoing links, it contributes 0.20 to each. A full PageRank calculation also requires other incoming contributions, damping and iterations.

Exam phrase	Precise meaning
Client-side	Processing on the user device / browser.
Server-side	Processing on a remote server enforcing authoritative rules.
Crawling	Finding and retrieving web pages.
Indexing	Organising documents and terms for fast retrieval.
PageRank	Importance score influenced by link structure.

COMMON MISTAKES

More incoming links alone does not calculate PageRank. Client-side validation is not secure server validation. CSS changes presentation, not page structure.

Web technologies and client-server processing / exam practice

Q1 [4 marks] Explain different roles of HTML, CSS and JavaScript in a webpage. What does the DOM represent?

Q2 [6 marks] Write a small HTML GET form with a label and a text input named topic that submits to /find. State the resulting URL query when the user types binary.

CODE / PSEUDOCODE

Web technologies and client-server processing / exam practice

Q3 [6 marks] A site checks that a booking quantity is between 1 and 5 in the browser. Evaluate why this is insufficient for placing a secure order and describe the server-side responsibilities.

Q4 [5 marks] Explain the difference between web crawling and indexing. A page A has rank 0.60 and three outgoing links; calculate its contribution to each target in a simplified undamped link model.

WORKING / METHOD

Model answers and marking points

Web technologies, client/server processing and search

Compare with your own response. Credit equivalent technically accurate statements.

Q1 / 4 marks

1. HTML identifies page structure and semantic elements [1].
2. CSS controls visual presentation [1].
3. JavaScript enables behaviours/interaction [1].
4. The DOM is the object/tree model of document nodes that scripts may inspect or change [1].

Q2 / 6 marks

1. An opening <form> element uses method=GET [1].
2. The form action targets /find [1].
3. A visible <label> is included [1].
4. The text <input> has name=topic [1].
5. Label and input are associated correctly, for example matching for/id [1].
6. Submitting binary produces a query equivalent to /find?topic=binary [1].

Q3 / 6 marks

1. Browser-side validation helps the user but can be bypassed [1].
2. A request may be forged or altered outside the browser [1].
3. The server must independently check the quantity range [1].
4. It must authenticate/authorise the user if appropriate [1].
5. It must check stock, limits or other business rules [1].
6. It must store the order securely/consistently, for example as a transaction [1].

Q4 / 5 marks

1. Crawling finds and retrieves pages/links [1].
2. Indexing extracts/organises content for later search [1].
3. There are three outgoing links in the simplified model [1].
4. Divide 0.60 by 3 to calculate an equal contribution [1].
5. Each target receives 0.20 of rank from A, ignoring damping/other inputs [1].

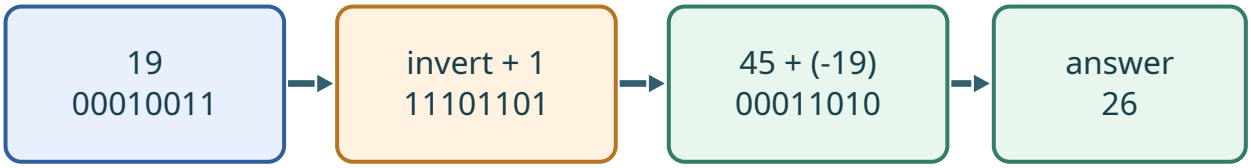
Exam tip: The page-rank computation is a simplified undamped example, not the complete PageRank algorithm.

Data types and eight-bit arithmetic

Theory / Worked Examples

Representations describe both the type of a value and the bit patterns used to store it. Binary subtraction is most reliable when the working shows every stage.

Subtract by adding the opposite



8-bit arithmetic; discard the final carry when representing the 8-bit result.

REMEMBER THIS

CRIBS, then invert-and-add. CRIBS: Character, Real, Integer, Boolean, String. To subtract using two's complement, invert the second binary number and add one.

Now be precise

• **Integer:** exact whole number; **real:** an approximation may be necessary in binary floating point. **Boolean:** true or false; **character:** a single code point/unit; **string:** a sequence of characters.

• **Eight-bit subtraction:** 45 − 19. 19=00010011; invert=11101100; add one=11101101; 45=00101101; add → 1 00011010; discard carry → 00011010 = 26.

• **Overflow:** with signed eight-bit two's complement the representable range is −128 to +127. 124 + 10 = 134 is out of range; the retained bit pattern 10000110 is interpreted as −122.

WORKED EXAMPLE 1

Calculate 45 − 19 in eight-bit two's complement.

Method: 00101101 + 11101101 = 1 00011010; discard the ninth carry. Answer 00011010 (26). This is a subtraction, not a bitwise NOT alone.

WORKED EXAMPLE 2

Calculate 31 − 12 in eight-bit two's complement.

Method: 31 = 00011111; 12 = 00001100; invert 11110011, add 1 = 11110100; add = 1 00010011; answer 00010011 (19).

COMMON MISTAKE

A carry out is not the same as signed overflow; compare operand/result signs or representable range.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

Bit	One binary digit: zero or one.
Two's complement	A fixed-width representation of signed integers.
Overflow	A result that falls outside the range of the chosen representation.
Mantissa	Significand bits carrying the meaningful digits of a floating-point number.
Exponent	Bits encoding the power-of-two scale for a floating-point number.
Normalisation	Adjusting significand and exponent to standard form without changing value.
Character encoding	Rules for representing character information using binary data.
Bit mask	Pattern used with bitwise operators to select, set, clear or toggle bits.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 3 marks. Give a suitable primitive type for an exact pupil age, a decimal temperature and a true/false flag.

Q2 · 5 marks. Calculate 27 − 9 using eight-bit two’s complement. Show inversion, +1 and final binary answer.

Q3 · 4 marks. Explain signed overflow when 124 and 10 are added in eight-bit two’s complement.

Mark scheme

Compare only after completing the questions.

Q1 · 3 marks

Integer for age [1]; real/float for measured decimal temperature [1]; Boolean for flag [1].

Q2 · 5 marks

27=00011011 [1]; 9=00001001, invert=11110110 [1]; +1=11110111 [1]; sum=1 00010010 [1]; retain 00010010 = 18 [1].

Q3 · 4 marks

Signed range −128..127 [1]; true sum 134 exceeds max [1]; resulting 10000110 [1]; interpreted as −122 due wrap/overflow [1].

RECALL

CRIBS, then invert-and-add: CRIBS: Character, Real, Integer, Boolean, String. To subtract using two’s complement, invert the second binary number and add one.

1.4.1 / Binary, hexadecimal and signed values

Start with positional values; build towards signed arithmetic and bitwise masks.

OCR 1.4.1 • SPECIFICATION CHECK

- Convert unsigned binary and hexadecimal; select suitable primitive types.
- Represent negative integers in sign-and-magnitude and two’s complement.
- Perform fixed-width arithmetic and identify overflow; apply AND, OR, XOR and shifts.

Every 1-bit contributes the value of its position.

WORKED EXAMPLE 1 / $00101101_2 = 45$



1-bits: 32 + 8 + 4 + 1 = 45

Representation	Method and warning
Unsigned 8-bit	0–255: every column is a positive power of two.
8-bit sign magnitude	First bit is sign; remaining seven bits hold magnitude; two zeros exist.
8-bit two’s complement	For a negative value, invert the positive binary pattern and add 1. Range −128 to 127.
Hexadecimal	Group bits in nibbles; $1010\ 1101_2 = AD_{16}$.
Bitwise AND	1 only if both bits are 1; useful to isolate selected positions.

Common misconception: left and right shifts are not always equivalent to multiply/divide when overflow, fixed width, sign extension or discarded bits matter.

WORKED EXAMPLE 1 → WORKED EXAMPLE 2 / signed representation

WORKED EXAMPLE 1 / WRITE -13 AS 8-BIT TWO'S COMPLEMENT

Step	Working
Positive 13	00001101
Invert all bits	11110010
Add 1	11110011
Check	Interpret as -128 + 64 + 32 + 16 + 2 + 1 = -13.

WORKED EXAMPLE 2 / a different value

Represent −19 as 8-bit two's complement. Show positive 19, inversion and +1.

Guided check: +19 is 00010011; invert → 11101100; add 1 → 11101101.

Worked bitwise mask

Given value 11010110 and mask 00001111, AND gives 00000110. The high four bits are cleared while the low four remain. XOR with 00000001 would toggle the least significant bit.

Signs and arithmetic

Two's-complement signed overflow occurs when adding two operands of the same sign gives a result with the opposite sign in the fixed-width format.

Binary, signed numbers and data representation

Q1 [3 marks] Convert 10110110 in binary to hexadecimal and denary (unsigned).

WORKING / METHOD

Q2 [4 marks] Express −27 as an 8-bit two’s-complement integer; show your method.

WORKING / METHOD

Q3 [4 marks] Add 01011111 and 00100010 as signed 8-bit values. Give the 8-bit result and say whether overflow occurs.

WORKING / METHOD

Binary, signed numbers and data representation

Q4 [3 marks] Evaluate 11001100 XOR 00001111 and describe one use of XOR.

WORKING / METHOD

Q5 [3 marks] Challenge: Which ASCII/Unicode distinction is most relevant when storing text in multiple languages?

Model answers and marking points

Binary conversion, two’s complement and encodings

Compare with your own response. Credit equivalent technically accurate statements.

Q1 / 3 marks

1. Split 10110110 into 1011 0110 and convert to hexadecimal B6 [1].
2. Use the correct place values 128+32+16+4+2 [1].
3. Give the unsigned denary value 182 [1].

Q2 / 4 marks

1. Represent positive 27 as 00011011 [1].
2. Invert all eight bits to 11100100 [1].
3. Add one to give 11100101 [1].
4. Identify this as the 8-bit two’s-complement representation of −27 [1].

Q3 / 4 marks

1. Convert or interpret the two operands as 95 and 34 [1].
2. Add to obtain the low eight bits 10000001 [1].
3. Recognise 10000001 as −127 when interpreted as signed 8-bit two’s complement [1].
4. State that signed overflow occurs because two positives produce a negative sign bit / true sum 129 is above 127 [1].

Q4 / 3 marks

1. The XOR result is 11000011 [1].
2. A bit becomes 1 when corresponding inputs differ (0 otherwise) [1].
3. Give a valid use, such as toggling selected flag bits using a mask [1].

Q5 / 3 marks

1. ASCII has a relatively limited character repertoire [1].
2. Unicode supports a much wider set of characters across languages and scripts [1].
3. A code point is an abstract character identifier, while a scheme such as UTF-8 encodes it into bytes [1].

Exam tip: Check signed and unsigned interpretations separately; an eight-bit pattern can mean different denary values.

Shifts and bitwise masks

Use a fixed width: all examples below retain exactly 8 bits.
Bit 0 is the rightmost bit; bit 7 is the leftmost. A mask selects positions.

WORKED EXAMPLE 1 / Move the bits, then interpret the result

```
Original:       10100101 = 165 (unsigned)

Logical left 1: 01001010 =  74   [leftmost 1 lost; 0 enters right]

Logical right 1:01010010 =  82   [rightmost 1 lost; 0 enters left]
```

A logical shift fills vacated positions with zeros; bits leaving the width are lost.
Left by k multiplies an unsigned value by 2^k only if significant bits are not lost.
Here 165 x 2 = 330 does not fit in 8 bits, so the retained result is 74.
Right by k gives the unsigned integer quotient by 2^k: 165 divided by 2 gives 82.
Arithmetic right shift repeats the sign bit instead; do not confuse the two.

MASKS / Apply the operation separately to each bit

Operation	Mask bit 1	Mask bit 0	Use
AND	preserves data	clears data	isolate / clear
OR	sets data to 1	preserves data	set flags
XOR	toggles data	preserves data	flip flags

Same data, same mask, three different results

```
Data: 10100101     Mask: 00001111

AND:  00000101     keep only the low four data bits

OR:   10101111     set the low four bits to 1

XOR:  10101010     flip the low four data bits
```

SELECT A SINGLE BIT / Change only the intended flag

Set bit 3: 10100101 OR 00001000 = 10101101.
Clear bit 5: 10100101 AND 11011111 = 10000101.
Toggle bit 0: 10100101 XOR 00000001 = 10100100.

WORKED EXAMPLE 2 / Use a different value before checking

Start from 11001010. Apply AND 00001111. Which bits have been preserved?
Then, starting again from 11001010, perform a logical right shift by 2.
Check: AND gives 00001010; the low four bits are preserved.
Right shift gives 00110010 (50); two rightmost bits are discarded.

1.4.1 / Floating-point numbers

Representation, normalisation and signed arithmetic

REMEMBER THIS Scientific notation with a sliding binary point: mantissa = significant digits; exponent = scale.

ALIGN EXPONENTS BEFORE ADDING

A: 1.0
01000000 0001

B: 0.5
01000000 0000

→

B aligned
00100000 0001

01000000 + 00100000 = 01100000
Result: 01100000 0001 = 1.5

NOW BE PRECISE OCR examples here use a two's-complement fractional mantissa and exponent; do not confuse this with IEEE 754.

KNOW IT PRECISELY

Mantissa. 8-bit signed fraction: binary point after the sign bit.

Exponent. 4-bit signed integer: multiply the mantissa by 2 raised to this power.

Normalised. Leading mantissa bits differ: 01 for positive, 10 for negative.

Alignment. Before addition or subtraction, use the same exponent.

SIGNED FRACTION: BINARY POINT AFTER SIGN BIT

-1	1/2	1/4	1/8	1/16	1/32	1/64	1/128
0	1	1	0	0	0	0	0

Example 01100000: 1/2 + 1/4 = 0.75

Floating-point numbers / Worked method

Worked examples and a different guided problem.

UNDERSTAND IT

- Decode 01100000 0011: $0.1100000_2 = 0.75$; exponent 0011 = +3; value = 0.75 × 8 = 6.
- Normalise 00011000 0100: shift mantissa left twice (00011000 → 00110000 → 01100000); reduce exponent from 4 to 2. Value remains 3.
- For negative numbers, the two's-complement fractional mantissa 10100000 equals −0.75. With exponent 0010, the stored value is −3.

WORKED EXAMPLE 1 / SUBTRACT 6 − 2

6 = 01100000 0011; 2 = 01000000 0010. Align the smaller exponent: 01000000 → 00100000 at exponent 0011. Subtract: 01100000 − 00100000 = 01000000. Answer: 01000000 0011 = 4.

PRECISION CHECK

If an arithmetic right shift discards non-zero low-order bits, the result can lose precision. The discarded bits are not restored by normalisation.

WORKED EXAMPLE 2 / ADD 1 + 0.5

1 = 01000000 0001 and 0.5 = 01000000 0000. Align 0.5 to exponent 1, obtaining 00100000 0001. Add mantissas to obtain 01100000 0001 = 1.5.

Signed floating-point arithmetic

Trace the mechanism carefully; these examples are not questions to memorise.

NEGATIVE MANTISSA + POSITIVE MANTISSA

Represent −3 as 10100000 0010 (−0.75 × 4), and +1 as 01000000 0001 (0.5 × 2). Both use signed two's-complement mantissas.

C# / OCR-style code

```
−3:       10100000 0010
+1:       01000000 0001
ALIGN +1:00100000 0010
ADD:      11000000 0010
NORMAL:   10000000 0001
```

The aligned mantissas add to 11000000, which starts 11 and is not normalised. Left-shift to 10000000 while reducing the exponent to 0001. This represents −1 × 2 = −2.

EXAM CHECK A negative normalised mantissa begins 10, not 11. A lost low-order bit during alignment cannot be recovered.

Floating-point numbers / Exam practice

Complete the questions independently. Use the space provided for your working.

Q1 4 MARKS

Decode 01010000 0011, stating the signed fractional mantissa, exponent and denary value.

Q2 5 MARKS

Normalise 00011000 0100. Show each mantissa shift, the updated exponent and why the value is unchanged.

Floating-point numbers / Exam practice

Complete the questions independently. Use the space provided for your working.

Q3 6 MARKS

Add 01000000 0001 and 01000000 0000. Show exponent alignment, mantissa addition and the normalised result.

Floating-point numbers / Check your answers

Use these mark points after completing the independent questions.

Q1 / 4 MARKS

[1] Mantissa $0.1010000_2 = 0.625$.

[2] Exponent 0011 = +3.

[3] Calculate 0.625×2^3.

[4] Answer 5.

Q2 / 5 MARKS

[1] Identify that the mantissa begins 00 and is not normalised.

[2] First left shift gives 00110000; exponent becomes 0011.

[3] Second left shift gives 01100000; exponent becomes 0010.

[4] First two bits are now 01.

[5] Verify $0.75 \times 2^2 = 3$, same original value.

Q3 / 6 MARKS

[1] First number is 1 and second is 0.5.

[2] Raise smaller exponent 0000 to 0001.

[3] Arithmetic-right-shift its mantissa to 00100000.

[4] Add 01000000 + 00100000 = 01100000.

[5] Keep aligned exponent 0001.

[6] Result 01100000 0001 is normalised and represents 1.5.

Character sets: ASCII and Unicode

A character has a code; an encoding decides how that code becomes bytes.

KNOW IT PRECISELY

Character set: a repertoire of characters with assigned numeric codes.
Code point: a numeric identifier for a character, such as U+0041 for A.
Encoding: rules for representing code points as bytes, for example UTF-8.
The receiver needs the correct encoding to interpret the stored bytes.

ASCII: 7 bits, 128 possible codes

Standard ASCII has codes 0-127, including letters, digits and control characters.

It covers a limited repertoire; it cannot represent all writing systems.

A 7-bit ASCII code can be stored in an 8-bit byte with a leading zero.

That extra zero does not increase the standard ASCII repertoire.

WORKED EXAMPLE 1 / Encode "AB" - the same method as Lesson 008

1. Find codes: A = 65 and B = 66.
2. Convert to 7-bit binary: A = 1000001; B = 1000010.
3. Store in 8-bit bytes: 01000001 01000010.
4. Storage: 2 bytes = 16 bits (ignore metadata).

UNICODE / Separate the repertoire from its encoding

Unicode supports many writing systems and symbols. Its first 128 code points match ASCII, but Unicode text does not have one universal byte width.
UTF-8 uses 1-4 bytes per code point; ASCII code points use one byte.
UTF-16 uses one or two 16-bit code units per code point.
A displayed character can consist of more than one code point.

WORKED EXAMPLE 2 / Use the stated byte widths

A UTF-8 message contains 8 ASCII code points and 2 euro-sign code points.
Use 1 byte per ASCII code point and 3 per euro sign. Ignore metadata.
Calculate its storage before checking: _____
Check: (8 x 1) + (2 x 3) = 14 bytes; 14 x 8 = 112 bits.

EXAM PRECISION

Both ASCII and Unicode assign numeric codes to characters. Unicode has a much larger repertoire. Do not claim that Unicode is always 16 bits per character.
Capacity: n bits give 2^n codes. For 300 codes, 8 bits give only 256; 9 bits give 512, so a fixed-width code needs at least 9 bits.

Revisit Lesson 008: New Learning and WORKED EXAMPLE 1.

1.4.2 / Arrays, records, stacks and trees

Choose the structure from the operations the problem actually needs.

OCR 1.4.2 • SPECIFICATION CHECK

- Read 1D–3D arrays, records, lists and tuples.
- Trace stack, queue, linked list, BST and graph operations.
- Explain which structures support rapid search, insertion, deletion or ordered access.

Use the same operations to trace arrays, records and linked structures.

Visualise the order of removal



Stack (LIFO) Queue (FIFO)

Structure	Useful invariant / operation
Stack	LIFO: push and pop operate on one end (top).
Queue	FIFO: enqueue at rear, dequeue at front.
Linked list	Nodes store values and links; deletion changes references, not an array shift.
Binary search tree	Left subtree values smaller, right subtree values greater (with a stated duplicate policy).
Graph	Vertices and edges; directed edges have an orientation.
Hash table	Hash function gives a slot; collisions need a resolution strategy.

C# arrays: `int[,,] x = new int[2,3,4];` holds 24 integer elements, with length 4 on dimension 2.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

Array	Indexed elements normally sharing the same type.
Record	Composite value with named fields, possibly of different types.
Stack	Last In, First Out structure using push and pop.
Queue	First In, First Out structure using enqueue and dequeue.
Linked list	Nodes containing values and links defining list order.
Binary search tree	Node ordering with smaller values left and larger values right.
Graph	Vertices connected by edges that may be directed or weighted.
Hash table	Uses hash values for storage locations, with collision handling.

WORKED EXAMPLE 1 → WORKED EXAMPLE 2 / manipulate structures

WORKED EXAMPLE 1 / STACK STATE

An empty stack receives Push(4), Push(9), Push(17), Pop(). The last inserted value, 17, is returned by Pop. The stack now contains 4 (bottom) and 9 (top).

WORKED EXAMPLE 2 / different operations

An initially empty queue receives Enqueue(8), Enqueue(3), Enqueue(11), Dequeue(), Enqueue(6). Draw its contents from front to rear and state the dequeued value.

_____

_____

_____

Check: 8 is removed; queue from front to rear becomes 3, 11, 6.

Records and tuples

A record groups fields with names. A tuple groups a fixed set of values. Both can model a pupil with candidate number and score, but choose named fields when readable record structure matters.

Linked-list trace

HEAD → [4 | next] → [9 | next] → [17 | null]. To remove 9, make node 4 link directly to node 17. Deleting the link without reconnecting the list would lose access to the remaining node.

Data structures · operations and explanations

Q1 [3 marks] State the stack contents after Push(6), Push(2), Pop(), Push(8).

Q2 [3 marks] In a FIFO queue containing front → [5, 7, 12] ← rear, enqueue 4 then dequeue once. Give the resulting order.

Q3 [6 marks] Insert 14, 6, 18, 2, 9 into an initially empty binary search tree. Give the pre-order traversal (root, left, right) and post-order traversal (left, right, root).

DIAGRAM / EXPLANATION

Data structures · operations and explanations

Q4 [4 marks] A program must add and delete nodes frequently without shifting a contiguous array. Justify one structure and identify one drawback.

DIAGRAM / EXPLANATION

Model answers and marking points

Stacks, queues, BSTs and linked lists

Compare with your own response. Credit equivalent technically accurate statements.

Q1 / 3 marks

1. The final stack contains 6 at the bottom [1].
2. The final stack contains 8 at the top [1].
3. 2 is no longer present because it was removed by Pop [1]. Award all three for the unambiguous final state bottom-to-top 6,8.

Q2 / 3 marks

1. The new queue front is 7 [1].
2. The next element is 12 [1].
3. The new rear is 4 [1]. Award all three for the complete ordered state 7,12,4.

Q3 / 6 marks

1. Root is 14 [1].
2. 6 is the left child and 18 the right child of 14 [1].
3. 2 and 9 are the left and right children of 6 [1].
4. Pre-order: 14,6,2,9,18 [1].
5. Post-order correctly visits the left subtree as 2,9,6 [1].
6. Post-order completes with right subtree 18, then root 14 [1].

Q4 / 4 marks

1. A linked list is a suitable structure for frequent known-position insertions/deletions [1].
2. The necessary links can be changed without shifting all later elements of a contiguous array [1].
3. Unlike an array, indexing/search normally requires sequential traversal [1].
4. Links/references add memory overhead (or equivalent supported drawback) [1].

Exam tip: State stack and queue order clearly. For a BST, identify the traversal method before listing values.

Data structures: indexes and operations

Arrays (1D-3D), records, lists and tuples. Select the appropriate representation, then trace an operation.

REMEMBER / THE BLOCK OF FLATS

A **one-dimensional array** is one row of rooms. A **two-dimensional array** adds floors and rooms; a **three-dimensional array** adds blocks. The precise computer-science model is an array indexed along one, two or three axes, not physical flats.

ONE ARRAY CAN HAVE ONE, TWO OR THREE INDEXES

1D marks[2]

selects the third item

5	7	9	4

2D grid[row, col]

grid[1, 2] = 22

Two rows, three columns

10	11	12
20	21	22

3D cube[layer, row, col]

cube[1, 0, 2] = 8

First select layer 1, then row 0, column 2.

Highlighted cells show the exact indexed element.

Layer 0			Layer 1		
0	1	2	6	7	8
3	4	5	9	10	11

In C#, a multidimensional array such as **int[,,] cube = new int[2,2,3]** has two layers, two rows and three columns. Valid indexes for each dimension begin at zero. The diagram highlights **cube[1,0,2] = 8**; there are 12 elements in total.

WORKED EXAMPLE 1 / TRACE A DIFFERENT EXAMPLE

```
int[,] seats = { {4, 7, 9}, {2, 6, 8} };
seats[0,1] = seats[1,2] + 1;
// seats[0,1] becomes 9 (not 7).
```

Records, tuples, lists and arrays

Different structures are useful for different operations.

CHOOSING A STRUCTURE: WHAT CAN CHANGE?

ARRAY

Fixed dimensions

grid[1,2] = 22

LIST

Variable length

Add(9), Remove(5)

RECORD

Named fields

Student: Name, Mark

TUPLE

Ordered values

(Room, Period)

Arrays and lists contain sequences; records and tuples group fields.

A C# record is one language mechanism for named data.

NOW BE PRECISE

A fixed-size array supports indexed access. A List<T> can grow or shrink. A tuple groups ordered values; a record groups named attributes. The **C# record** construct is a language-specific way to model named data and is not identical to every OCR exam-reference-language record declaration.

WORKED EXAMPLE 2 / DIFFERENT QUESTION

```
var rooms = new List<int> { 5, 8, 3 };
rooms.Remove(8);
rooms.Add(6);
// What are rooms[1] and rooms.Count?
```

Check: The list becomes [5,3,6]; rooms[1] = 3 and Count = 3. Remove(8) deletes a matching value, not the element at index 8. Be careful with RemoveAt(1), which removes by position.

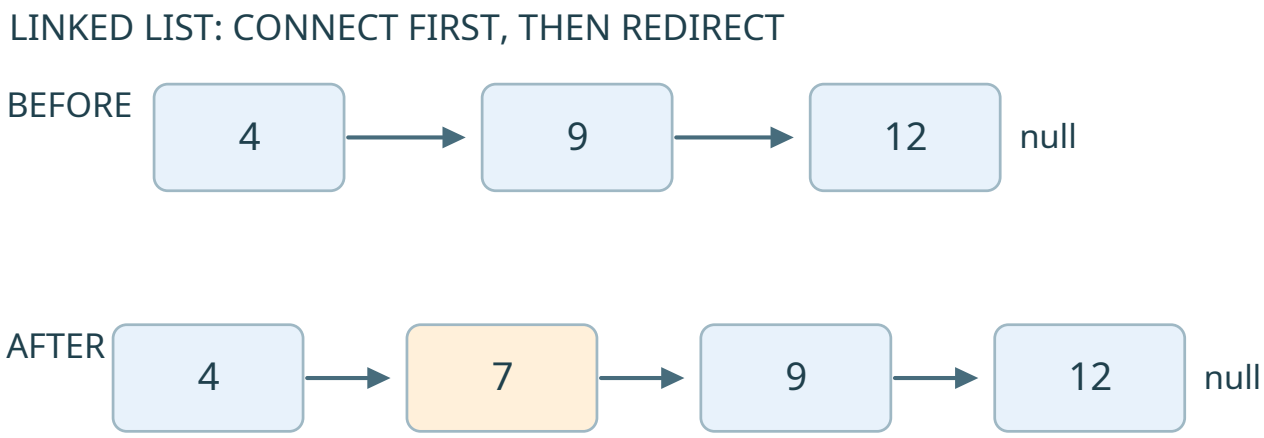
A named record for a pupil might have **Name** and **Mark**. A tuple **("A1", 5)** can represent a room and period. Choose based on whether named fields, indexing or variable length are needed.

Linked-list operations

The train-carriage analogy is useful only when you follow the references accurately.

REMEMBER / TRAIN COUPLINGS

Insert a carriage by connecting it to the remainder first, then redirect the previous carriage. To remove a carriage, change the previous link to bypass it. A singly linked list needs a head reference to find its first node.



1 newNode.Next = node4.Next

2 node4.Next = newNode

REMOVE 9:	node7.Next = node9.Next Result: 4 -> 7 -> 12

WORKED EXAMPLE 1 / CREATE AND REMOVE

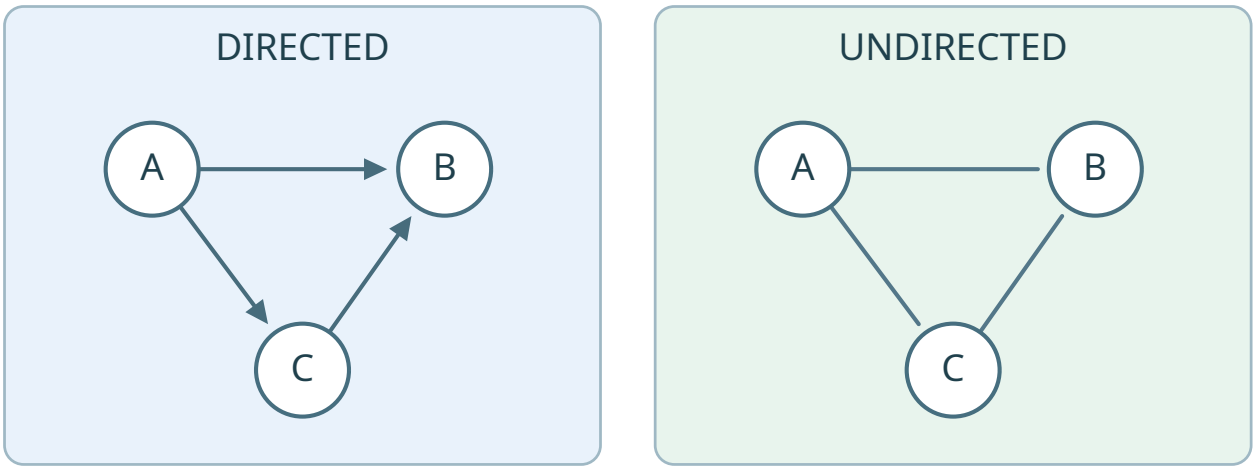
```
// Custom node with writable Next field:
Node inserted = new Node(7, node4.Next);
node4.Next = inserted;
// To remove 9 after node7:
node7.Next = node9.Next;
```

The final chain is **4 -> 7 -> 12**. Removing the head instead requires **head = head.Next**. This example uses a custom Node type; .NET LinkedListNode<T>.Next is not writable.

Graphs: directed versus undirected

Edges encode relationships; their direction determines which neighbours can be reached.

GRAPH EDGES: DIRECTION CHANGES THE MEANING



Directed: A -> B does NOT imply B -> A.

Undirected: A - B allows travel both ways.

Directed graph adjacency list: A: [B,C], B: [], C: [B]. An edge A -> C does not automatically provide C -> A. **Undirected graph:** each A-B link appears in the neighbour lists of both A and B.

WORKED EXAMPLE 2 / A DIFFERENT GRAPH

A road graph has directed edges **P -> Q**, **Q -> R** and **P -> R**. Remove **P -> R**. Which routes from P to R remain, and what changes in the adjacency list?

Check: P can still reach R via Q: P -> Q -> R. Its adjacency list now has P: [Q], Q: [R], R: []. If this represented an undirected connection, both matching neighbour entries would need updating.

COMMON MISTAKE

A traversal is not the same as creating or modifying a graph. Adding an edge changes connectivity; breadth-first traversal uses a queue to visit reachable vertices in layers. A cycle needs a visited set to avoid repeat processing.

Exam practice / data structures

Write answers independently. Use the marked questions before checking the worked answers.

Q1 [4 marks] An array cube has dimensions [2,2,3]. State its total number of elements, the valid indexes of its third dimension, the value of cube[1,0,2] in the diagram, and why cube[2,0,0] is invalid.

Q2 [6 marks] A program needs (i) a fixed grid of marks, (ii) a growing collection of pupil IDs, (iii) one pupil with Name and Mark fields. Choose structures and justify each choice with a suitable operation.

Exam practice / linked lists and graphs

Q3 [6 marks] A singly linked list is 3 -> 8 -> 12. Describe the two reference assignments needed to insert 10 after 8, then the change needed to remove 8 while preserving all other nodes.

Q4 [4 marks] Directed edges are A -> B, A -> C, C -> B. Give the adjacency list. Then delete C -> B and describe whether B can still be reached from A.

Exam practice / choosing operations

Q5 [6 marks] A system must store tasks in arrival order, undo the most recent action, and find a particular key in a binary search tree. Recommend a suitable structure for each and explain the relevant removal/search operation.

Q6 [4 marks] A FIFO queue contains [4,6] from front to rear. Enqueue 8, then Dequeue once. State the two queue states and explain how a stack would remove a different value.

Check your answers / data structures

Marking points credit equivalent correct terminology and operations.

Q1 [4] 2 x 2 x 3 = 12 [1]; third dimension indexes 0,1,2 [1]; cube[1,0,2] = 8 [1]; index 2 is out of range for first dimension of length 2 (only 0 and 1 valid) [1].

Q2 [6] 2D array for fixed grid [1] with row/column indexing [1]; List for changing collection length [1] with Add/Remove [1]; record or suitable named-field structure [1] because Name and Mark are grouped attributes [1].

Q3 [6] New node 10 points to previous successor 12 [2]; node 8 now points to 10 [1]; list after insertion 3,8,10,12 [1]; to delete 8, predecessor node 3 points directly to 10 [2].

Q4 [4] A:[B,C] [1]; B:[] [1]; C:[B] [1]; after deleting C->B, B remains reachable directly by A->B [1].

Q5 [6] Queue/FIFO for arriving tasks and Dequeue oldest [2]; stack/LIFO for undo and Pop newest [2]; BST search compares key and follows left/right child according to ordering until found or null [2].

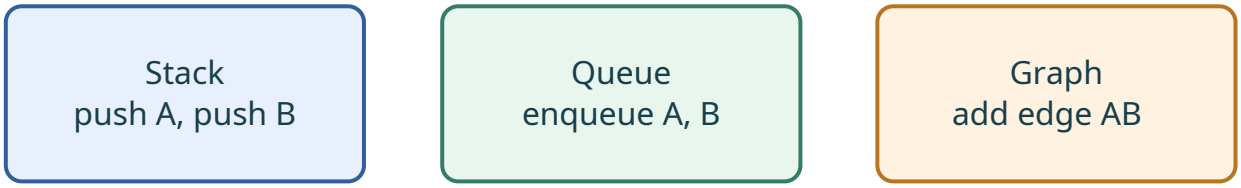
Q6 [4] After Enqueue 8: [4,6,8] [1]; Dequeue removes front 4 [1]; queue is [6,8] [1]; Pop on a stack of 4,6,8 would remove most recently pushed 8 [1].

Data structures: choose the correct operation

Theory / Worked Examples

OCR expects learners to create, traverse, add to and remove from the named structures—not just recognise their diagrams.

A structure changes state



LIFO: pop B · FIFO: dequeue A · Directed edge: AB BA

REMEMBER THIS

One building, different entrance rules. A stack behaves like plates (LIFO); a queue like a lunch line (FIFO); a graph is a set of vertices joined by directed or undirected edges.

Now be precise

• **Arrays/list/tuple:** arrays permit indexed traversal and replacement; dynamic lists allow insert/remove; tuples have fixed collections of elements in many languages, with mutation depending on the implementation.

• **Stack/queue:** push/pop for a stack; enqueue/dequeue for a queue. Both must handle empty structure before a removal.

• **Linked list/tree/BST/hash table/graph:** insertion and deletion must preserve links, ordering/lookup invariants or edges; BST two-child deletion typically replaces with predecessor/successor; deleting an open-address hash entry needs a tombstone or rehash so searches continue.

WORKED EXAMPLE 1

A stack receives 3, 8 and 5, then one pop. Give its content and output.

Method: Push 3, push 8, push 5 → top 5; pop returns 5; remaining bottom→top [3,8].

WORKED EXAMPLE 2

A queue receives 3, 8 and 5, then one dequeue.

Method: FIFO: dequeue returns 3; remaining front→rear [8,5]. Different from the stack despite identical inserts.

COMMON MISTAKE

The operations vary by implementation. Deleting a hash bucket by simply making it empty can break later searches.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 4 marks. Trace push(2), push(7), pop(), push(4), pop() on a stack. Give outputs and final stack.

Q2 · 3 marks. A directed graph has edges A→B and B→C. Remove B→C. Describe the resulting edge set and the reachability change.

Q3 · 6 marks. Explain one critical step when deleting (a) the head of a linked list, (b) a BST node with two children, and (c) a key from a linear-probed hash table.

Mark scheme

Compare only after completing the questions.

Q1 · 4 marks

First pop 7 [1]; second pop 4 [1]; final stack [2] [1]; operations use LIFO [1].

Q2 · 3 marks

Only A→B remains [1]; B no longer directly reaches C [1]; removing B→C does not imply removing A→B [1].

Q3 · 6 marks

Head: move head to next [2]; BST: replace with in-order successor/predecessor and restructure [2]; hash: tombstone/rehash to maintain probe chain [2].

RECALL

One building, different entrance rules: A stack behaves like plates (LIFO); a queue like a lunch line (FIFO); a graph is a set of vertices joined by directed or undirected edges.

Every structure, every operation

Create / Traverse / Add / Remove

The important examination skill is to show the new state of a structure after an operation, not simply to name the structure.

The operations checklist

Structure	Create / traverse	Add / remove
Array (1D-3D)	Declare dimensions; visit valid indexes.	Replace cell; fixed array size cannot shrink in place.
Record	Declare named fields; read fields.	Set a field; change schema or rebuild to remove a field.
Tuple	Group ordered values; access positions.	Create a new tuple if its components need changing.
List	Create sequence; iterate values.	Insert, append, remove value or index.
Linked list	Create linked nodes; follow Next.	Redirect links; update head if necessary.
Stack / queue	Initialise; inspect in permitted order.	Push/pop; enqueue/dequeue.
Tree / BST	Create root; visit nodes.	Attach/remove subtrees; preserve BST order.
Graph	Create vertices/edges; BFS/DFS.	Add/remove edges and vertices; update references.
Hash table	Allocate buckets; probe or iterate.	Insert with collisions; delete safely.

Remember this

Different structures have different rules for changing their contents. A stack is plates; a queue is a lunch line; a linked list is coupled train carriages; a BST is an ordered family tree.

Do not use a generic “delete” explanation. Say exactly which indexes, pointers, references, buckets or entries change.

Arrays and dynamic lists

Fixed dimensions versus a collection that can grow

WORKED EXAMPLE 1 / arrays: create, traverse, update

```
int[,,] cube = new int[2, 2, 3];
cube[1,0,2] = 8; // layer 1, row 0, column 2
for(int layer=0; layer<2; layer++)
  for(int row=0; row<2; row++)
    for(int col=0; col<3; col++)
      Console.Write(cube[layer,row,col]);
```

Two layers x two rows x three columns = 12 cells. All indexes begin at zero. Traversal visits every cell once; an assignment replaces the value at one valid location.

WORKED EXAMPLE 2 / remove from a fixed array

```
int[] a = {3, 8, 12, 15};
// Remove value at index 1 from the logical sequence.
int[] smaller = {3, 12, 15};
```

An array has fixed length. Create a shorter array and copy the retained elements, or keep capacity and track a smaller logical count. Do not claim that the original C# array itself shrinks.

Remember / flats versus a growing guest list

An array has numbered rooms; replacing a resident does not remove the room. A List<T> can grow or shrink as entries are added or removed.

```
var rooms = new List<int> {5, 8, 3};
rooms.Insert(1, 7);   // [5,7,8,3]
rooms.RemoveAt(2);   // [5,7,3]
foreach(int room in rooms) Console.Write(room);
```

C# RemoveAt removes by index; Remove(value) removes the first matching value. They are different operations.

Records and tuples

Changing a field is not the same as changing a structure

WORKED EXAMPLE 1 / record: named fields

```
record Student(string Name, int Mark);
Student pupil = new Student("Ada", 70);
Console.WriteLine(pupil.Name);
pupil = pupil with { Mark = 78 };
```

The C# record in this example has init-only positional properties. The with expression produces a record with the changed Mark; it does not remove the property from the type.

WORKED EXAMPLE 2 / tuple: ordered components

```
var booking = (Room:"B2", Period:3);
Console.WriteLine(booking.Room);
booking.Period = 4;
string roomOnly = booking.Room; // one value
```

The final line extracts Room as an ordinary string. To remove a tuple component, redesign/reconstruct the representation; the fixed-arity tuple does not shrink.

A precise comparison

Question	Record	Tuple
How do you create it?	Declare a type with named properties.	Group ordered values into fixed positions.
How do you traverse it?	Read its known named fields.	Read its components; not an indexed list.
How do you add/remove a field?	Change its type/schema or rebuild the record.	Rebuild a different type/shape.

Exam warning: a tuple is not a general-purpose growable list. Traversing means accessing its components; the technique depends on the language and tuple representation.

Stacks and queues

Same inputs, different removals

WORKED EXAMPLE 1 / stack: the plates

```
Stack<int> s = new();
s.Push(3); s.Push(8); s.Push(5);
int removed = s.Pop();       // 5
// Remaining bottom to top: 3, 8
```

Push and pop operate at the same end. LIFO means the most recently added element is removed first. In an abstract stack, traversal is not a fundamental operation; a language collection may offer non-destructive iteration, or values can be popped and restored.

WORKED EXAMPLE 2 / queue: lunch line

```
Queue<int> q = new();
q.Enqueue(3); q.Enqueue(8); q.Enqueue(5);
int served = q.Dequeue();    // 3
// Remaining front to rear: 8, 5
```

Enqueue adds at the rear; dequeue removes the front. FIFO means the oldest waiting item is served first. Do not confuse Peek (inspect) with Pop or Dequeue (remove).

Visual memory check

STACK: LIFO

Insert 3, 8, 5

Pop returns 5

QUEUE: FIFO

Insert 3, 8, 5

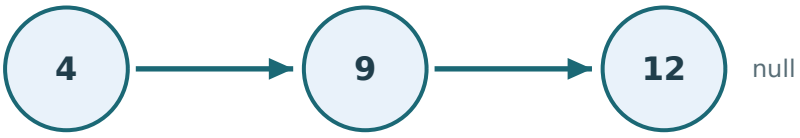
Dequeue returns 3

Before removing an element, check that the structure is not empty. Underflow (removing from empty) must be prevented or handled.

Linked lists: preserve the chain

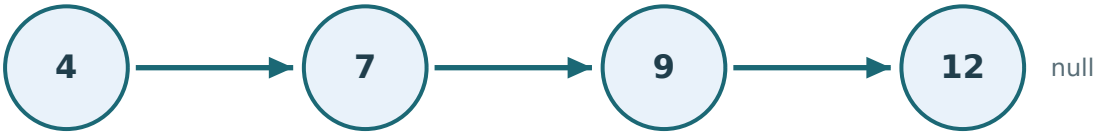
Insert and delete by redirecting references

WORKED EXAMPLE 1 / insert 7 between 4 and 9



Start: head -> 4 -> 9 -> 12 -> null.

```
Node inserted = new Node(7, node4.Next);
node4.Next = inserted;
```



Connect the new node to the existing successor first. Then redirect node4.Next. Otherwise, you could lose access to the remainder of the list.

WORKED EXAMPLE 2 / remove the middle node 9

```
node7.Next = node9.Next;
// New order: 4 -> 7 -> 12 -> null
```

To remove the head: head = head.Next. To remove a tail node, its predecessor.Next becomes null. Traversal starts at head and follows Next until null.

Code uses a custom Node class with writable Next, not .NET LinkedListNode<T>.Next, which is not directly assignable.

Trees and binary search trees

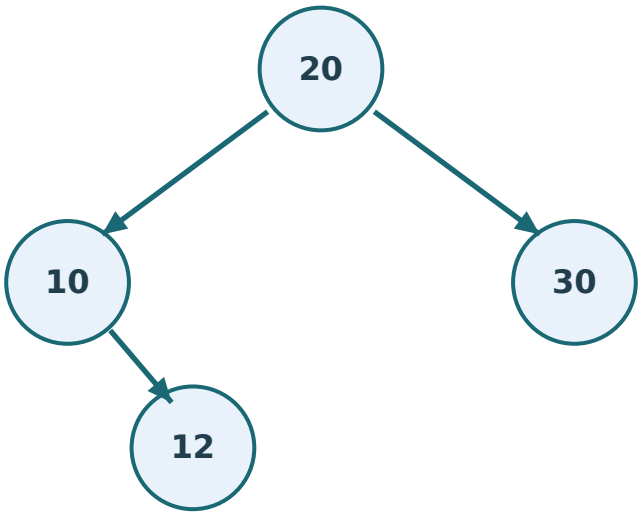
Keep parent-child links and ordering correct

A general tree

Create one root and attach child nodes. Traverse using a specified order (for example, depth-first or breadth-first). Deleting a general-tree node needs an explicit rule: remove its entire subtree or reattach its children.

WORKED EXAMPLE 1 / BST insert

Insert 12 into a BST with root 20, left child 10 and right child 30. Compare 12 < 20, then 12 > 10: make 12 the right child of 10.



WORKED EXAMPLE 2 / remove a BST node with two children

Suppose the root 20 also has a right subtree containing 25 and 30. Replace 20 with its inorder successor 25 (the smallest value in the right subtree), then delete that original successor node. Update its parent link.

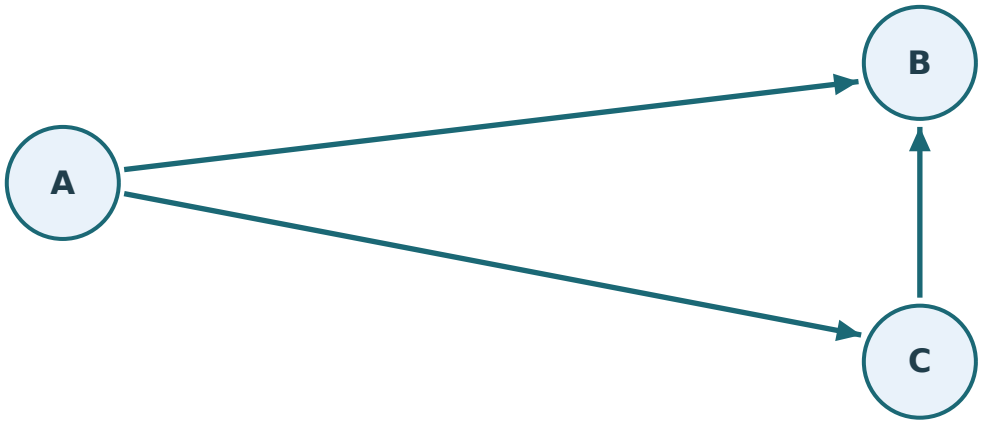
Other cases: a leaf can be detached; a node with one child can be replaced by its child. Search and inorder traversal depend on the BST invariant. State a duplicate-key rule if necessary.

Graphs: add, search, remove

Direction affects reachability

WORKED EXAMPLE 1 / directed graph

Create vertices A, B and C, then edges A->B, A->C and C->B. Store adjacency list A:[B,C], B:[], C:[B]. An arrow is one-way.



WORKED EXAMPLE 2 / remove C->B

Update C from [B] to []. A can still reach B directly. To delete vertex C entirely, remove the C entry AND remove every incoming reference to C from other neighbour lists.

Traverse without looping forever

```
BFS from A:
visited = {A}; queue = [A]
remove A -> visit B,C and enqueue them
remove B -> no new neighbours
remove C -> B already visited
```

For an undirected edge A-B, adjacency lists must have A:[B] and B:[A]. Removing the edge must remove both entries. DFS uses a stack/recursion; BFS uses a queue. Both need visited tracking in cyclic graphs.

Hash tables: collisions and deletion

A deleted slot must not break later searches

WORKED EXAMPLE 1 / create, insert and traverse

Create 5 buckets, indexed 0-4. Use h(k) = k MOD 5 and linear probing. Insert 6, 11, 16 in that order.

Bucket	0	1	2	3	4
Value	empty	6	11	16	empty

6 hashes to 1. So do 11 and 16, but occupied buckets make them probe to 2 and 3. A search for 16 follows 1 -> 2 -> 3.

WORKED EXAMPLE 2 / delete the middle key 11

Bucket	0	1	2	3	4
Value	empty	6	DELETED	16	empty

Mark bucket 2 with a tombstone (DELETED), not empty. The lookup for 16 must probe past bucket 2 to reach 3. Rehashing the affected cluster is another valid approach.

Remember / the postcode collision

A postcode points to the first bucket, but two people may have the same postcode. Linear probing checks nearby addresses. Removing a resident must not close the road before later residents.

Traversal means inspecting occupied buckets or records using the implementation’s rules. A hash table does not guarantee sorted order. Always specify the collision policy and capacity.

Exam practice: linear structures

Work independently; marking points are after all the questions.

Q1 / 6 marks

An array grid has 2 rows and 3 columns, and a growing list is [4,7,9]. State the total number of array cells; give a valid last-column index; show the list after Insert(1,5) and RemoveAt(2); explain how to remove an array element when its size is fixed.

Q2 / 5 marks

A stack and a queue both receive 2, 6 and 8 in that order. State the value removed first from each, the remaining content of each, and the operation responsible for the different results.

Exam practice: links and search trees

State exactly what changes. Draw references or explain them precisely.

Q3 / 6 marks

A singly linked list contains 4 -> 9 -> 12. Insert a new node 7 after 4, then remove node 9. Give the two insertion assignments, the removal assignment and the final list. Why is the update order important?

Q4 / 6 marks

A BST has root 20, left child 10 and right child 30; node 30 has left child 25. Describe how to remove root 20 while retaining valid BST ordering. Include the replacement value, links that change, and an inorder traversal afterwards.

Exam practice: graphs and hashing

Show the state before and after each change.

Q5 / 5 marks

A directed graph has vertices A, B, C and edges A->B, A->C and C->B. Give the original adjacency list, remove vertex C correctly, then state the remaining directed edge and the vertices reachable from A.

Q6 / 6 marks

Insert 6, 11 and 16 into five hash buckets using key MOD 5 and linear probing. Show the occupied bucket indexes, then remove 11. Explain why an empty slot would be unsafe and how the search for 16 must behave.

Exam practice: trees, records, tuples

Be precise about what a type allows you to change.

Q7 / 4 marks

A general tree has root R with children A and B. Node A has child C. Give a valid breadth-first traversal. If deleting A removes its complete subtree, state the remaining nodes and name the node relationships that disappear.

Q8 / 5 marks

A C# record Student(Name, Mark) and tuple (Room, Period) hold different kinds of grouped data. Explain how to create and access a value in each, how to change Mark or Period in the given examples, and why neither structure supports deleting one component as if it were a dynamic list.

Check your answers: Q1–Q3

Credit equivalent precise explanations; do not award the same marking point twice.

Q1 / 6 marks

- Array size 2 x 3 = 6 [1]; final column index is 2 [1].
- Insert(1,5): [4,5,7,9] [1]; RemoveAt(2): [4,5,9] [1].
- An array has fixed size [1]; construct a new shorter array or shift within a separate logical length [1].

Q2 / 5 marks

- Stack pop returns 8 [1]; remaining stack is [2,6] [1].
- Queue dequeue returns 2 [1]; remaining queue is [6,8] [1].
- The stack follows LIFO (pop newest), whereas the queue follows FIFO (dequeue oldest) [1].

Q3 / 6 marks

- Create new node 7 with Next = old node4.Next (node 9) [1].
- Assign node4.Next = newNode7 [1].
- Assign node7.Next = node9.Next to bypass 9 [1].
- Final list: 4 -> 7 -> 12 [1].
- Updating the new node link first protects access to the remainder [2].

Check your answers: Q4–Q6

Detailed marking points for the tree, graph and collision questions.

Q4 / 6 marks

- Inorder successor of 20 is 25 [1].
- Replace root 20 by 25 (or move the successor node to the root) [1].
- Remove 25 from its previous position as the left child of 30 [1].
- Root left child 10 remains [1], right child 30 remains [1].
- Inorder traversal after deletion: 10,25,30 [1].

Q5 / 5 marks

- Initial adjacency list A:[B,C], B:[], C:[B] [2].
- Remove C and its adjacency-list entry [1].
- Remove incoming reference to C from A, leaving A:[B] [1].
- A still reaches A and B (B by A->B); remaining edge is A->B [1].

Q6 / 6 marks

- 6 at bucket 1 [1]; 11 at bucket 2 [1]; 16 at bucket 3 [1].
- On deleting 11, mark bucket 2 deleted/tombstone, or rehash cluster [1].
- Making 2 empty could terminate a search too early [1].
- Lookup for 16 must probe through bucket 2 to bucket 3 [1].

Check your answers: Q7–Q8

Apply the structure’s actual rules rather than a memorised generic deletion.

Q7 / 4 marks

• Breadth-first order: R,A,B,C [1].

• Delete A and its child C when the subtree-removal convention is used [1].

• Remaining nodes R and B [1].

• Edges R-A and A-C are removed; R-B stays [1].

Q8 / 5 marks

• Record can be constructed with Student("Ada",70) and Name/Mark accessed by name [1].

• C# record with init-only properties can be updated via with { Mark = 78 } [1].

• Tuple created (Room:"B2", Period:3) and fields accessed by name [1].

• Mutable value tuple Period can be reassigned to 4 [1].

• Removing a field/component changes the type/shape, so rebuild rather than using a list-style remove [1].

Remember this

An operation must respect the structure’s invariant: fixed dimensions, field shape, LIFO/FIFO order, links, BST comparisons, graph direction or collision-probe continuity.

1.4.2 / HASH TABLES AND COLLISIONS

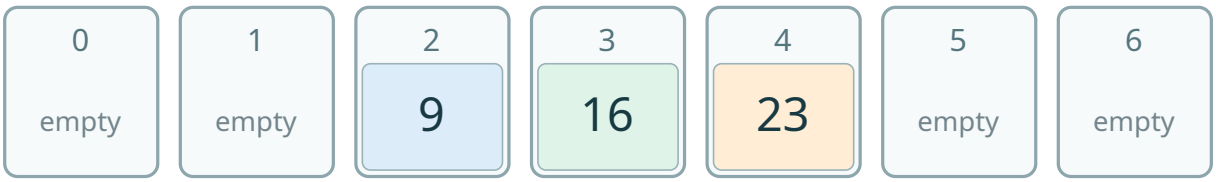
A calculated index gets you close; collision handling tells you where the value actually lives.

REMEMBER: THE POSTCODES CLASH

A hash function acts like a postcode that sends a key to a bucket. Two keys may calculate the same bucket: this is a collision. Linear probing looks for the next available bucket and wraps around at the end.

Hash table: collision then linear probing

h(key) = key MOD 7 Insert 9, 16, 23



9 -> 2 16 -> 2 occupied -> 3 23 -> 2,3 full -> 4

WORKED EXAMPLE 1: trace the school-lesson example

Seven buckets numbered 0–6; h(k) = k MOD 7. Insert 9, 16 and 23. All three calculate index 2. Insert 9 at 2; 16 probes to 3; 23 probes past both to 4. A lookup must follow the same probing sequence.

INDEX	0	1	2	3	4	5	6
VALUE	—	—	9	16	23	—	—

WORKED EXAMPLE 2: a wraparound collision

Five buckets 0–4, h(k) = k MOD 5. Insert 8, 13 and 18. They all hash to bucket 3. The values occupy buckets 3, 4 and 0 respectively. The last probe wraps around.

Deletion and a common mistake

With open addressing, simply making a deleted occupied bucket look completely empty can break later lookups. A tombstone records a deleted position so the search can continue along a valid probe chain. An alternative is separate chaining, storing multiple values in a bucket collection.

NOW BE PRECISE

A good hash function spreads keys; it does not guarantee unique indices. Expected O(1) access is possible with controlled load factor, but a long collision chain can degrade access towards O(n). Hashing does not itself sort keys.

Independent practice

Q1 [5 marks]

A hash table has seven buckets numbered 0–6 and h(k) = k MOD 7. Insert keys 10, 17, 24 using linear probing. State each calculated bucket and the final occupied buckets.

Q2 [5 marks]

Five buckets use h(k) = k MOD 5. Insert 4, 9, 14 using linear probing. Show the final table and explain the wraparound.

Q3 [6 marks]

Explain why a collision occurs, compare linear probing with separate chaining, and explain why a tombstone can be necessary when deleting from an open-addressed table.

Check and improve: hash tables marking points

Q1 [5 marks] All keys hash to bucket 3 [2]; 10 placed at 3, 17 probes to 4, 24 probes to 5 [3].

Q2 [5 marks] Each key hashes to bucket 4 [1]; 4 placed at 4 [1], 9 wraps to bucket 0 [1], 14 probes to bucket 1 [1]; wraparound follows bucket 4 with bucket 0 [1].

Q3 [6 marks] Collision: two distinct keys map to the same index [1]. Linear probing checks following slots [1] and can cluster [1]. Chaining stores several entries associated with one bucket [1]. A tombstone marks a deleted entry [1] without ending the search for keys inserted farther along the probe chain [1].

Logic gates: recognise the rule

Inputs and outputs use 1 = true / on and 0 = false / off.
Read from inputs on the left to output Q on the right.

AND



Both inputs must be 1.

`Q = A AND B`

OR



At least one input must be 1.

`Q = A OR B`

NOT



Reverse one input.

`Q = NOT A`

XOR



Exactly one input must be 1.

`Q = A XOR B`

TRUTH TABLES / List every input combination

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

A	NOT A
0	1
1	0

Two inputs give $2^2 = 4$ rows; one input gives $2^1 = 2$ rows.
OR includes the case 1 OR 1 = 1. XOR gives 1 XOR 1 = 0.
NOT has one input; the small output circle means inversion.
Symbol shorthand: AND = ∧, OR = ∨, NOT = ¬, XOR = ⊻.

Revisit Lesson 026: New Learning and WORKED EXAMPLE 1.

From a problem to a truth table

WORKED EXAMPLE 1 / Secure door access - Lesson 026 model

A door unlocks when a valid card OR a correct PIN is used, but only when the building is open and lockdown is not active.
Name inputs: A = valid card; B = correct PIN; C = open; D = lockdown.
Name output: Q = door unlocks.

BUILD THE EXPRESSION IN CHUNKS

1. Alternatives: A OR B. Both may be true, so use OR rather than XOR.
2. Required condition: AND C. Exception: AND (NOT D).
3. Combine: Q = (A OR B) AND C AND (NOT D).

Gate plan: OR(A, B), then AND with C; invert D; final AND gives Q.

CHECK ALL 16 ROWS / Use intermediate columns

A	B	C	D	A OR B	NOT D	Q
0	0	0	0	0	1	0
0	0	0	1	0	0	0
0	0	1	0	0	1	0
0	0	1	1	0	0	0
0	1	0	0	1	1	0
0	1	0	1	1	0	0
0	1	1	0	1	1	1
0	1	1	1	1	0	0
1	0	0	0	1	1	0
1	0	0	1	1	0	0
1	0	1	0	1	1	1
1	0	1	1	1	0	0
1	1	0	0	1	1	0
1	1	0	1	1	0	0
1	1	1	0	1	1	1
1	1	1	1	1	0	0

Example row 0110: A OR B = 1, C = 1, NOT D = 1, so Q = 1.
Example row 1111: lockdown is active, so NOT D = 0 and Q = 0.

WORKED EXAMPLE 2 / A different three-input problem

A warning W is on when temperature is high (T) OR smoke is detected (S), provided maintenance mode (M) is not active. Write W and test T=0, S=1, M=0.
Check after trying: W = (T OR S) AND (NOT M); the stated input gives W = 1.
A complete table would need $2^3 = 8$ rows. Write T OR S and NOT M first.

Revisit Lesson 026: New Learning and WORKED EXAMPLE 1.

01 / Retrieve and prepare

Before today: Boolean NOT, AND, OR; truth tables; De Morgan’s laws. Try these without notes; check in your class materials.

Retrieval question	Your short answer
1. Evaluate 1 AND 0.	_____
2. Evaluate NOT 0.	_____
3. Expand NOT(A AND B) using De Morgan.	_____
4. Challenge: Is A OR (A AND B) equivalent to A? Why?	_____

Key vocabulary

Keyword	Exam-accurate meaning
Adjacent	Cells that differ in one input only; opposite edges also touch.
Gray code	An arrangement where neighbouring binary codes differ by one bit: 00, 01, 11, 10.
Group	A rectangle containing 1, 2, 4, 8, ... cells of value 1.
Literal	One variable or its complement (for example A or ¬A).
Overlapping groups	A 1 may be used in more than one group if this simplifies the expression.
Product term	An AND expression formed from the unchanged inputs within one group.

Exam reminder • In a Karnaugh map, diagonal squares are not adjacent. When the value of an input changes across a group, that input disappears from the simplified term.

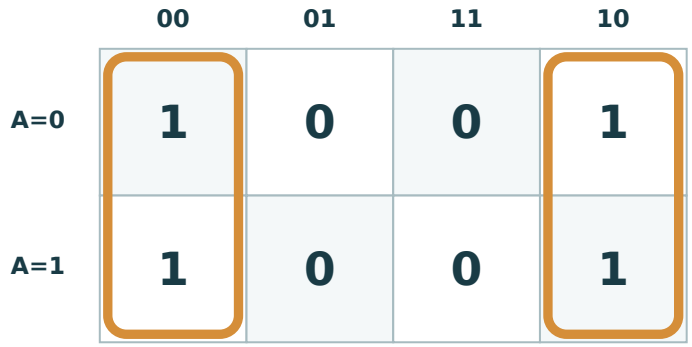
02 / Understand it

Start with a truth-table output pattern. Place its 1s on a Karnaugh map in Gray-code order, cover them using the largest useful rectangular groups, then translate each group into a term.

FIVE STEPS / CIRCLE → COMPARE → REMOVE → WRITE → DRAW

Example A / edge wraparound

Gray-code column order; row and column headings specify the inputs.



The outside columns 00 and 10 touch around the edge of the map.

Read the group: A changes between the rows. B changes between 00 and 10. C remains 0. The resulting expression is Q = ¬C.

Why the group has four cells

Opposite edges count as adjacent because only one input changes between 00 and 10. This is why the largest valid group crosses the edges rather than becoming two separate pairs.

Do	Avoid
Group sizes 1, 2, 4 or 8 (for three variables).	Groups of three cells; diagonal groupings.
Use overlapping groups if useful.	Insisting every 1 belongs to exactly one group.
Keep variables with a constant value.	Including a variable that changes from 0 to 1.

Quick check • If C = 0 makes Q = 1 whatever A and B are, a single NOT gate on C gives the same result.

03 / WORKED EXAMPLE 1 - fully worked

From RHSC Lesson 029: two groups produce two AND terms and one final OR.

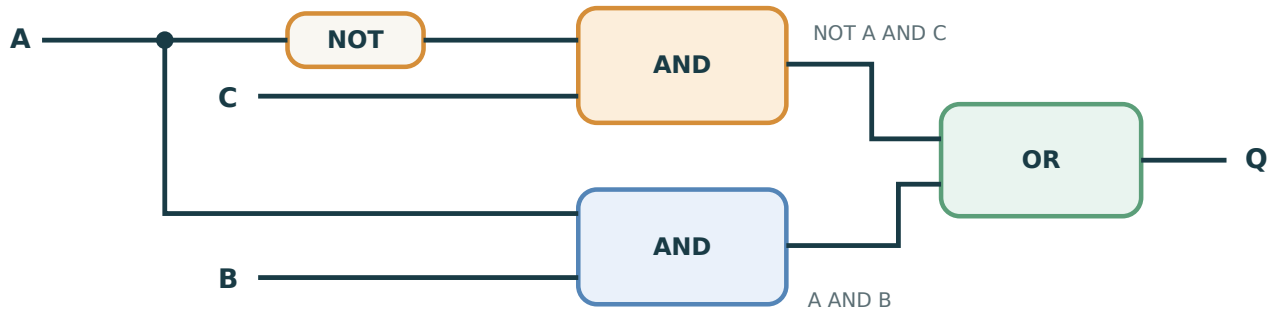
Worked K-map / orange top pair and blue bottom pair

Gray-code column order; row and column headings specify the inputs.

	00	01	11	10
A=0	0	1	1	0
A=1	0	0	1	1

Step	What you write and why
1. Orange	A = 0 and C = 1 in both squares; B changes. ¬A ∧ C.
2. Blue	A = 1 and B = 1 in both squares; C changes. A ∧ B.
3. Join	Either group produces Q = 1, so OR the terms: Q = (¬A ∧ C) ∨ (A ∧ B).
4. Check	Test one input for each group and a 0-cell, e.g. A=0,B=0,C=1 gives 1; A=1,B=0,C=0 gives 0.

Single-source inputs: A branches to NOT and directly to the lower AND



Tip: R combines different groups; AND connects the unchanged literals within a group. Label each wire before drawing the final circuit.

04 / WORKED EXAMPLE 2 - a different map

Work through each prompt. Reveal the checking point only after you have written your answer.

Guided task / one pair

Gray-code column order; row and column headings specify the inputs.

	00	01	11	10
A=0	0	0	0	0
A=1	0	1	1	0

Your step	Answer or working
1. Circle the largest group.	_____
2. State which two variables stay at 1.	_____
3. State which variable changes.	_____
4. Write Q and describe the gates needed.	_____

Guided check

The pair is on the A=1 row across BC=01 and BC=11. A=1 and C=1 stay constant; B changes. Therefore Q = A ∧ C, so the circuit needs one AND gate.

Stretch: spotting overlap

For this two-variable map, draw two groups: row A=0 contains 1,1; column B=1 contains 1,1. Explain why the top-right 1 belongs to both groups.

Overlap practice / two inputs

Gray-code column order; row and column headings specify the inputs.

	B=0	B=1
A=0	1	1
A=1	0	1

Answer: Q = ¬A ∨ B. The shared 1 belongs to both groups; each group simplifies to one literal.

05 / EXAM PRACTICE - independent practice

Attempt the tasks before using the answers. Draw your groups directly on the maps and show the unchanged inputs.

FOUNDATION / QUESTION 1

Simplify Q and draw its gate plan

Gray-code column order; row and column headings specify the inputs.

	00	01	11	10
A=0	1	1	0	0
A=1	0	0	0	0

Which variables stay constant? _____

Write Q: _____ Gate(s): _____

CORE / QUESTION 2

Simplify Q (overlapping groups are permitted)

Gray-code column order; row and column headings specify the inputs.

	B=0	B=1
A=0	1	1
A=1	1	0

Write Q: _____

CHALLENGE: On the next page you will simplify a four-variable map with edge wraparound.

06 / EXAM PRACTICE - four-input challenge

The top-right 2 × 2 block is one group. The bottom-left and bottom-right columns make another group across the map edges. Identify the two simplified terms.

Four-input K-map / columns CD

Gray-code column order; row and column headings specify the inputs.

	00	01	11	10
AB=00	0	0	1	1
AB=01	0	0	1	1
AB=11	1	0	0	1
AB=10	1	0	0	1

Circle two groups of four. Use wraparound for the bottom outer columns.

Top-right term: _____

Bottom edge group term: _____

Final expression P: _____

Extension: Draw the NOT gates, two AND gates and one OR gate needed for your expression.

Karnaugh maps · independent answers (use the original maps)

Q1 [4 marks] Foundation: simplify the first three-variable K-map, explaining which inputs remain constant. Include the Boolean expression and a gate plan.

DIAGRAM / EXPLANATION

Q2 [4 marks] Core: simplify the second K-map. Explain any overlapping groups and give the gate-level expression.

DIAGRAM / EXPLANATION

Karnaugh maps · independent answers (use the original maps)

Q3 [5 marks] Four-variable challenge: use edge-wrap groups. State the two terms and the final simplified expression. Include the gate plan.

DIAGRAM / EXPLANATION

Karnaugh maps · original exam-style application

Q1 [2 marks] Explain why cells at opposite horizontal edges of a Karnaugh map can be grouped.

DIAGRAM / EXPLANATION

Q2 [3 marks] A group of four has A and B changing, while C = 0 throughout. Write the simplified Boolean term and explain.

Q3 [4 marks] The output is Q = (¬A ∧ C) ∨ (A ∧ B). Describe a gate-level implementation.

CODE / PSEUDOCODE

Karnaugh maps · original exam-style application

Q4 [4 marks] Simplify the four-input map on PDF page 231 Show both **terms and the final expression.**

Karnaugh maps / Check your answers

Use these mark points after completing the independent questions.

Q1 / 2 MARKS

[1] Karnaugh-map columns use Gray-code ordering.

[2] The outer columns differ in one input bit and are therefore neighbours.

Q2 / 3 MARKS

[1] The resulting term is NOT C.

[2] A and B change inside the group, so they are omitted.

[3] C remains 0, so retain its negation.

Q3 / 4 MARKS

[1] Invert A with a NOT gate.

[2] Connect NOT A and C through an AND gate.

[3] Connect A and B through a second AND gate.

[4] OR the two AND outputs to form Q.

Q4 / 4 MARKS

[1] Top-right group simplifies to NOT A AND C.

[2] The lower outer wraparound group simplifies to A AND NOT D.

[3] Identify the unchanged input values in each group.

[4] OR the two terms for the complete expression.

Boolean algebra: justify each rewriting step

Theory / Worked Examples

A correct result is not enough. In extended questions, write which law allows each step and check your simplification with a truth table.

Change form without changing truth

NOT(A AND B) NOT A OR NOT B	A AND (B OR C) AB OR AC	NOT(NOT A) A

Use a truth table to verify each identity for every input combination.

REMEMBER THIS

Five legal moves on a puzzle. De Morgan, distribution, association, commutation and double negation let you rewrite an expression without changing its truth.

Now be precise

- **De Morgan:** NOT(A AND B) = (NOT A) OR (NOT B); NOT(A OR B) = (NOT A) AND (NOT B).
- **Distribution:** A AND (B OR C) = (A AND B) OR (A AND C); A OR (B AND C) = (A OR B) AND (A OR C).
- **Association:** group the same operator differently; **commutation:** swap operands; **double negation:** NOT(NOT A) = A. Together with complement rules they support simplification.

WORKED EXAMPLE 1

Simplify NOT((A OR B) AND C).

Method: De Morgan (outer): NOT(A OR B) OR NOT C. De Morgan (inner): (NOT A AND NOT B) OR NOT C.

WORKED EXAMPLE 2

Expand A AND (B OR NOT C).

Method: Distribution: (A AND B) OR (A AND NOT C). One AND factor becomes two terms.

COMMON MISTAKE

When applying De Morgan’s law, swap AND/OR as well as negating each term.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 2 marks. Rewrite NOT(A OR B) using De Morgan’s law.

Q2 · 4 marks. Simplify NOT(NOT A) OR (B AND A), naming two laws used.

Q3 · 5 marks. Show NOT((A AND B) OR C) in a form without a NOT outside brackets.

Marking points

Check the creditworthy points after completing the independent questions.

Q1 / 2 marks

1. Negate the individual inputs: NOT A and NOT B.
2. Change OR to AND: (NOT A) AND (NOT B).

Q2 / 4 marks

1. Remove the double negation to obtain A OR (B AND A).
2. Name the double-negation law.
3. Simplify A OR (B AND A) to A.
4. Name the absorption law.

Q3 / 5 marks

1. Use De Morgan on the outer brackets: the original OR becomes AND.
2. The C term is negated, giving a final NOT C term.
3. Use De Morgan on the inner brackets: A AND B becomes an OR.
4. Negate both inputs within the inner brackets: NOT A and NOT B.
5. Give the complete grouped result: (NOT A OR NOT B) AND NOT C.

REMEMBER

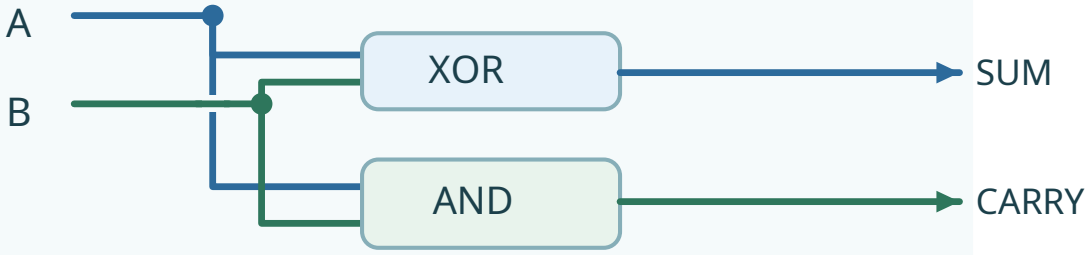
De Morgan changes both the operator and the negation placement. Always preserve grouping and check a truth-table case if unsure.

1.4.3 / Flip-flops and binary adders

State storage, sum and carry

REMEMBER THIS A flip-flop is a one-bit snapshot; an adder is a tiny arithmetic workshop.

HALF ADDER: ONE SOURCE FOR A AND B



SUM = A XOR B

CARRY = A AND B

NOW BE PRECISE D flip-flops are edge-triggered in these examples; a half adder has no carry-in, a full adder does.

KNOW IT PRECISELY

D-type flip-flop. At the selected active clock edge, Q becomes the value of D; Q holds between active edges.

Half adder. S = A XOR B; Cout = A AND B.

Full adder. S = A XOR B XOR Cin; Cout = (A AND B) OR ((A XOR B) AND Cin).

D FLIP-FLOP: SAMPLE D ON THE RISING EDGE

RISING EDGE 1	RISING EDGE 2	RISING EDGE 3
D = 0 Q = 0	D = 1 Q = 1	D = 0 Q = 0

Between clock edges: Q holds its previous value.

Flip-flops and binary adders / Worked method

Worked examples and a different guided problem.

UNDERSTAND IT

- Read the half-adder truth table: 0+0 → S0,C0; 0+1 or 1+0 → S1,C0; 1+1 → S0,C1.
- Full-adder example: A=1, B=0, Cin=1 gives S=0 and Cout=1, because the three inputs total 2 (10_2).
- All coloured wire branches in the circuit originate from ONE A input and ONE B input; junction dots indicate shared signals.

WORKED EXAMPLE 1 / HALF ADDER

A=1 and B=1. XOR produces S=0; AND produces Cout=1. Together the result is 10_2.

WORKED EXAMPLE 1 / D-TYPE

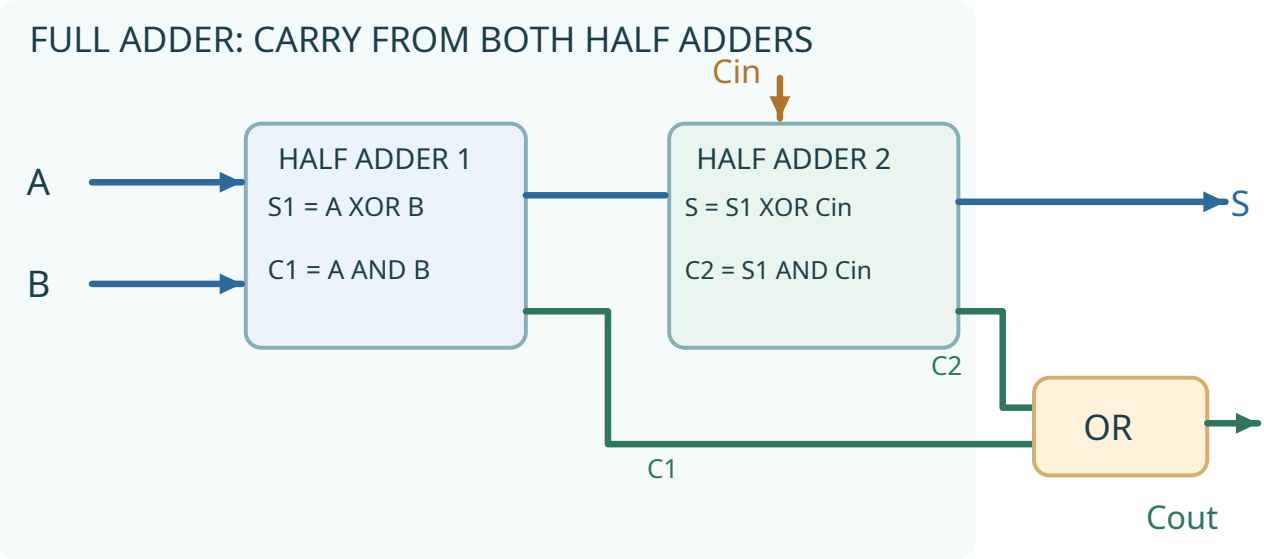
Q initially stores 1. D at successive rising edges is 0,1,0. Q after each edge is 0,1,0; changes to D between edges do not change Q.

WORKED EXAMPLE 2 / FULL ADDER

For A=1, B=1, Cin=1: S1=A XOR B=0; sum S=S1 XOR Cin=1; carry = (1 AND 1) OR (0 AND 1)=1. Result is 11_2.

Complete adders and D-type truth tables

Trace the mechanism carefully; these examples are not questions to memorise.



FULL-ADDER TRUTH TABLE

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

D-TYPE CHECK At a rising clock edge, Q(next) = D. Between active edges, Q keeps its previous value.

Flip-flops and binary adders / Exam practice

Complete the questions independently. Use the space provided for your working.

Q1 4 MARKS

For a half adder, find S and Cout for input pairs (0,1) and (1,1), stating the gates used.

Q2 5 MARKS

A rising-edge D flip-flop initially has Q=1. At three rising edges D equals 0, 1, 0. Complete the Q sequence and explain what happens between edges.

Flip-flops and binary adders / Exam practice

Complete the questions independently. Use the space provided for your working.

Q3 6 MARKS

For a full adder with A=1, B=0, Cin=1, calculate S1, S, Cout and the two-bit result. Explain why a half adder alone is insufficient.

Flip-flops and binary adders / Check your answers

Use these mark points after completing the independent questions.

Q1 / 4 MARKS

[1] (0,1): XOR gives S=1.

[2] (0,1): AND gives Cout=0.

[3] (1,1): XOR gives S=0.

[4] (1,1): AND gives Cout=1.

Q2 / 5 MARKS

[1] At first rising edge Q becomes 0.

[2] At second rising edge Q becomes 1.

[3] At third rising edge Q becomes 0.

[4] Q samples D on the rising edge.

[5] Between edges Q retains its previous stored state.

Q3 / 6 MARKS

[1] S1 = A XOR B = 1.

[2] S = S1 XOR Cin = 0.

[3] A AND B = 0.

[4] S1 AND Cin = 1, so Cout = 1.

[5] Two-bit result is 10_2 (decimal 2).

[6] A half adder has no Cin input; a full adder must include it.

1.5 Legal, moral, social and ethical issues

Remember: law asks “is it permitted?”; ethics asks “is it justified?” Both questions matter when computing systems affect people.

Legislation and ethical judgement answer different questions

LEGAL QUESTION	ETHICAL QUESTION
What does the law allow or require?	Is this fair, justified and proportionate?

An action may be lawful yet raise serious ethical concerns.

SPECIFICATION AND CURRENT LAW

OCR H446 explicitly lists the Data Protection Act 1998 (historical syllabus wording), Computer Misuse Act 1990, Copyright, Designs and Patents Act 1988, and Regulation of Investigatory Powers Act 2000. When discussing UK data protection today, refer to UK GDPR and the Data Protection Act 2018, as amended. Do not describe the 1998 Act as the current framework.

WORKED EXAMPLE 1 / Unauthorised access

A person uses another employee’s login credentials without permission to access private files. The central legal issue is unauthorised access under the Computer Misuse Act; copying personal information or protected work may raise additional legal questions.

WORKED EXAMPLE 2 / Copyright and copying

A student copies commercial software and distributes it. Explain why this may breach copyright legislation and whether the fact that it is easy to copy makes it ethically acceptable.

Guided response

Copyright protection can apply to computer software. Redistribution without authorisation may infringe rights; ease of copying does not establish permission. Consider the impact on rights holders as well as fair access and legal licensing.

Theory / Key terminology

Use these precise terms when explaining concepts and constructing examination answers.

UK GDPR	Rules regulating the processing of personal data in the UK.
Data Protection Act 2018	UK legislation supplementing data protection law.
Computer Misuse Act 1990	UK legislation concerning unauthorised access and computer misuse.
Copyright	Protection for original works, including software, subject to legal exceptions.
Ethical issue	Question of what is right or fair even if an action is lawful.
Algorithmic bias	Systematic unfairness in algorithmic outcomes.
Digital divide	Differences in access to technology, connectivity or digital skills.

Build an argument about AI monitoring

MATCHING LESSON 296 / POINT → EXPLANATION → CONSEQUENCE → COUNTERARGUMENT → JUDGEMENT

The shared scenario. A fictional school proposes AI monitoring of activity on school-managed devices. It would flag potentially concerning activity for trained safeguarding staff. Staff check context before acting; there are no automatic sanctions. **Should the school introduce it?**

WORKED EXAMPLE 1 / Watch each sentence develop the argument

POINT · Who is affected?

Pupils could benefit from earlier safeguarding support.

Names a stakeholder and a specific benefit.

EXPLANATION · Why?

The system could flag concerning activity sooner than staff manually reviewing the same activity.

Gives a mechanism; the benefit still needs evidence.

CONSEQUENCE · What follows?

If trained staff check those alerts promptly, they could offer help before a concern becomes more serious.

Develops the impact and identifies a condition.

COUNTERARGUMENT · What weakens it?

However, legitimate research could trigger false alerts, distressing pupils and diverting staff from genuine concerns.

Challenges the benefit with a competing consequence.

JUDGEMENT · Why this decision?

On balance, I would support a limited, reviewed trial only if staff can check alerts promptly, collection is restricted and pupils can challenge mistakes; otherwise, I would delay introduction.

Weighs the effects and states conditions. Review does not guarantee fairness.

Knowledge to use, not just name. A prediction is not proof: data can be incomplete or biased. Check context, accuracy and effects on pupils, families, staff and leaders. In recruitment, proxy variables, unrepresentative training data or unsuitable thresholds can disadvantage applicants with career breaks; audit outcomes and provide meaningful review.

Accessibility. A red/green-only school chart can exclude users with colour-vision differences. Add labels or symbols, sufficient contrast, readable text, keyboard access, language support and suitable character sets.

Wider issues. Employment and skills; energy, manufacturing and e-waste; surveillance and personal information; censorship, offensive communication and piracy; cultural expectations about layout, colour and language. Use these prompts flexibly, with scenario evidence; lawful does not automatically mean ethically justified.

Exam Practice

Q1 · 4 marks Identify two laws relevant to unauthorised computer access and software copying, and explain which situation each law addresses.

Q2 · 4 marks Explain why a school should not automatically penalise a pupil based solely on an AI-generated risk score.

Q3 · 6 marks Discuss three ways to make a school progress dashboard accessible to a wide range of users.

Q4 · 6 marks A council proposes automated surveillance cameras in a public car park. Explain two possible benefits and two legal or ethical concerns.

Develop, then practise independently

PRACTICE A / Complete the argument

Use the original monitoring proposal from page 248: trained staff review alerts; there are no automatic sanctions. Complete the three missing parts before viewing page 252. This scaffold task is unmarked.

Point: Pupils could benefit from earlier safeguarding support.
Explanation: The system could flag concerning activity sooner than manual review.
Consequence: If staff check promptly, ...
Counterargument: However, ...
Judgement: On balance, I would ... because ...; this depends on ...

Q5 · 9 marks / Extended evaluation

Evaluate whether a school should use an automated system to recommend pupils for extra support. Consider accuracy, privacy, fairness and different stakeholders before reaching a conclusion.

Write a developed response on separate paper. Weigh effects rather than count advantages and disadvantages. The guide’s Q5 self-marking bands remain on page 252.

PRACTICE B / Transfer without sentence starters

An employer proposes AI shortlisting trained on past hiring decisions. It would reject applicants with career breaks without review. Evaluate the proposal in a short paragraph, considering applicants and recruitment staff. Use your own structure. This additional task is unmarked.

Before checking: have you explained a mechanism, developed a consequence, weighed a challenge and justified your decision? A list of issues is not enough.

Check your reasoning

Complete the questions before reading this page. Compare your reasoning with the guidance and models below.

Q1 / 4 marks

Computer Misuse Act 1990 applies to unauthorised computer access [1]; explain deliberate access without permission [1]. Copyright, Designs and Patents Act 1988 protects copyright works [1]; unauthorised software copying/distribution may infringe those rights [1].

Q2 / 4 marks

Automated scores may be based on inaccurate or biased data [1]; a score is not conclusive evidence about a pupil [1]; human review checks accuracy and context [1]; decisions must allow a proportionate response, correction or challenge [1].

Q3 / 6 marks

Three suitably developed changes, up to 2 marks each: readable text and sufficient contrast; more than colour alone (labels/symbols); keyboard access and accessible names, or suitable equivalent. Award 1 mark for a valid measure and 1 for a relevant explanation of whom/how it helps.

Q4 / 6 marks

Two explained benefits, 1 mark each: detecting incidents and deterrence or faster response. Two explained concerns, up to 2 marks each: lawful, secure and proportionate handling of personal data; intrusive or inaccurate monitoring with unfair effects. Accept equivalent case-linked reasoning.

Q5 / 9 marks - extended evaluation

7-9: Sustained, accurate and balanced evaluation applied to the school, weighing accuracy, fairness, privacy and stakeholder effects, ending with a justified judgement and practical safeguards.

4-6: Some relevant and developed benefits/risks with limited comparison or application; a conclusion may be present but not well justified.

1-3: Basic or descriptive points with little analysis, application or balance. Award zero where there is no relevant material.

Indicative content: Early support, human review, input-data quality, bias, transparent explanations, lawful use of personal data, security and pupils' ability to challenge errors. Do not require all points for the top band.

These are booklet self-marking guidelines, not an official OCR mark scheme. Credit equivalent technically accurate responses.

Practice A / One possible completion

If trained staff check promptly, pupils could receive help before a concern escalates. However, a false alert could distress a pupil and divert staff from a genuine concern. I would support a limited, reviewed trial because earlier support may help pupils, but only with restricted collection, prompt checks and a way to challenge mistakes; otherwise I would delay it.

Practice B / One possible developed answer

Automated shortlisting could help recruiters process applications faster, leaving time for interviews. However, past decisions may contain bias, and a career break does not establish a lack of skill. Rejecting these applicants without review could exclude capable people unfairly. I would not approve this design: speed does not justify unexplained exclusion. Assess job-relevant evidence, audit outcomes and offer meaningful human review before relying on it.

These are examples, not scripts to memorise. Credit a different judgement supported by accurate, scenario-linked reasoning. The five prompts are not five automatic marks; the question and its marking guidance determine credit.

Technology, law, culture and society

Theory / Worked Examples

A-level discussion is not a list of advantages and disadvantages. State a stakeholder, explain a consequence and evaluate a proportionate response.

REMEMBER THIS

Point → explanation → consequence → counterargument → justified judgement.
Use scenario evidence and precise law where relevant. These are prompts, not a fixed formula.

Now be precise

- **RIPA 2000:** addresses certain interception and investigatory powers under conditions and safeguards; distinguish authorised statutory activity from arbitrary access. UK investigatory law has since evolved.

- **Employment:** automation can displace tasks and create new work; impacts vary by sector, training and access. **Environment:** electricity, manufacturing emissions and e-waste must be balanced against potential efficiency gains.

- **Censorship and digital communication:** analyse safeguarding and harm reduction alongside free expression and proportionality. **Piracy:** unauthorised reproduction/distribution raises copyright issues; offensive communication may engage other laws depending on content and context.

- **Culture/accessibility:** language, writing direction, character sets, icons and colour meanings affect inclusivity; use Unicode, culturally appropriate formats and labels instead of colour alone.

WORKED EXAMPLE 1

A company replaces repetitive entry work with software. Explain two possible consequences.

Method: Some clerical tasks may be removed; staff may be retrained for data-quality, support or development tasks. Evaluate training cost, transition support and unequal access.

WORKED EXAMPLE 2

A social platform proposes removing every post containing a disputed word. Evaluate.

Method: It may reduce some harmful content but also suppress legitimate context, satire and discussion. A proportionate policy uses moderation, appeals and evidence.

COMMON MISTAKE

Differentiate the issue, the law and the ethical judgment. Disagreeable content is not automatically unlawful.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 4 marks. Explain one environmental cost and one potential environmental benefit of moving a service online.

Q2 · 4 marks. A school app shows errors only in red and uses culturally unfamiliar icons. Recommend two changes and justify them.

Exam practice · continued

Q3 · 6 marks. Evaluate an employer using automated monitoring and filtering of all staff communication.

Q4 · 4 marks. Explain why unauthorised copying and distributing a film differs from expressing a controversial opinion online.

Q5 · 3 marks. Describe the general purpose of RIPA 2000 and give one safeguard concern when using investigatory powers.

Mark scheme

Compare only after completing the questions.

Q1 · 4 marks

Manufacturing/energy or e-waste cost identified and linked to impact [2]; a reduction in physical travel/printing or improved resource use explained [2].

Q2 · 4 marks

Add written status label/symbol [2]; use tested familiar icon and language/accessible character encoding [2].

Q3 · 6 marks

Potential security/compliance rationale [1]; staff privacy/fairness [1]; clear lawful basis and transparency [1]; proportionality/data minimisation [1]; human oversight/appeal [1]; balanced conclusion grounded in context [1].

Q4 · 4 marks

Copyright/piracy relates to reproduction/distribution without permission [2]; expression raises content/context and possible harmful communications rules rather than automatically being infringement [2].

Q5 · 3 marks

Legal framework for defined investigatory/interception powers [2]; authorised, necessary and proportionate use/oversight concern [1].

RECALL

Point → explanation → consequence → counterargument → justified judgement.
Use scenario evidence and precise law where relevant. These are prompts, not a fixed formula.

H446 MEMORY BANK

Read the hook, then cover the precise wording and try to explain the concept without the analogy.

Processor / 1.1.1 / CPU as head chef

NOW BE PRECISE

PC = next instruction address; MAR = memory address; MDR = data/instruction; CIR = fetched instruction.

Careful: A chef cannot model processor pipelining or precise register transfers.

Binary / 1.4.1 / Two’s complement = invert + one

NOW BE PRECISE

For a fixed width: flip all bits, then add 1 to form a negative value.

Careful: Check representable range and overflow; this is not sign-magnitude.

Data structures / 1.4.2 / Plates & queues

NOW BE PRECISE

Stack = LIFO; queue = FIFO.

Careful: An abstract data type defines operations independent of implementation.

Boolean / 1.4.3 / K-map = joined-edge map

NOW BE PRECISE

Opposite edges are adjacent. Circle power-of-two groups of 1s.

Careful: Do not group diagonals; draw only one source for each logic input.

MEMORY BANK / continued

Operating systems / 1.2.1 / Airport controller

NOW BE PRECISE

Scheduler allocates CPU time; ready processes wait for service.

Careful: Scheduling is a defined algorithm, not intuition.

Applications / 1.2.2 / Translator + linker

NOW BE PRECISE

Lexing → parsing → code generation → linking/loading.

Careful: Implementations vary; compilation does not guarantee correctness.

Data exchange / 1.3.1 / ZIP, lock, fingerprint

NOW BE PRECISE

Compression reduces representation; encryption protects confidentiality; hashing produces a digest.

Careful: Hashes have collisions and are not decrypted.

Databases / 1.3.2 / Library card number

NOW BE PRECISE

Unique primary key identifies one row; foreign key links related rows.

Careful: Human names may repeat; FK values can repeat.

REVISION ROUTINE

1. Recall the hook without looking. 2. Explain in precise H446 language. 3. Trace or draw an example. 4. Attempt a fresh exam question.

MEMORY BANK

Cover the precise meaning and explain each memory hook aloud. The analogy helps you remember; the technical explanation earns the marks.

WATERFALL	Staircase
EXAM PRECISION	Distinct phases with defined outputs; late change can mean rework.
AGILE	Short laps
EXAM PRECISION	Iterative increments incorporate frequent stakeholder feedback.
XP	Engineering pit crew
EXAM PRECISION	An Agile approach emphasising testing, pair work and integration.
SPIRAL	Risk radar
EXAM PRECISION	Revisit risk identification and evaluation in each cycle.
RAD	Prototype workshop
EXAM PRECISION	Rapid prototypes help users refine unclear requirements.
LAW / ETHICS	Permitted / justified
EXAM PRECISION	Legislation establishes duties and rights; ethics weighs fairness and impact.
AI DECISIONS	Human check
EXAM PRECISION	Model outputs require reliable data, oversight and explainability.

Memory bank · additional recall

Cover the precise wording, recall the memory hook, then explain the computer-science meaning aloud.

Recipe cards	Procedural Procedures/functions organise operations; data is supplied as arguments or shared state.
Vending machine	OOP Objects combine attributes and methods. Encapsulation regulates state changes.
Numbered mailboxes	LMC ACC is working storage; STA/LDA transfer data to and from addressed mailboxes.
Follow the pointer	Indirect address First access an address held in memory, then use that address for the operand.
Offset on a map	Indexed address Calculate effective address from base plus index or displacement.

QUICK RECALL / PAPER 1

Memory bank — data and network models

C-S-M-E

Capture the input; **Select** the records; **Manage** the stored data; **Exchange** it with another system.

Keyword	Remember it precisely
Validation	Check an input against rules or constraints.
Verification	Check captured data against a source.
Foreign key	Field referencing a key in a related table.
CSV	Row-and-column exchange format.
JSON	Structured objects/arrays, including nested data.
Client-server	Managed services supplied to clients.
Peer-to-peer	Peers may be both resource clients and providers.
Topology	Physical/logical connection layout; not the service model.

TWO EXAM TRAPS

SQL SELECT does not change data. **Client-server** does not imply exactly one server, and **peer-to-peer** does not require a physical mesh.

Five one-minute retrieval prompts

- Give one appropriate data-capture method for a barcode-labelled library book.
- Explain validation and verification using the same captured identifier.
- Explain when JSON might be preferable to CSV.
- Give one benefit and one cost of central network management.
- Explain why a star network may support either client-server or peer-to-peer.

Quick recall / memory management and databases

Cover the precise explanation, recall the memory image, then explain it in Computer Science vocabulary.

Remember	H446 concept	Exam-accurate meaning
Hotel room key	Paging	Page table maps fixed-size virtual pages to RAM frames.
Moving boxes	Virtual memory	Absent pages can be retrieved from backing storage.
Fire alarm	Interrupt / ISR	Save context, service the event, restore and resume.
Library card	Referential integrity	Foreign keys reference existing parent keys.
One fact, one home	3NF	Remove partial and transitive dependencies.
Find the rows	SQL JOIN / WHERE	JOIN matches keys; WHERE filters.
All or nothing	Atomicity	Commit all dependent changes or roll back.
One ticket left	Isolation / locking	Control concurrent work to avoid conflicting updates.

A 30-second check

Can you explain page vs frame, interrupt vs ISR, primary vs foreign key, and DELETE vs DROP TABLE without looking at your notes?

Memory bank: systems and applications

Cover the precise definition. Recall the hook, then explain the real Computer Science.

Memory hook	Specification term	Use exam language
Airport control desk	OS purpose	Allocate CPU, memory, files, I/O and secure access.
Five questions	OS types	Machine / device / tasks / users / deadline.
Opening the building	BIOS / UEFI	Initialise platform and begin boot sequence.
Device translator	Driver	OS-specific request mapped to device operations.
Machine made of software	Virtual machine	Guest OS on virtual hardware OR bytecode runtime.
User’s job	Application	Choose functions that meet a concrete user need.
Backstage crew	Utility	Maintain, protect, compress or recover system data.
Join, then start	Linker / loader	Resolve modules, then prepare executable to run.

Five 30-second retrieval prompts

- Why is a real-time system not merely a fast system?
- What separate roles do UEFI firmware and the boot loader play?
- Why can a missing driver stop a connected printer working?
- How does a guest OS VM differ from a bytecode VM?
- What is the distinction between linking and loading?